

目录

第一部分 知识准备

第一章 开源飞控.....	5
1.1、开源飞控的发展.....	5
1.2、开源飞控的未来.....	7
1.3、微处理器的发展.....	10
1.4、常见的开源飞控介绍.....	11
第二章 系统配套飞控详细介绍.....	15
2.1 硬件介绍	15
2.2、设置飞行模式和试飞.....	28
2.3、PID 参数的调整	29
2.4 软件开发及环境介绍.....	32
第三章 PIXHAWK 程序解读	36
3.1、pixhawk 程序框架	36
3.2、pixhawk 基本组成模块及功能.....	36
3.3、pixhawk 模块间消息传递	38
3.4、Pixhawk 的软件启动与运行	39
第四章 MEMS 传感器.....	40
4.1、MEMS 传感器的发展	40
4.2、MEMS 传感器的应用	43
4.3、常用的 MEMS 传感器	47

第二部分 E360 开发

电气连接说明	69
第一章 PIXHAWK 整体架构的认识.....	72
1.1、NuttX 实时操作系统	72
1.2、Pixhawk 飞行控制栈	72
1.3、Pixhawk 飞行控制栈	72
1.4、源码框架介绍.....	82
第二章 PIXHAWK 原生码 RCS 分析	86
2.1、代码执行流程.....	86
2.2、代码注解	88
第三章 PIXHAWK 硬件构架	137
3.1、Pixhawk 硬件芯片列表	137
3.2、Pixhawk 传感器列表	138
3.3、pixhawk 硬件构架	139
第四章 PIXHAWK PX4FMU 和 PX4IO 最底层启动过程分析.....	141
4.1、PX4IO 与 PX4FMU	141
4.2、NuttX	143
4.3、APP	147
第五章 PIXHAWK 姿态与控制	150
5.1、姿态解算	150
5.2、姿态控制	159
5.3、角度控制环（control_attitude() 函数）	184
5.4、control_attitude(dt) 返回以后	199
5.5、姿态速度控制（角速度环）	200

5.6、发布控制量.....	208
5.7、总结	210
第六章 飞行模式切换.....	212
6.1、整体流程	212
6.2、遥控器端	213
6.3、飞控端	224
第七章 DEMO 程序讲解.....	252
7.1、Demo 程序功能介绍	252
7.2、Demo 程序通信协议介绍	252
7.3、Demo 程序分析	254

第一部分

知识准备

要点提示： 开源飞控的发展

系统配套飞控介绍

PIXHAWK 程序解读

MEMS 传感器发展与应用



学前提示：

飞控系统是无人机完成起飞、空中飞行、执行任务和返场回收等整个飞行过程的核心系统，飞控对于无人机相当于驾驶员对于有人机的作用，我们认为无人机最核心的技术之一。飞控一般包括传感器、机载计算机和伺服作动设备三大部分，实现的功能主要有无人机姿态稳定和控制、无人机任务设备管理和应急控制三大类。

第一章 开源飞控

1.1、开源飞控的发展

1.1.1、什么是开源飞控

开源项目，顾名思义是指开放式源代码项目。飞控则根据源程序开放程度分为开源飞控和商业（非开源）飞控两类。开源飞控是指源代码公开的，可由玩家自主编程、开发的飞控，可以自己写源码添加一些功能。开源飞控的发展是建立在开源项目的基础上的。

1.1.2、开源飞控的起源

当今的开源飞控全部源于一款非常成功的商业自动驾驶仪：MicroPilot。在2003年，MicroPilot公司设计并发行了该系列的第一块集成飞控，从那年起自动驾驶仪进入高集成时代。该公司最著名的产品是MP2028，该自动驾驶仪全部集成在一块电路板上，包括10DOF传感器、气压高度、空速和GPS接收机，重量为28克。该商业飞控被美军微型无人机大量采用。



图 3.1 MP2028 自动驾驶仪

1.1.3、开源飞控的鼻祖

随后，集成飞控高速发展，传感器的成本也在不断降低，为了得到更好的飞控设计源代码，Arduino 公司决定开放其飞控的源代码。他们建立了便捷灵活、方便上手的开源电子原型平台，包含硬件（各种型号的 Arduino 板）和软件（Arduino IDE）。可以说是他们开启了开源飞控的发展道路。

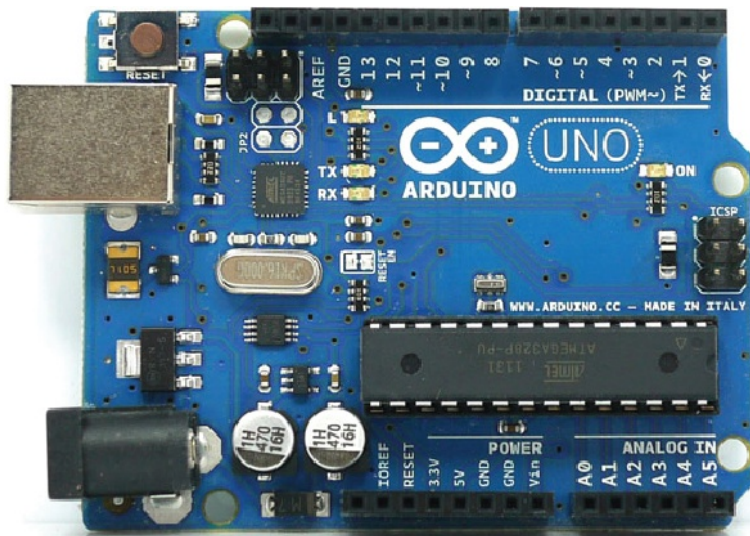


图 3.2 Arduino 开发板

1.1.4、开源飞控的发展阶段

之后开源飞控的发展一共经历了三个阶段：

- 能够让多旋翼无人机能平稳地飞起来，把拥有这种功能的控制系统叫做第一代无人机飞控系统，第一代飞控的主要特点是模块化、可扩展。
- 能够让多旋翼无人机自主悬停、自动定高、按设定航线飞行的控制系统称为第二代无人机飞控系统，第二代飞控的主要特点是高度集成、高可靠。
- 飞控系统的进化远未停步于此，新的应用需求催生了飞控系统的革新。例如：图像识别、自主避障、自动跟踪飞行等功能，将成为第三代无人

机飞控系统的标志，并且会向机器视觉、集群化、开发过程平台化的方向发展。

1.2、开源飞控的未来

1.2.1、机器视觉

机器视觉就是用机器代替人眼来做测量和判断。机器视觉系统是指通过机器视觉产品（即图像摄取装置，分 CMOS 和 CCD 两种）将被摄取目标转换成图像信号，传送给专用的图像处理系统，根据像素分布和亮度、颜色等信息，转变成数字化信号；图像系统对这些信号进行各种运算来抽取目标的特征，进而根据判别的结果来控制现场的设备动作。

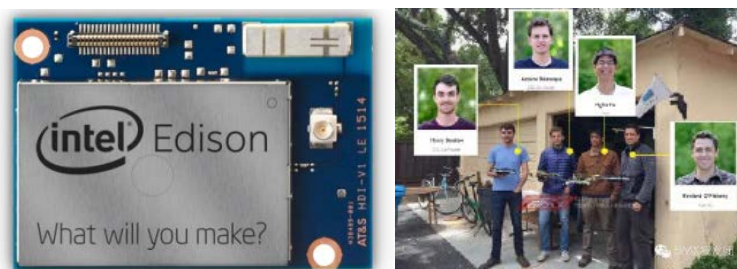


图 3.3 机器视觉

1.2.2、集群技术

集群（cluster）技术是一种较新的技术，通过集群技术，可以在付出较低成本的情况下获得在性能、可靠性、灵活性方面的相对较高的收益，其任务调度则是集群系统中的核心技术。



图 3.4 集群飞行

1.2.3、嵌入式操作系统

但是要实现以上这些功能除了新算法的引入与新传感器的加入外，需要更好的操作系统与运算能力强大的处理器。下面是对操作系统和处理器的一些简要介绍。

嵌入式操作系统（Embedded Operating System，简称：EOS）是指用于嵌入式系统的操作系统。嵌入式操作系统是一种用途广泛的系统软件，通常包括与硬件相关的底层驱动软件、系统内核、设备驱动接口、通信协议、图形界面、标准化浏览器等。嵌入式操作系统负责嵌入式系统的全部软、硬件资源的分配、任务调度，控制、协调并发活动。当系统任务很多，持续、中断管理非常困难时，使用嵌入式操作系统能有效解决这个问题。

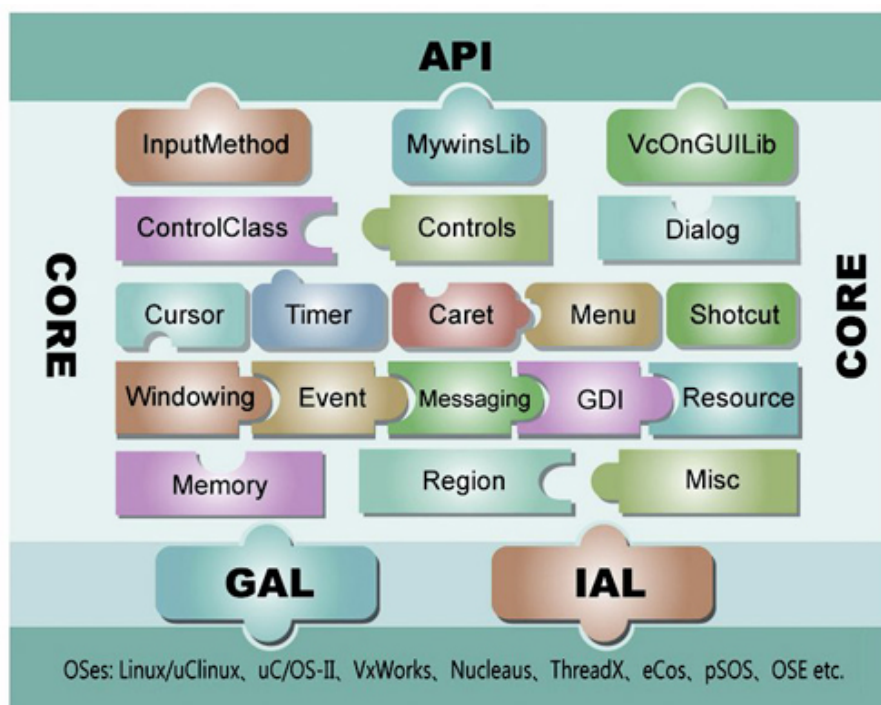


图 3.5 嵌入式系统

嵌入式系统的核心是嵌入式微处理器。嵌入式微处理器一般具备以下 4 个特点：

- 对实时任务有很强的支持能力，能完成多任务并且有较短的中断响应时间，从而使内部的代码和实时内核的执行时间减少到最低限度。
- 具有功能很强的存储区保护功能。这是由于嵌入式系统的软件结构已模块化，而为了避免在软件模块之间出现错误的交叉作用，需要设计强大的存储区保护功能，同时也有利于软件诊断。
- 可扩展的处理器结构，以能最迅速地开发出满足应用的最高性能的嵌入式微处理器。
- 嵌入式微处理器必须功耗很低，尤其是用于便携式的无线及移动的计算和通信设备中靠电池供电的嵌入式系统更是如此，如需要功耗只有 mW 甚至 μW 级。

下面介绍几个常用的嵌入式操作系统：

- FreeRTOS 是一个迷你操作系统内核的小型嵌入式系统。作为一个轻量级的操作系统，功能包括：任务管理、时间管理、信号量、消息队列、内存管理、记录功能等，可基本满足较小系统的需要。
- uC/OS II (Micro Control Operation System Two) 是一个可以基于 ROM 运行的、可裁减的、抢占式、实时多任务内核，具有高度可移植性，特别适合于微处理器和控制器，是和很多商业操作系统性能相当的实时操作系统 (RTOS)。
- NuttX 是一个实时嵌入式操作系统 (RTOS)，它有一个小巧是在微控制器的环境中使用。这是完全可扩展，从小型 (8 位) 至中型嵌入式 (32 位) 系统。它的目的还在于要完全符合标准，完全实时，并完全开放。
- 微软机器人系统这一被称为 Microsoft Robotics Studio 的技术是一种旨在简化机器人编程工作的基于 Windows 操作系统的软件平台。它与诸如 iRobot 的 Roomba 或 LEGO Mindstorms NXT 等硬件相兼容，这一软件能够使 用户命令机器人进行通讯、发送警报以及完成计划等任务。

1.3、微处理器的发展

美国 3DR 公司近期发表声明称将 Intel 开发的 Edison 芯片应用在了无人机上，并展示了在无人机上通过图像识别完成自动跟踪飞行的功能。3DR 称这一功能叫做 3PV Follow Me 自动跟拍。3DR 表示，Edison 芯片可以作为其开源飞控 Pixhawk 的补充处理芯片。通俗地说，有了 Edison 芯片优越的运算能力，3PV 跟随技术可以让我们的摄像头自动追随画面中的人或者物体，而不是仅仅依赖于 GPS 卫星导航信号。因此，无人机可以不必依赖被跟随者身上的任何导航设备，而直接通过画面来识别你的位置和移动，并且保持摄像头始终聚焦在人的身上。

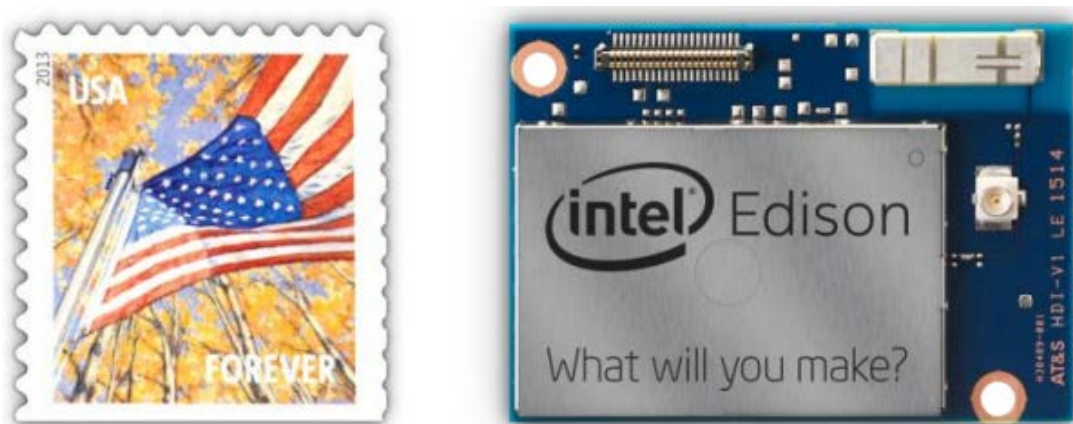


图 3.6 新型芯片

1.4、常见的开源飞控介绍

1.4.1、Arduino

Arduino 是最早的开源飞控，在 2005 年在意大利交互设计学院由 Massimo Banzi, David Cuartielles, Tom Igoe, Gianluca Martino, David Mellis 和 Nicholas Zambetti 合作开发而成。用户可以从 Arduino 官方网站取得硬件的设计档，加以调整电路板及元件，以符合自己实际设计的需求。Arduino 可以通过与其配套的 Arduino IDE 软件查看源代码和烧录自己编写的代码，Arduino IDE 使用的是基于 c 语言和 c++ 的 Arduino 语言，十分容易掌握，并且 Arduino IDE 可以在 Windows、Macintosh OSX、Linux 三大主流操作系统上运行。

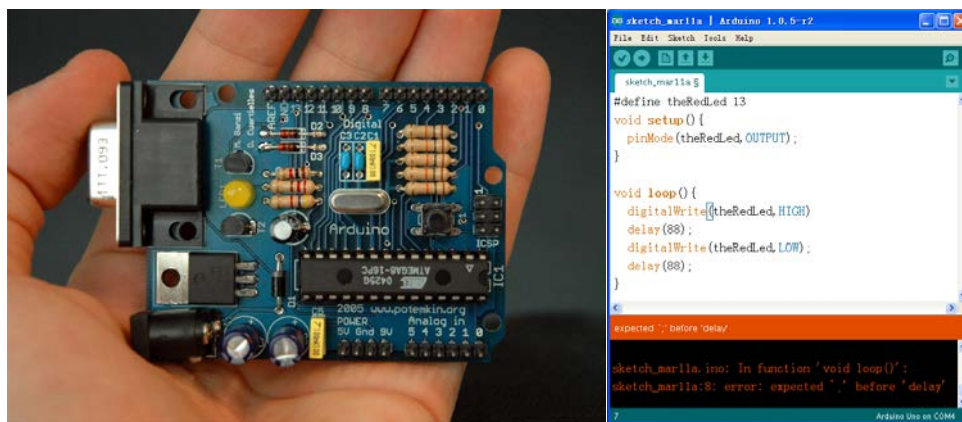


图 3.7 Arduino IDE

1.4.2、 APM

APM (ArduPilotMega) 是在 2007 年由 DIY 无人机社区 (DIY Drones) 推出的飞控产品。APM 基于 Arduino 的开源平台，对多处硬件做出了改进，包括加速度计，陀螺仪和磁力计组合惯性测量单元 (IMU)。由于 APM 良好的可定制性，APM 在全球航模爱好者范围内迅速传播开来。通过开源软件 Mission Planner，开发者可以配置 APM 的设置，接受并显示传感器的数据，使用 google map 完成自动驾驶等许多功能，但是 Mission Planner 仅支持 windows 操作系统。

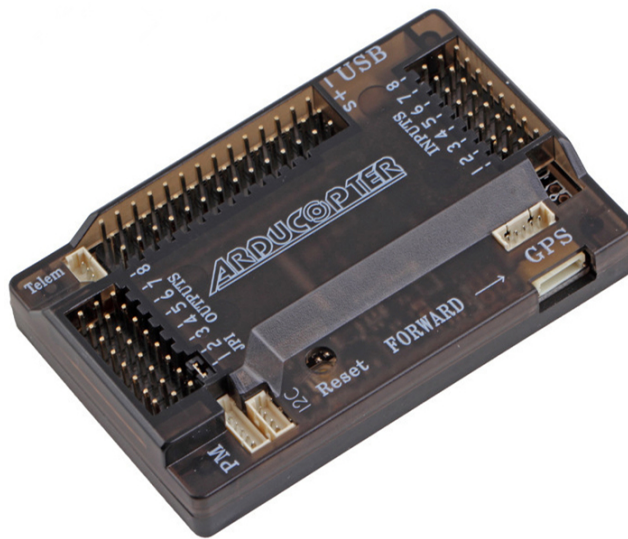


图 3.8 APM

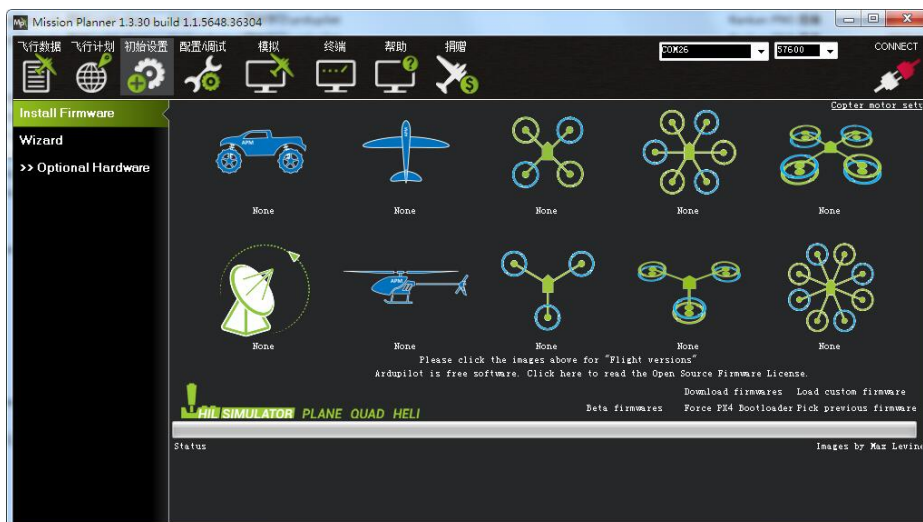


图 3.9 Mission Planner

1.4.3、PX4

PX4 是一个软、硬件开源项目（遵守 BSD 协议），目的在于为学术、爱好和工业团体提供一款低成本高性能的高端自动驾驶仪。这个项目源于苏黎世联邦理工大学的计算机视觉与几何实验室、自主系统实验室和自动控制实验室的 PIXHAWK 项目，PX4FMU 自动驾驶仪模块运行高效的 (RTOS) 实时操作系统，NuttX 提供 POSIX 类型的环境（例如，`printf()`，`pthread`，`/dev/ttyS1`，`open()`，`write()`，`poll()`，`ioctl()`）。软件可以使用 USB bootloader 更新。PX4 通过 MAVLink 同地面站通讯，兼容地面站有 QGroundControl 和 Mission Planner。软件全部开源且遵守 BSD 协议。

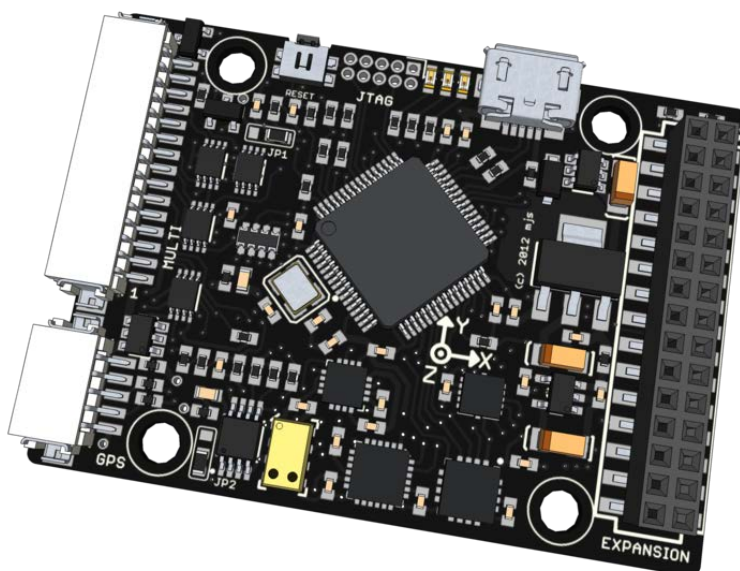


图 3.10 PX4

1.4.4、OpenPilot

OpenPilot 是在 2009 年由 OpenPilot 社区推出的自动驾驶仪项目，旨在为社会提供低成本但功能强大的稳定型自动驾驶仪。这个项目有两方面组成，包括 OpenPilot 自动驾驶仪以及与之相配套的软件，其中自动驾驶仪的固件部分由 c 语言编写，而地面站则用 c++编写，并且可以在 Windows、Macintosh OSX、Linux 三大主流操作系统上运行。

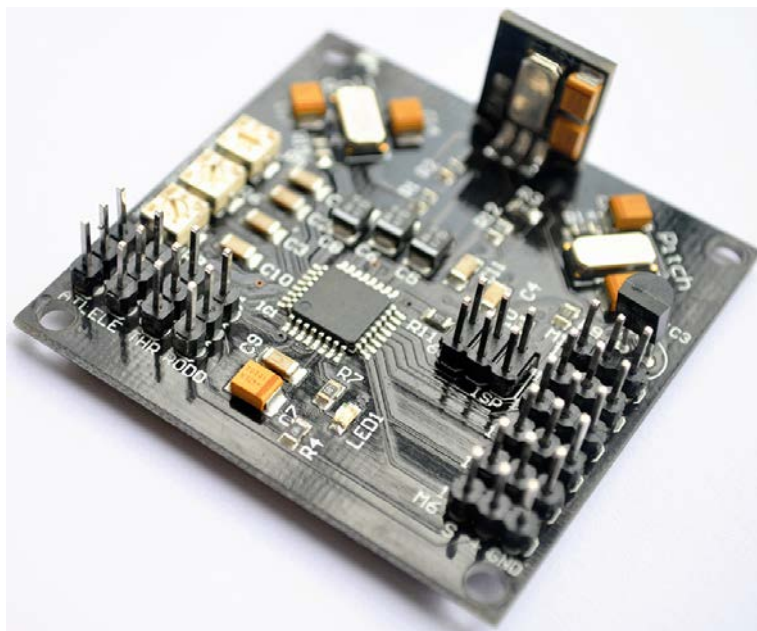


图 3.11 OpenPilot 飞控



图 3.12 OpenPilot 地面站

第二章 系统配套飞控详细介绍

2.1 硬件介绍

2.1.1 嵌入式软件介绍

常见的嵌入式软件有 Nuttx，它是一种实时的嵌入式操作系统（RTOS），可以使用在微控制器的环境中。在 Nuttx 嵌入式系统中，较为常用的是卡尔曼滤波。卡尔曼滤波典型的应用，简单的讲，就是从一组有限的包含噪声的信号序列中预测出被测物体的位置坐标及其速度。跟踪目标时，测量所得目标的位置、速度、加速度的信号往往包含有噪声，卡尔曼滤波则可以去除噪声的影响，得到一个较好的目标位置的估计值。

捷联惯导是利用惯性传感器（陀螺仪、角加速度传感器及线加速度传感器）及其基准位置和初始位置信息来计算获得飞行器的位置、速度及加速度的信息的导航方法。

捷联惯导算法的基本过程为：

- 初始化系统：给定飞行器的初始位置和初速度；校准数学平台；仪表校准。
- 误差补偿
- 姿态矩阵计算。
- 导航计算
- 输出导航信息

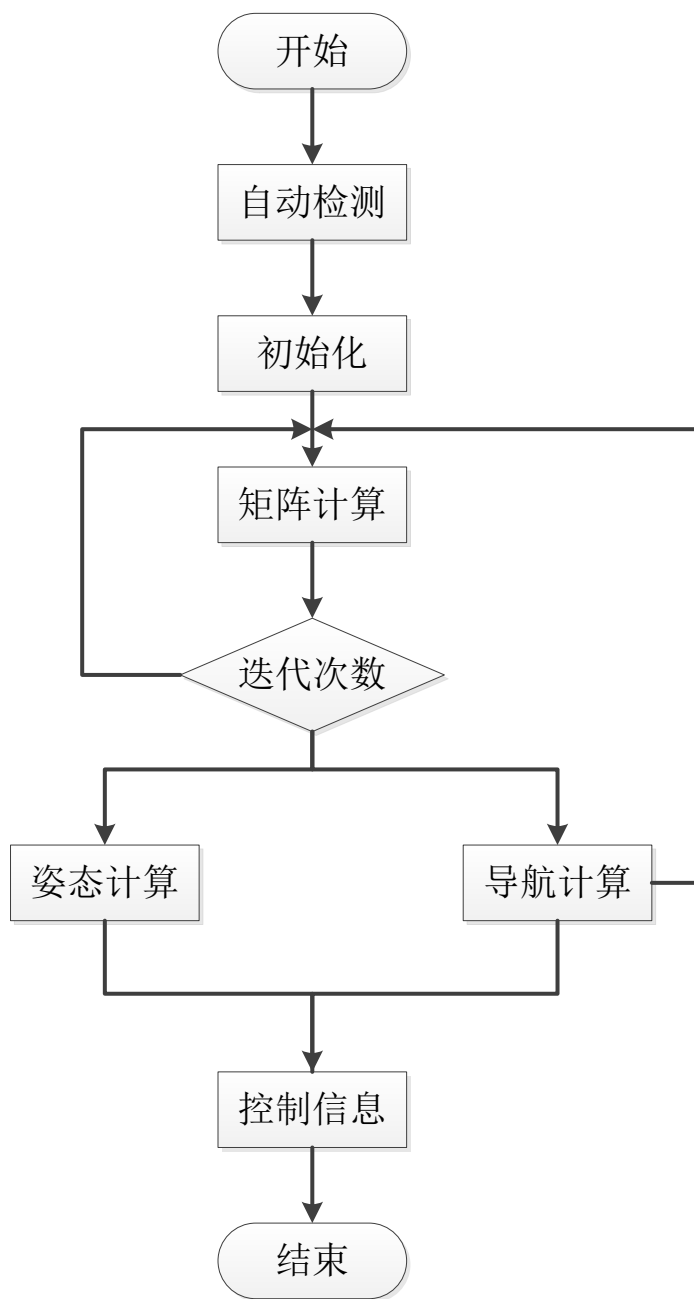


图 4.1 捷联惯导算法基本过程

惯性元件有固定的漂移率，这会给导航造成误差，因此捷联惯导系统还须采用指令、GPS 或其组合等方式定时进行修正，以获取持续准确的位置参数。

2.1.2 安装与调试

安装：飞控应该在多旋翼平面的几何中心，并固定在减震器上；连线：见下图。

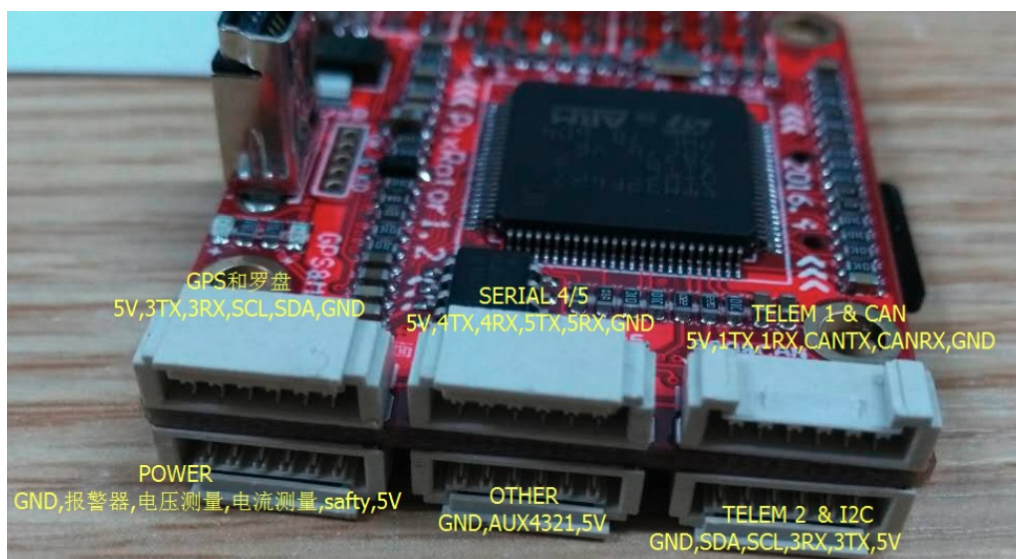


图 4.2 飞控接口



图 4.3 飞控接线标注

- 在安装完飞控之后（安装前也可以），我们就需要开始使用地面站软件，也就是 Mission Planner（下面使用缩写：MP）来对飞控上的很多传感器进行调试和校准。下面我们将详细介绍 MP 软件的使用。
- 将飞控和电脑用数据线连接。在烧录固件完成之前不要点击右上角的连接按钮。

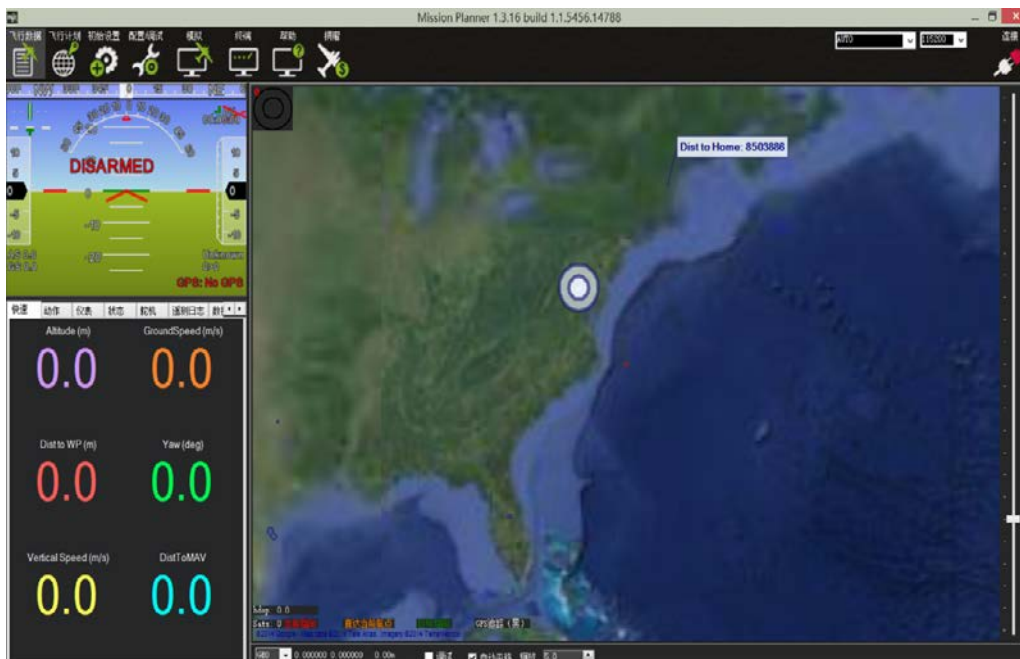


图 4.4 MP 软件界面

- 固件升级：最开始的工作就是往飞控内烧录多旋翼飞行器固件，也就是固件升级。在 MP 的主界面的左上角有一排按钮，我们仅仅使用前四个按钮。
- 点击初始设置，将看到很多图标。

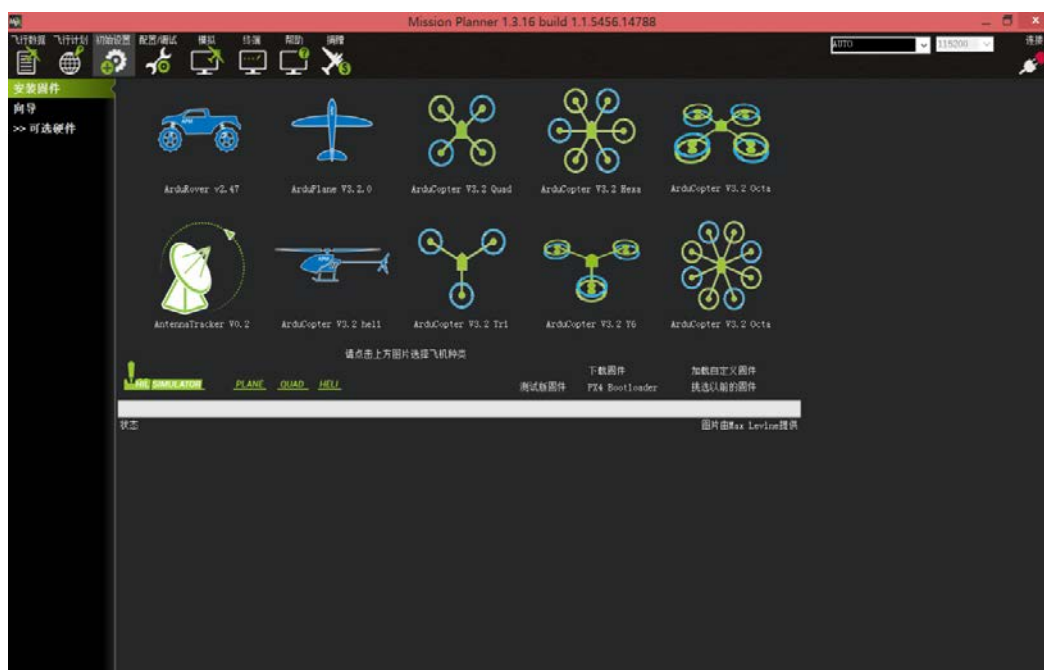


图 4.5 烧写固件

- 选择第三个图标（多旋翼飞行器）

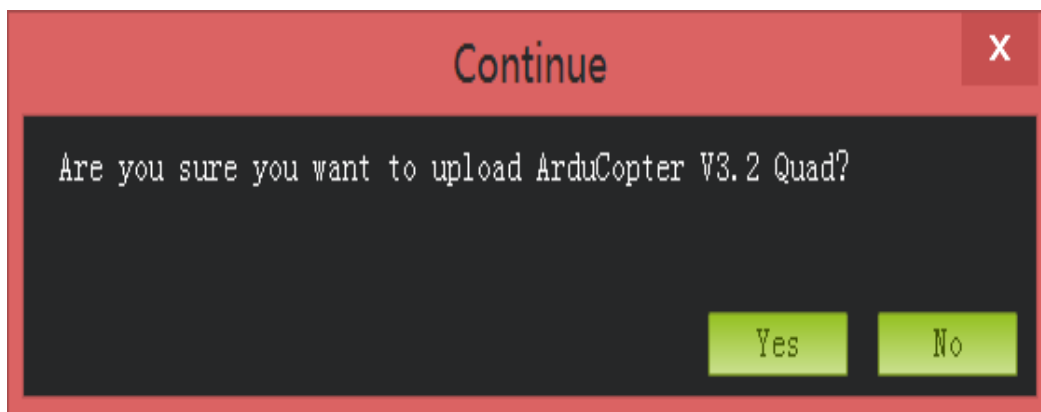


图 4.6 确认刷新固件

- 点击 Yes 开始上传固件。

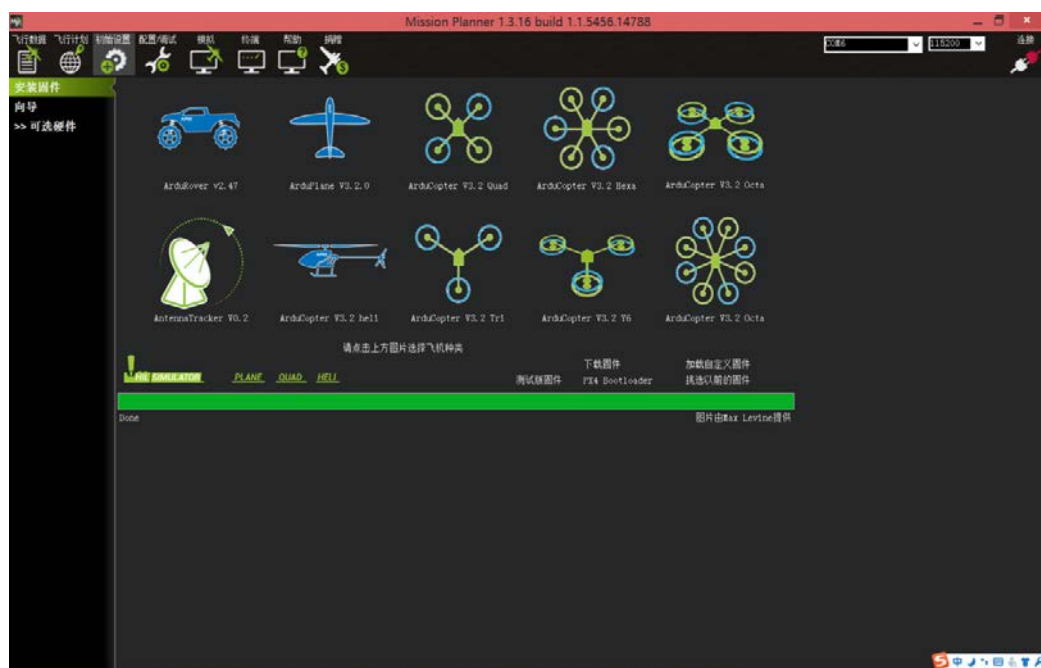


图 4.7 烧写固件中

- 固件烧录完成！

我们第一次使用配套飞控时, 也可以通过向导来烧录固件。点击向导。注意, 此时飞控出于断开状态。

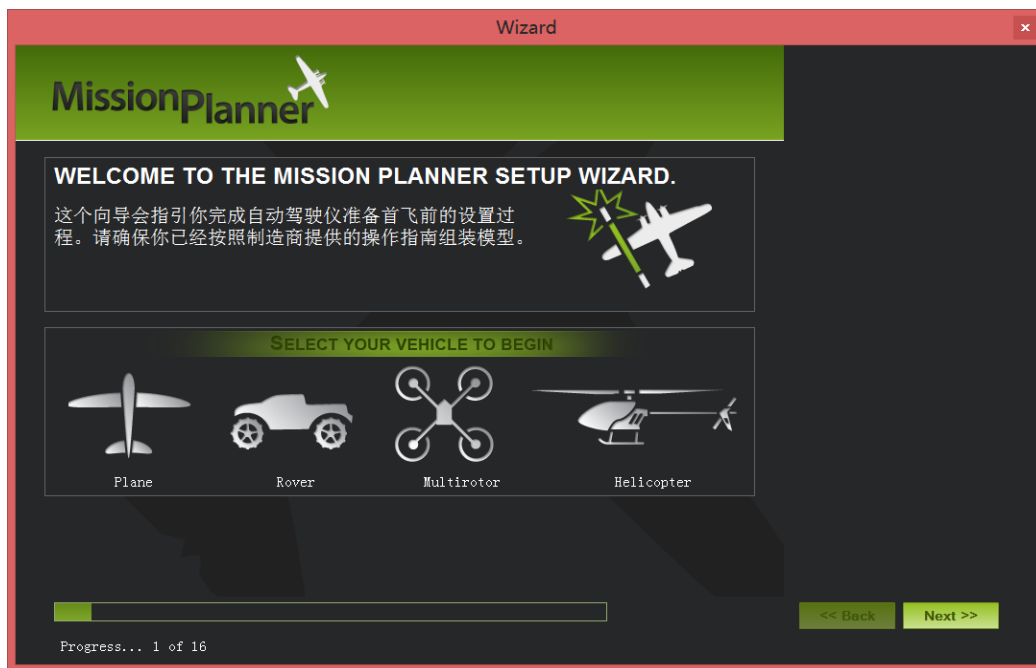


图 4.8 烧写完成

- 选择第三个 Multirotor，单击下一步。

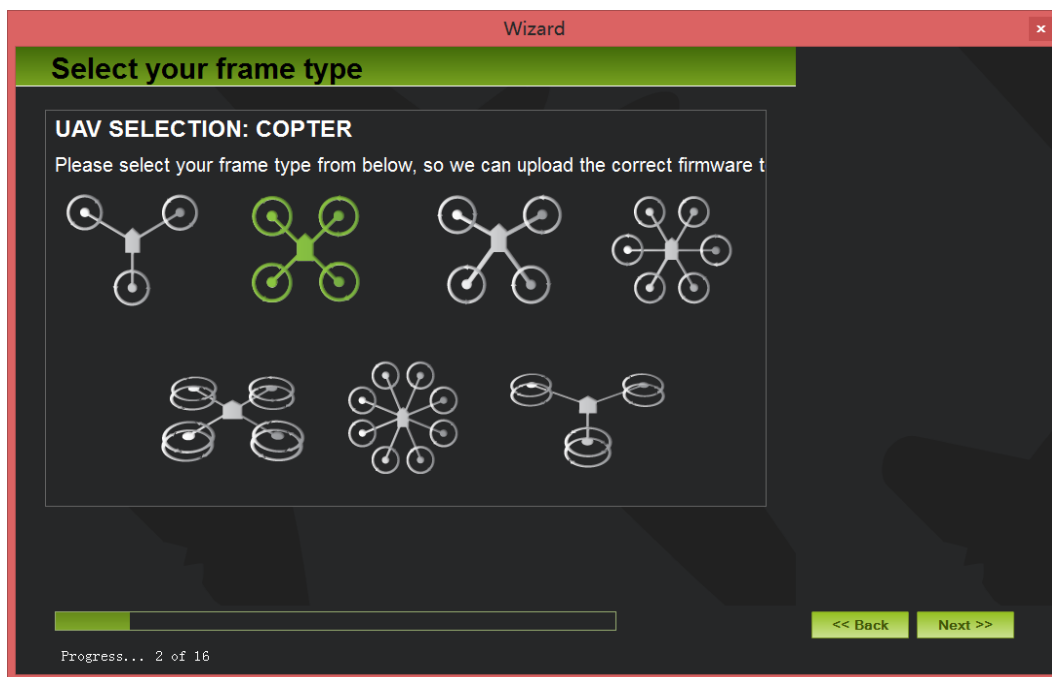


图 4.9 根据向导设置

- 选择多旋翼飞行器，单击下一步。

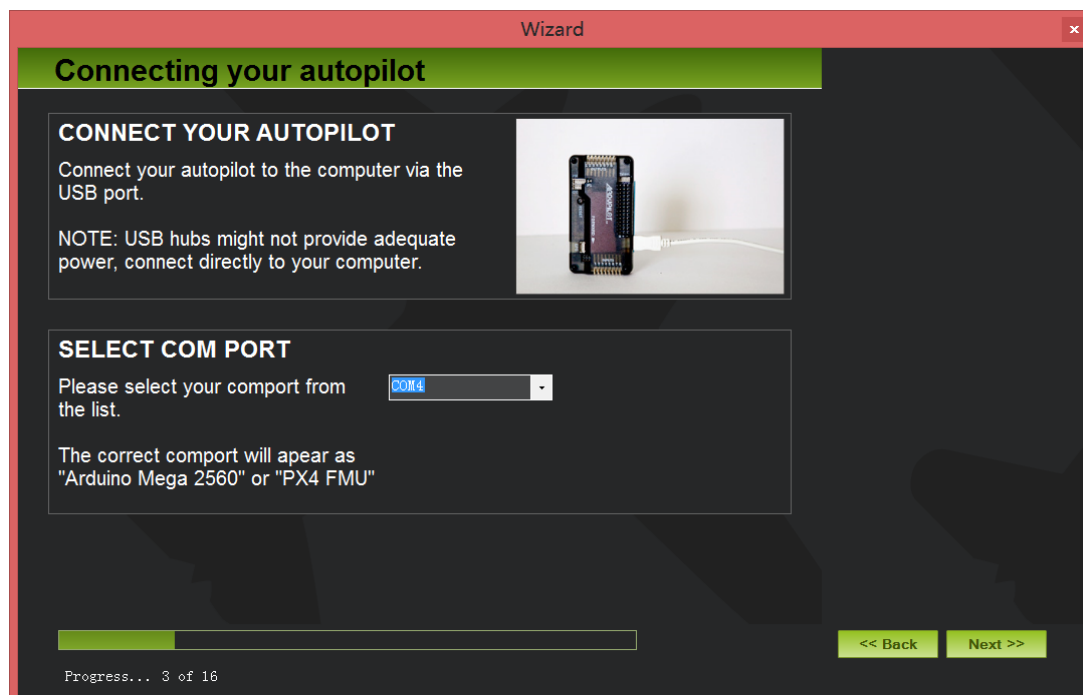


图 4.10 根据向导设置

- 在下拉菜单中选择飞控在电脑中对应的 COM 口，单击下一步。

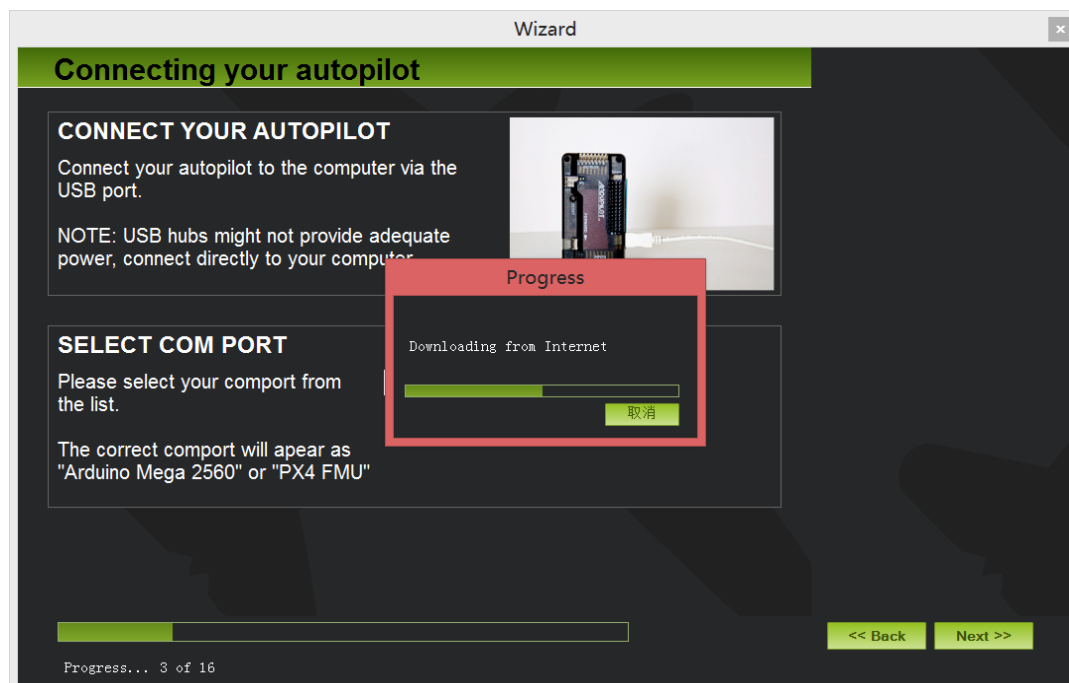


图 4.11 根据向导设置

- 此时电脑开始向飞控中烧录固件。

传感器校准：我们会利用 MP 中的传感器校准向导来完成飞控上的传感器校准。

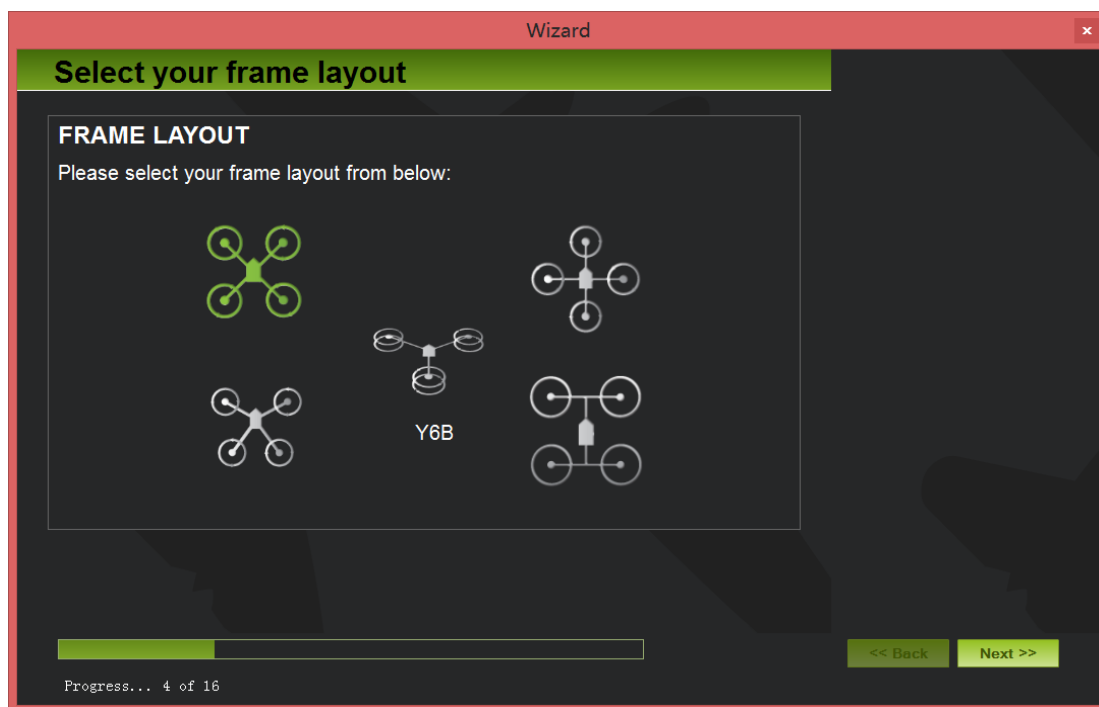


图 4.12 设置机架类型

● 点击左上角的多旋翼飞行器标志。下一步。

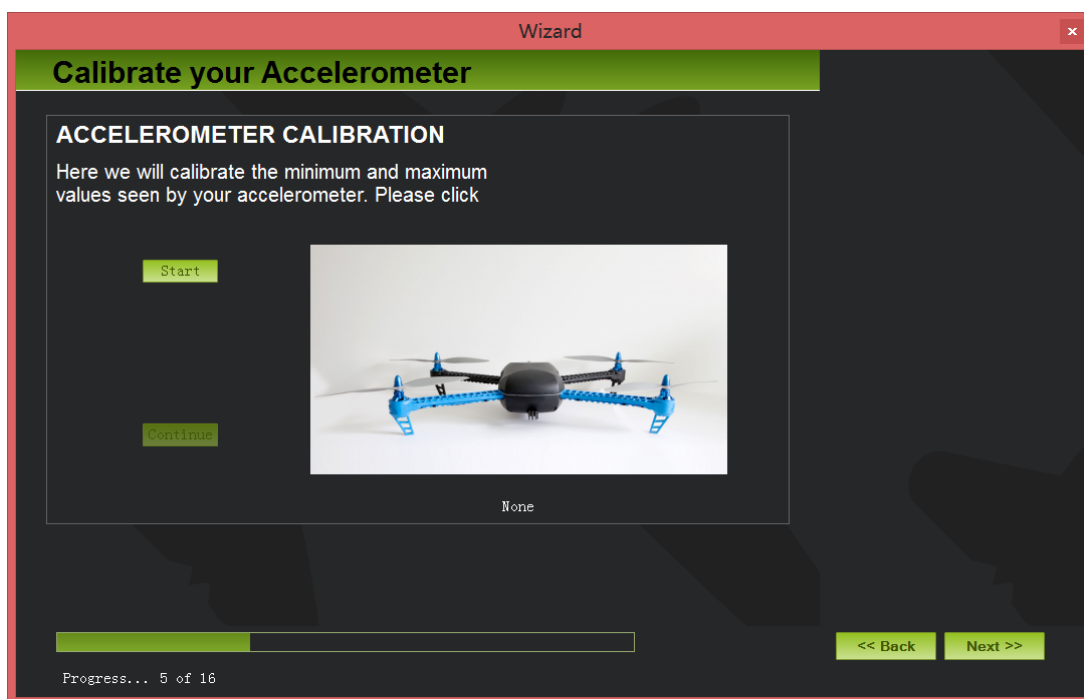


图 4.13 校准加速度计传感器

● 这是加速度计校准界面。单击 Start 开始校准。按照指示一步一步做。直到调试完成，单击下一步。

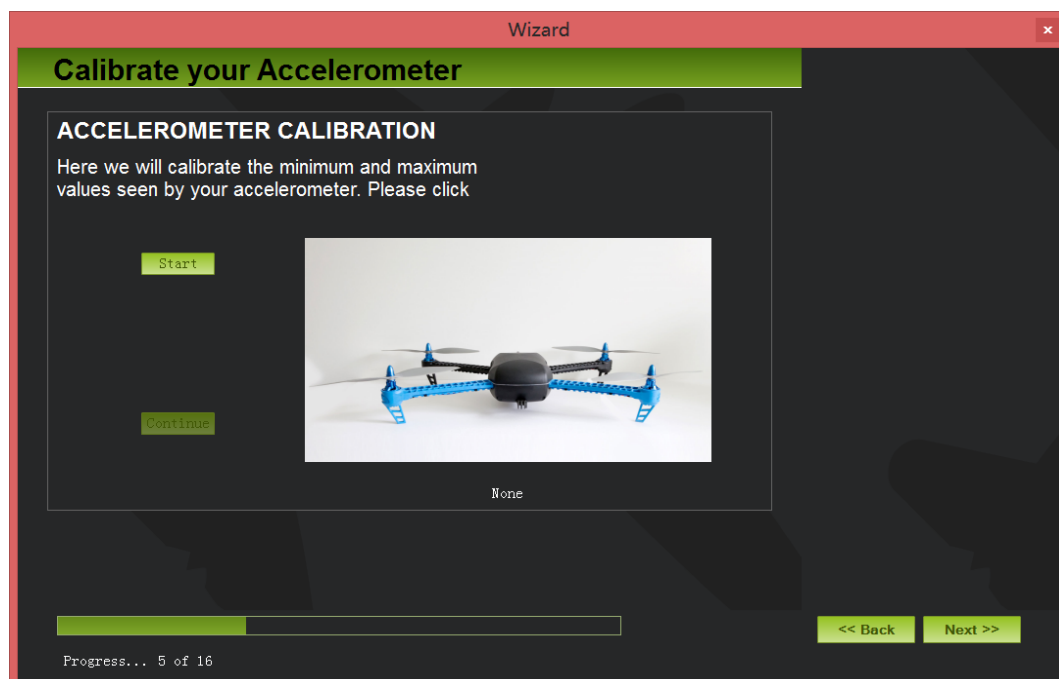


图 4.14 校准加速度计传感器

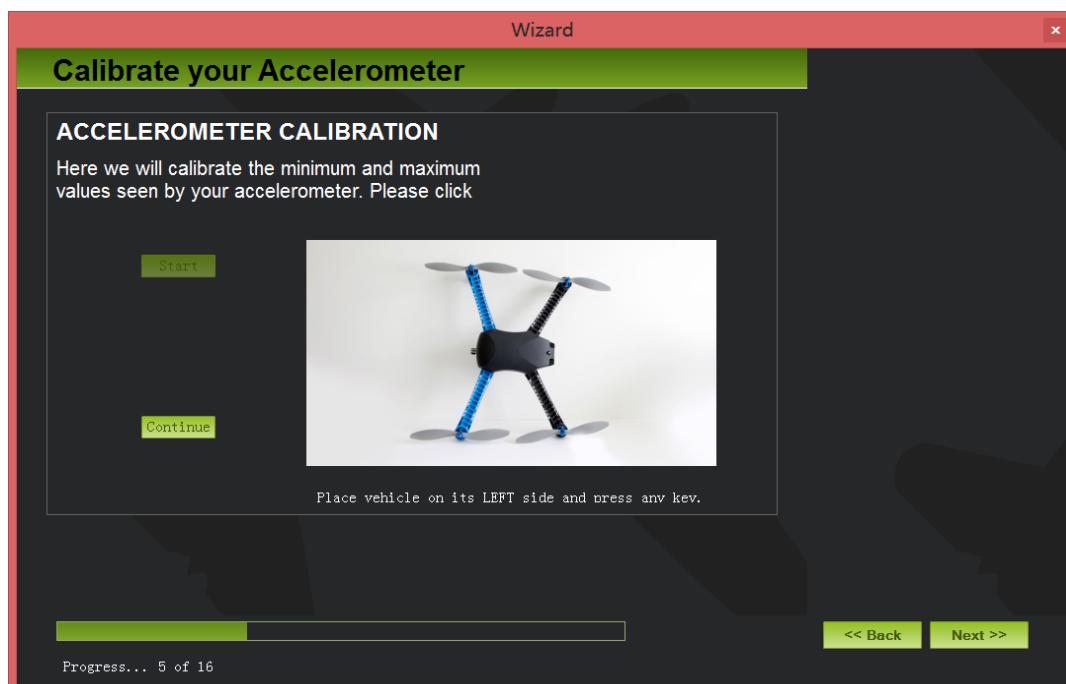


图 4.15 校准加速度计传感器

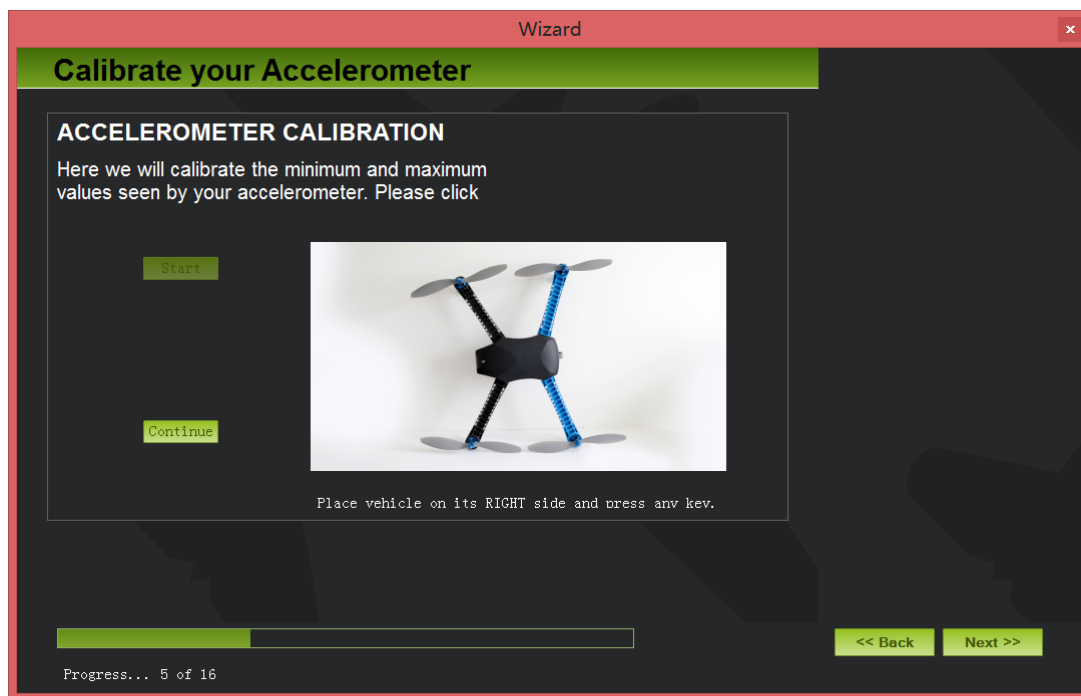


图 4.16 校准加速度计传感器

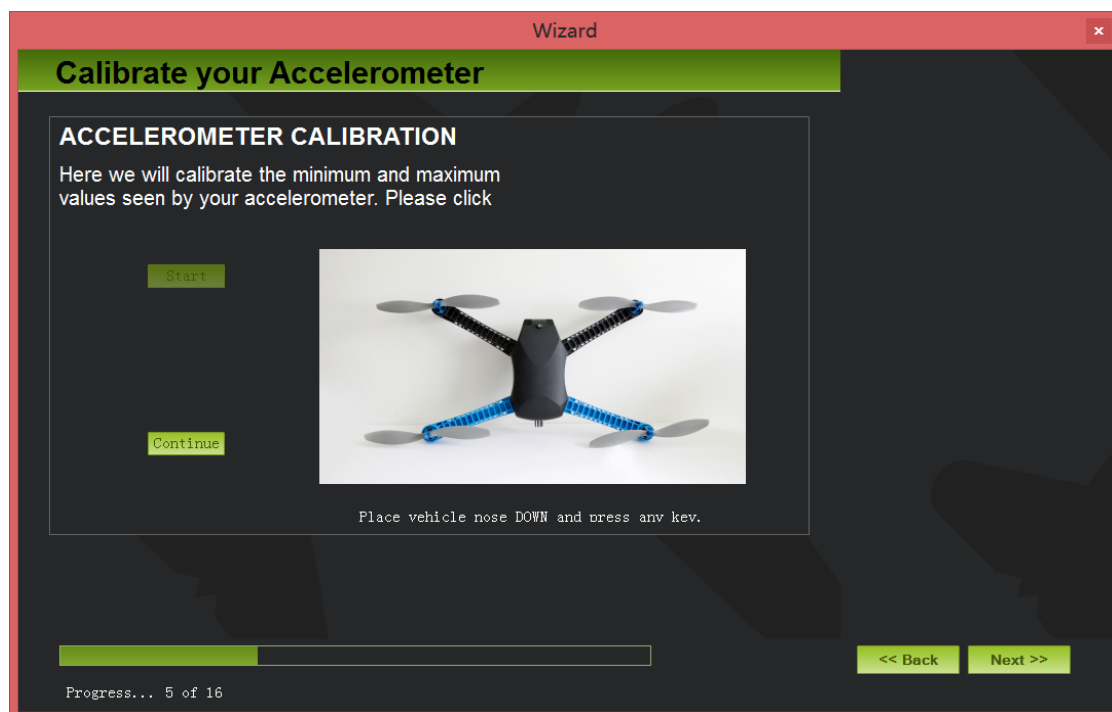


图 4.17 校准加速度计传感器

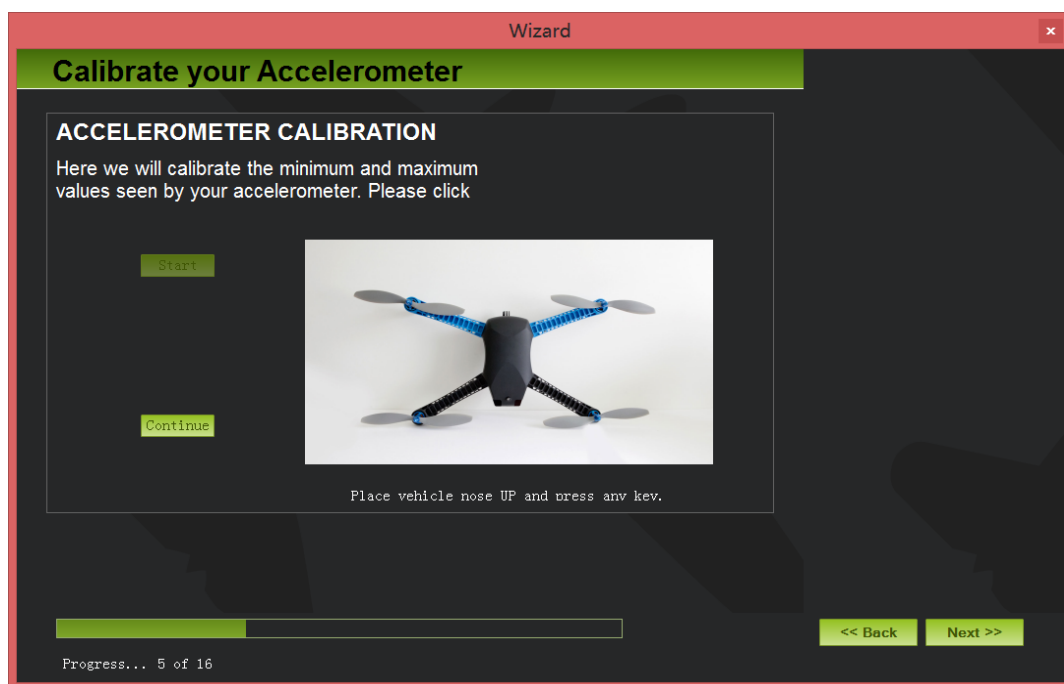


图 4.18 校准加速度计传感器

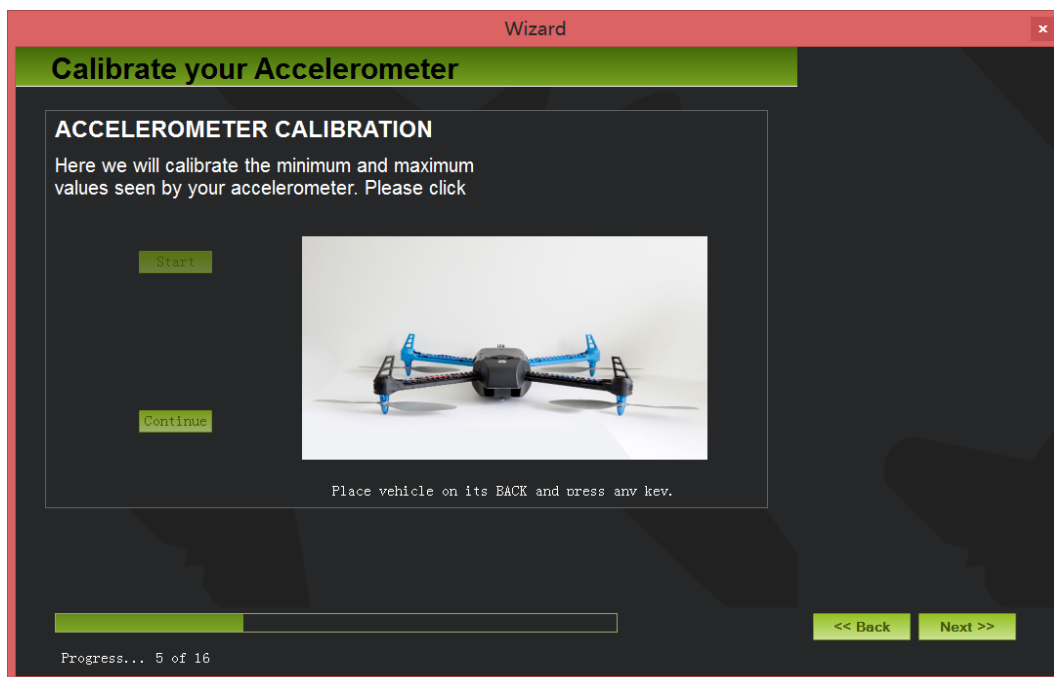


图 4.19 校准加速度计传感器

完成加速度计调试之后，单击下一步。我们将开始罗盘校准。

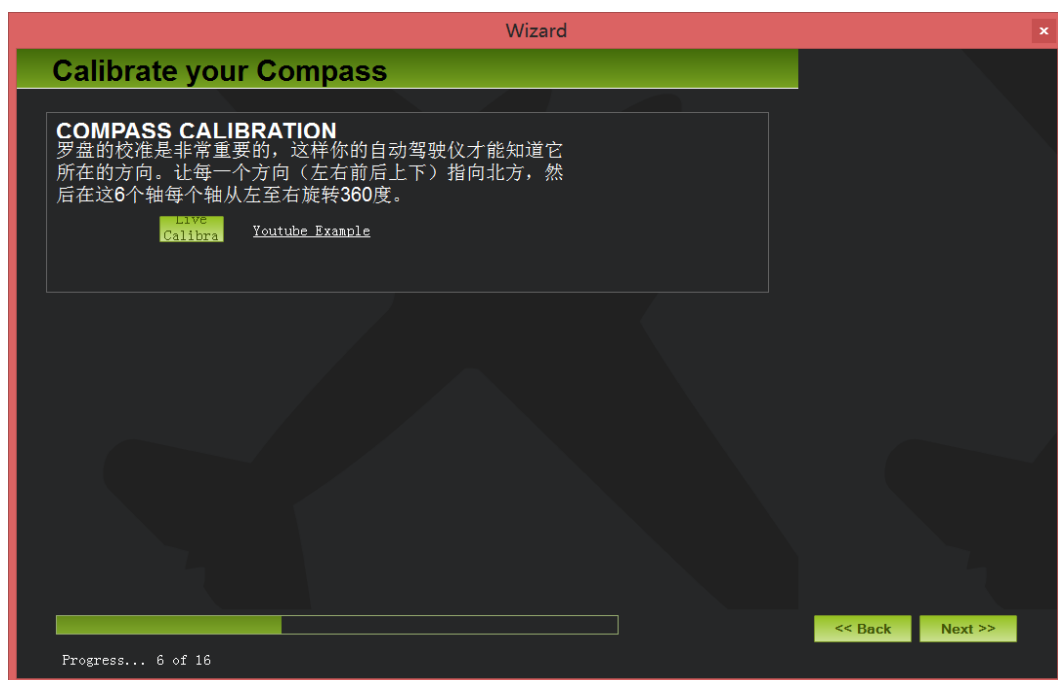


图 4.20 准备校准罗盘

- 单击 Live Calibration 开始校准。我们将会看到屏幕上弹出一个窗口。



图 4.21 校准罗盘

- 此时，我们将绕着飞控的 XYZ 三个轴充分旋转。直到坐标轴周围被点包围住为止，点击 Done。单击下一步，并跳过电源显示器设置，直到遥控器通道校准界面。

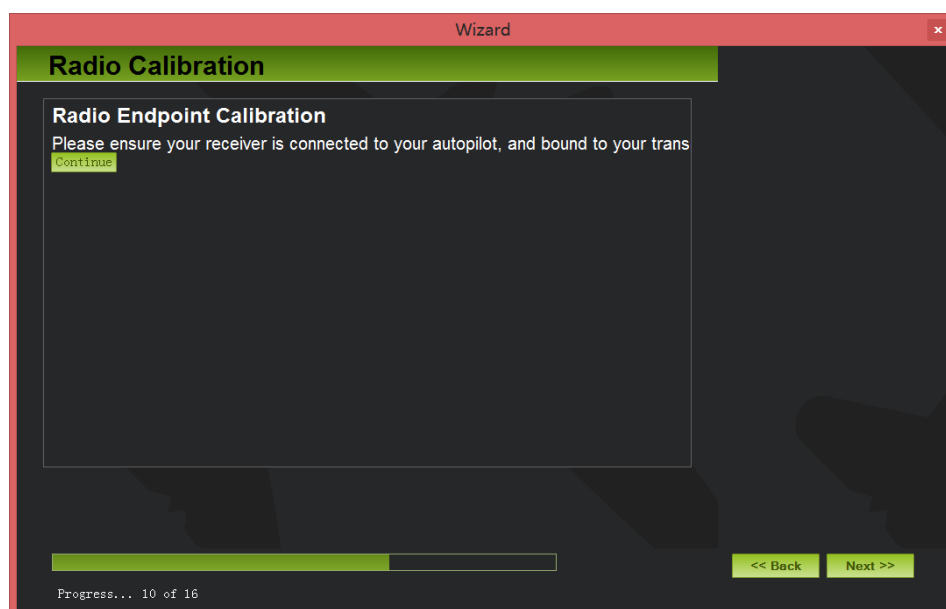


图 4.22 准备校准遥控

- 单击 continue，开始校准遥控器各个通道。

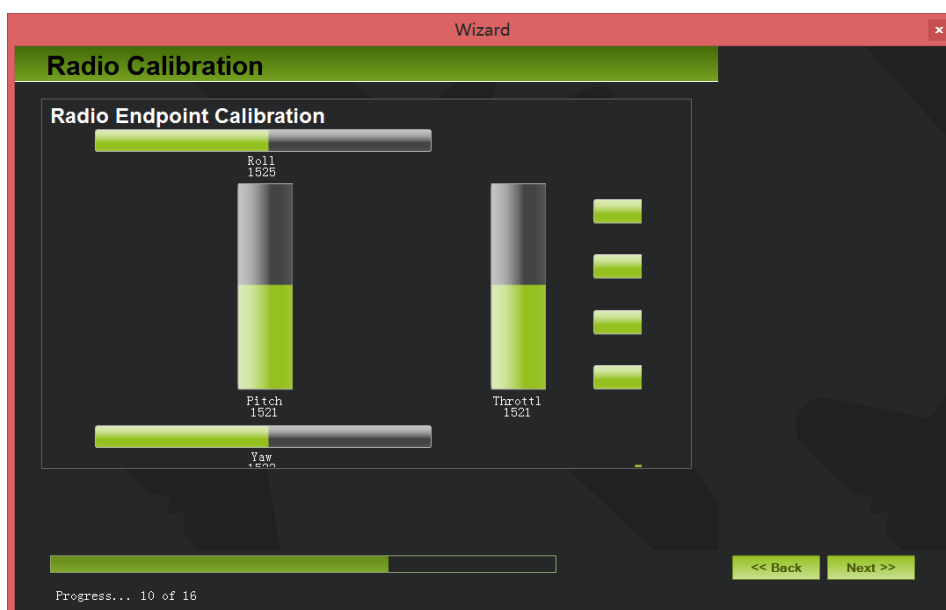


图 4.23 校准遥控

● 此时晃动遥控器上各个摇杆使其达到其上下左右端点以及每一个挡位。然后单击下一步，开始选择飞行模式。如下表调整选项，单击保存模式，并跳过剩下的所有步骤完成向导设置。



图 4.24 设置飞行模式

2.2、设置飞行模式和试飞

这里我们选择三种模式：增稳模式，定高模式，自降模式。

2.2.1、Stabilize mode 增稳模式：

增稳模式是飞控的基本模式，不控的时候飞机的航向俯仰和横测由飞机保持稳定，操纵的时候直接对飞行器姿态进行操作。油门直接由操控手操作。需要注意的是，此模式是飞控姿态控制功能的基本能体现，在进行所有其他模式之前，应该首先对此模式进行调试和试飞。起飞时，由于气流的的涡旋地效，飞行器离地瞬间会有姿态乱的时候，需要小心。

2.2.2、Altitude Hode 定高模式

当切换到此模式的时候，飞行器自动保持当前飞行高度，俯仰、横滚和航向则由遥控器控制。这个模式是很多其他飞行模式的组成部分，非常重要。需要注意的是，飞空使用气压传感器来控制高度，如果因为天气变化导致气压传感器输出发生变化，飞行器的高度也会随之改变。如果飞行器安装并启动的超声波传感器，当飞行高度降低到 79 米左右时则使用该传感器进行高度控制。

2.2.3、Land mode 直降模式

进入直降模式后，飞行器首先以设定的速度下降至 10 米高度，然后以默认 0.5m/s 的速度继续下降，直至判定飞行器落地，自动关闭电机。在下降的过程中，飞行器的俯仰和横滚可控。

2.3、PID 参数的调整

PID 参数的调整是飞控调试中最难的环节。说起来 PID 算法已经有七八十年的历史了，是目前应用最广泛的控制律算法，没有之一。因为其结构简单、稳定性好、工作可靠、调整方便而成为工业控制的主要技术之一。当被控对象的结构和参数不能完全掌握，或得不到精确的数学模型时，系统控制器的结构和参数必须依靠经验和现场调试来确定，这时应用 PID 控制技术最为方便。

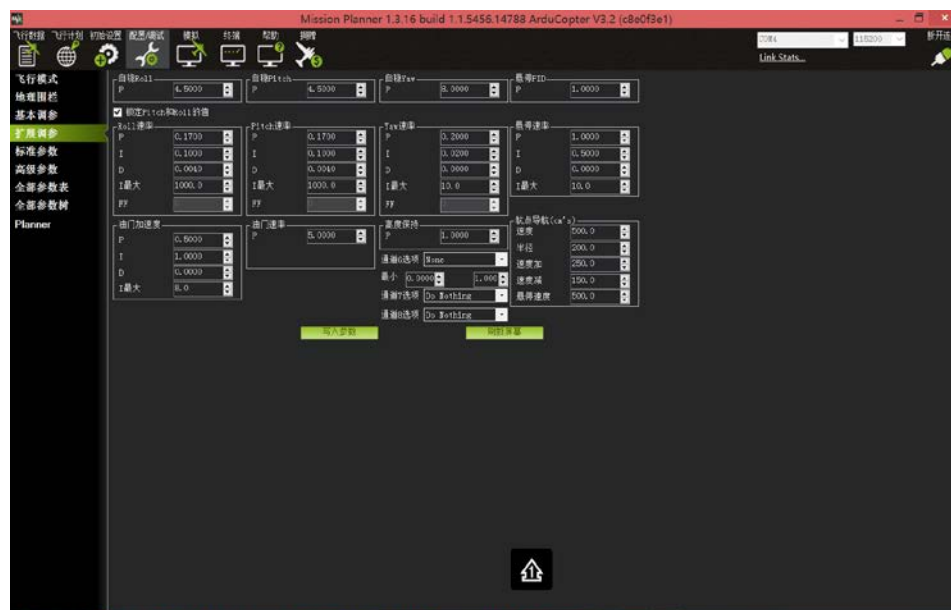


图 4.25 PID 参数设定界面

这是配套飞控在 MP 中的 PID 参数调整页面。

其中上面的一排数据就是飞控位置环的调整参数。

自稳Roll P 4.5000	自稳Pitch P 4.5000	自稳Yaw P 8.0000	悬停PID P 1.0000
--------------------	---------------------	-------------------	-------------------

图 4.26 PID 参数

当我们使用遥控器想让飞机向前运动的时候，就是这组 PID 值来控制飞机的坐标参数，因此叫做位置环，也称作外环。相对应的，在运动过程中控制飞行器姿态稳定的 PID 控制参数叫做姿态环，也叫做内环，下图为内环 PID 以及七八通道的额外功能开关。

☒ 锁定Pitch和Roll的值

Roll速率 P 0.1700 I 0.1000 D 0.0040 I最大 1000.0 FF 0	Pitch速率 P 0.1700 I 0.1000 D 0.0040 I最大 1000.0 FF 0	Yaw速率 P 0.2000 I 0.0200 D 0.0000 I最大 10.0 FF 0	悬停速率 P 1.0000 I 0.5000 D 0.0000 I最大 10.0
油门加速度 P 0.5000 I 1.0000 D 0.0000 I最大 3.0	油门速率 P 5.0000	高度保持 P 1.0000 通道6选项 None 最小 0.0000 1.000 通道7选项 Do Nothing 通道8选项 Do Nothing	航点导航 (cm/s) 速度 500.0 半径 200.0 速度加 250.0 速度减 150.0 悬停速度 500.0

图 4.27 PID 参数

PID 算法里面需要调参的就是 P、I、D 这三个参数，也就是调整者三种反馈信息在控制律中所占的比重。P 调整控制力度，P 作用越大手抬得越快，但是太快就难停下，造成来回找目标点变成抖动；I 是调节误差的，可以使运动更加精确，但是会使控制过程变慢；D 的作用是提供运动的预测，根据预测预先控制，可以在即将到达目标的情况下减小超过的幅度（这个叫做超调），从而加快整个过程。

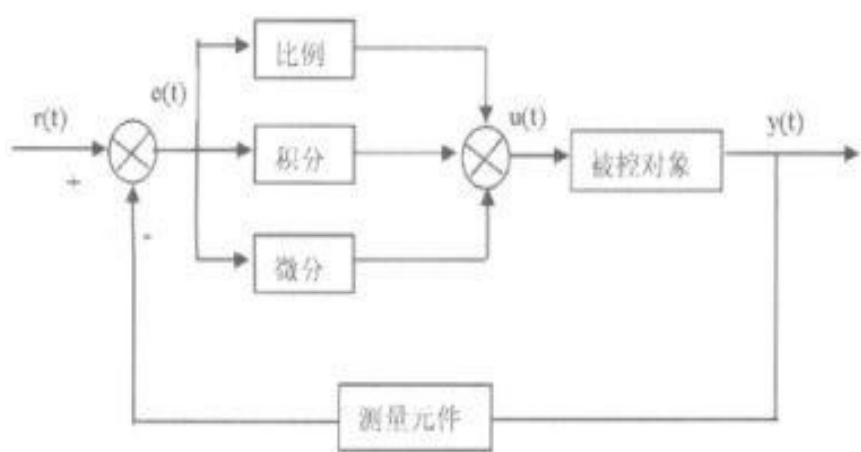


图 4.28 PID 原理

初始参数建议大家使用飞控所带的参数就可以了，一般问题不大；调整顺序一般先调整参数 P，再加入 I，至于 D 则很少用到。

如果飞机振动很强烈，就需要将 P 值减小。相反，如果飞机的姿态调整很慢那么就需要将 P 值增大，直到飞机的振动很“硬”。如果飞机在空中不定向的漂浮，则适当增加 I 值。

在实际调参过程中多旋翼最常出现的问题就是爬、抖、晃。

■ 爬

所谓“爬”指的是飞机在地上蹭来蹭去，蹭来蹭去，就是不起飞；你给大油门飞机就干脆翻个不玩儿了。

根据纯经验，减小 I 的数值可以解决问题。

■ 抖

所谓“抖”指的是飞机起飞后，高频抖动，电机的声音就像得了帕金森。

根据前面的内容，这基本上就是 P 过大造成的不断高速超调，所以减小 P 的数值基本上可以解决这个问题。

■ 晃

所谓“晃”指的是飞机飞起来一切感觉良好，但是你让它一挪地方，它就在那里刷碗，晃晃再稳定住。

根据经验，I 的数值还是大了，造成过程极其缓慢，所以适当减小 I 可以解决问题。

2.4 软件开发及环境介绍

安装 java 虚拟环境：安装步骤如下：

- 在 JAVA 网站上点击“免费下载”。（注意：必须安装 32 位 JAVA 环境）



图 4.29 Java 下载界面

- 点击“同意并开始免费下载”。
- 下载完成后双击启动安装。
- 选项全部选择 OK。
- 当显示“Welcome to Java”窗口是点击“安装”。
- 取消选择“Install the Ask Toolbar”，继续安装。

■ 显示 "You Have Successfully Installed Java Window" 表示安装完成，关闭窗口。



图 4.30 Java 安装界面

在 windows PC 上安装 PX4 Tool Chain 和 Git 的步骤：

- 下载 930M 左右的 PX4 Tool Chain。
- 运行 px4_toolchain_install。
- 运行提示对话框按确定。
- 在版权对话框点击 "I Agree"。
- 安装选项直接点击 next。
- 在 "Choose Install Location" 中选择你需要的安装路径，推荐 C:\px4\，点击安装。
- 其他选项全部选择 OK。
- 完成安装后点击 "Close"。
- 下载 Git。
- 运行安装程序，选择 "Download for Windows"，完成安装

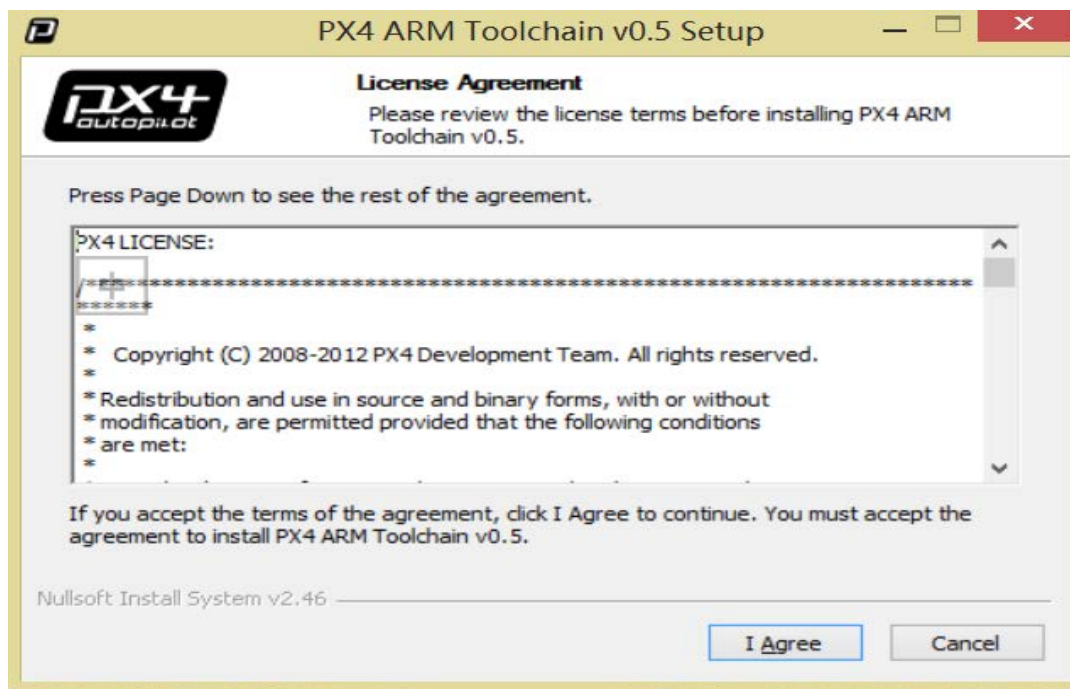


图 4.31 PX4 安装

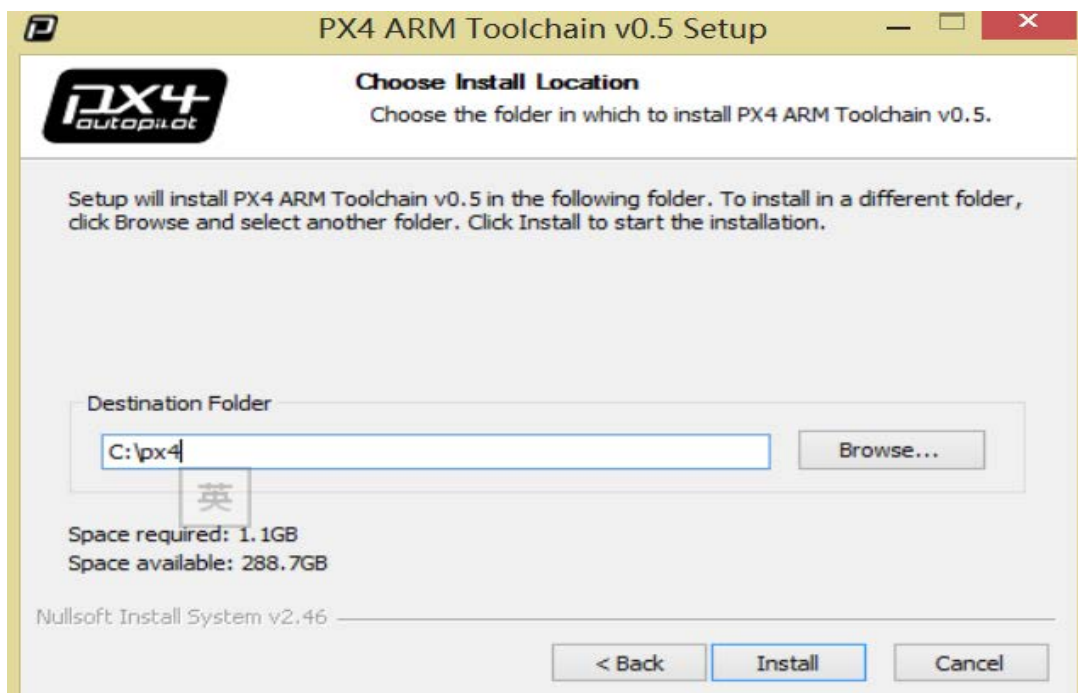


图 4.32 PX4 安装中

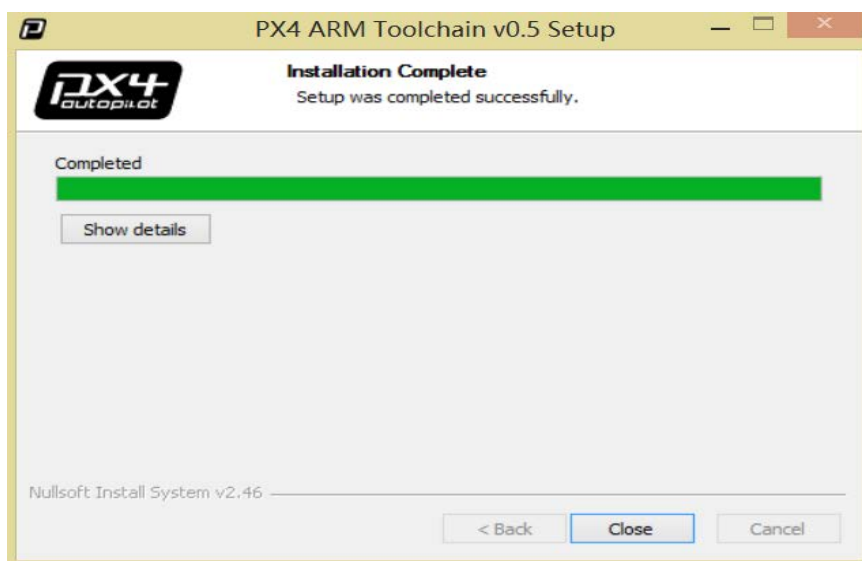


图 4.33 PX4 安装中

下载 PX4 源代码

- 从开始菜单的程序里面, PX4 Toolchain 里面运行"PX4 Software Download".
- 程序会在安装目录下自动下载和建立 3 个子目录(如果没有安装 Git, 此步骤不能执行)。
- 子目录名称: Firmware, libopenm3 和 Bootloader。
- 完成后关闭"PX4 Software Download"窗口。

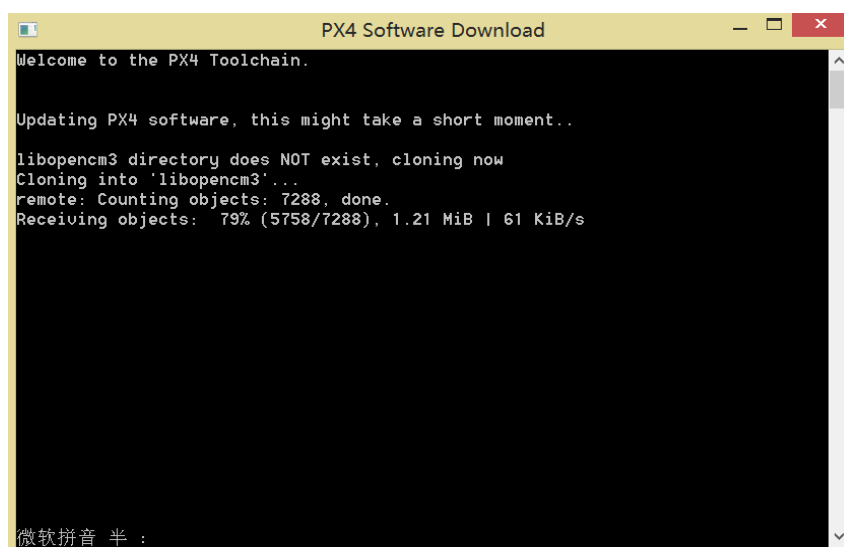


图 4.34 PX4 Software Download 运行界面

第三章 Pixhawk 程序解读

3.1、pixhawk 程序框架

pixhawk 自动驾驶仪软件可分为三大部分：实时操作系统、中间件和飞行控制件。

3.1.1、NuttX 实时操作系统

NuttX 是一个小型实时嵌入式开源操作系统 (RTOS)，它有一个小巧的内核可以在小型 (8 位) 至中型 (32 位) 嵌入式微控制器上运行。NuttX 遵循 POSIX 和 ANSI 标准运行。在 pixhawk 中 NuttX 操作系统负责底层的任务调度和提供可移植操作系统接口 (如 `printf()`, `threads`, `/dev/ttyS1`, `open()`, `write()`, `poll()`, `ioctl()`) 等。

3.1.2、pixhawk 中间件

PX4 中间件运行于操作系统之上，提供设备驱动和模块之间的通信架构 (uORB)，用于 pixhawk 自动驾驶仪上运行的单个任务之间的异步通信。

3.1.3、pixhawk 飞行控制栈

飞行控制栈可以使用 PX4 的控制软件栈，也可以使用其他的控制软件，如 APM:Plane、APM:Copter，但必须运行于 PX4 中间件之上。PX4 飞行控制栈遵循 BSD 协议，可实现多旋翼和固定翼完全自主的航路点飞行。采用了一套通用的基础代码和通用的飞行管理代码，提供了一种灵活的、结构化的方法，可以用相同的航路点和安全状态机来运行不同的固定翼控制器或旋翼机控制器。本文主要针对 PX4 的控制软件栈进行软件分析。

3.2、pixhawk 基本组成模块及功能

Px4 原版控制软件栈位于 `Firmware/src/modules` 中，主要包括以下的模块。

attitude_estimator_ekf	2014/12/17 22:57	文件夹
attitude_estimator_so3	2014/12/17 22:57	文件夹
bottle_drop	2014/12/17 22:57	文件夹
commander	2014/12/17 22:57	文件夹
controllib	2014/12/17 22:57	文件夹
dataman	2014/12/17 22:57	文件夹
ekf_att_pos_estimator	2014/12/17 22:57	文件夹
fixedwing_backside	2014/12/17 22:57	文件夹
fw_att_control	2014/12/17 22:57	文件夹
fw_pos_control_l1	2014/12/17 22:57	文件夹
gpio_led	2014/12/17 22:57	文件夹
mavlink	2014/12/17 22:57	文件夹
mc_att_control	2014/12/17 22:57	文件夹
mc_pos_control	2014/12/17 22:57	文件夹
navigator	2014/12/17 22:57	文件夹
position_estimator_inav	2014/12/17 22:57	文件夹
px4iofirmware	2014/12/17 22:57	文件夹
sdlog2	2014/12/17 22:57	文件夹
segway	2014/12/17 22:57	文件夹
sensors	2014/12/17 22:57	文件夹
systemlib	2014/12/17 22:57	文件夹
uavcan	2014/12/17 22:57	文件夹
unit_test	2014/12/17 22:57	文件夹
uORB	2014/12/17 22:57	文件夹
vtol_att_control	2014/12/17 22:57	文件夹

图 5.1 pixhawk 控制软件栈组成模块

pixhawk 基本组成模块根据功能主要分为以下几个部分：

■ 位姿解算模块

主要包括 attitude_estimator_ekf, attitude_estimator_so3, ekf_att_pos_eatimator 和 position_estimator_inav 四个模块。其中 attitude_estimator_so3 模块在老版本的程序中使用，在新版本中已不再使用。attitude_estimator_ekf 和 position_estimator_inav 主要用于多旋翼飞行器的姿态解算和位置估计，而固定翼飞行器使用 ekf_att_pos_estimator 进行位姿的解算和估计。

■ 无人机控制模块

无人机的控制模块分为多旋翼飞行器和固定翼飞行器两种，多旋翼飞行器的姿态和位置控制在 mc_att_control 和 mc_pos_control 两个模块中实现，而固定翼飞行器的姿态和位置控制是在 fw_att_control 和 fw_pos_control_l1 两个模块中实现。

■ 无人机路径规划模块

无人机的路径规划模块在 navigator 中实现, 主要功能是使无人机根据地面站设置的航路点和航路命令进行飞行。

■ 地面站通信模块

Pixhawk 与地面站的通信主要在 mavlink 模块中实现。此模块基于 mavlink 协议运行, 主要实现与地面站之间的交互。

■ 状态转换模块

Pixhawk 自驾中核心的状态转换和传感器校准以及模式切换在 commander 中进行。

■ 数据记录模块

Sdlog2 负责将数据记录到 SD 卡中, 通常情况下 sdlog2 默认启动, 但是只有在 arm 的情况下才开始记录, 在 disarm 下停止记录。如果想手动开启数据记录可以在 nuttx 的 shell 中先执行 sdlog2 stop 命令使程序停止, 然后利用 sdlog2 start -e 重新启动, 则数据将开始往内存卡中记录。

3.3、pixhawk 模块间消息传递

Pixhawk 不同模块之间通过消息 (topics) 传递信息, 一个消息中只包含一种类型的信息, 例如 vehicle_attitude 消息传递姿态相关的结构体信息包括俯仰滚转和偏航等信息。各模块可以发布或订阅不同的消息, 但是各模块之间并不知道消息由谁发布和被谁订阅。同一个消息可以有多个发布者和订阅者。这种发布订阅的机制在 uORB 中实现。

定义自己的消息的一般过程是:

在 uORB/topics 中增加一个头文件, 一般以你定义的消息名称作为头文件的文件名, 在头文件包含以下两个文件。

```
#include <stdint.h>和#include "../Uorb.h"
```

在头文件中定义消息的结构体，并通过语句 `ORB_DECLARE(topic_name)` 声明消息。通常结构体名字为消息名称加 `_s`，例如定义一个消息

`ORB_DECLARE(att_estimate)`，则结构体通常定义为 `att_estimate_s`。

在 `objects_common.cpp` 中将定义的消息文件包含进去，并且利用 `ORB_DEFINE(att_estimate, struct att_estimate_s)` 将消息和结构体关联起来。

这样就完成了一个消息的定义，可以在其他模块中使用这个消息了。

3.4、Pixhawk 的软件启动与运行

Pixhawk 的启动过程由 `/etc/init.d/rcS` 中的启动脚本文件控制，这个文件被编译在 `firmware` 中存储在 ROM 里。这个脚本文件负责监测飞控板的硬件，装在外围器件的驱动文件，并且根据个人的配置启动需要的程序模块。

启动脚本文件在源文件的 `ROMFS/px4fmu_common/init.d` 中存储。其中的 `rcS` 脚本文件主要执行以下的工作：

- SD 卡是否安装存在
- 内存卡中 `etc/rc.txt` 文件是否存在，如果存在则执行此文件。

如果没有找到 `etc/rc.txt`，则执行默认的自启动程序

第四章 MEMS 传感器

4.1、MEMS 传感器的发展

4.1.1、MEMS 传感器的历史

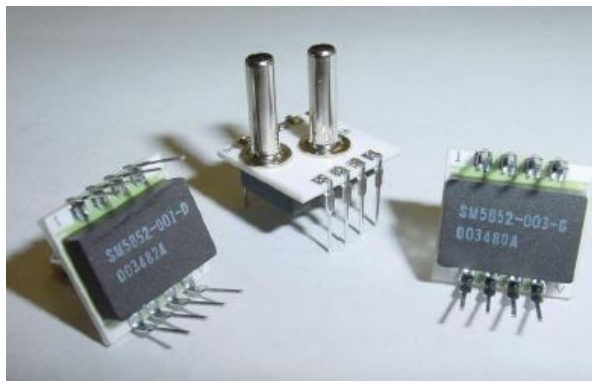


图 6.1 MEMS 传感器

MEMS (Microelectro Mechanical Systems)，即微机电系统，这个领域是以半导体制造技术为基础发展起来的多学科交叉的前沿研究领域。MEMS 传感器是采用微电子和微机械加工技术制造出来的新型传感器。与传统的传感器相比，它具有体积小、质量轻、成本低、功耗低、可靠性高、技术附加值高、适于批量化生产、易于集成和实现智能化等特点。中国对 MEMS 的发展也非常关注，国家在“十一五”期间投入大量资金从事该方向的研究，并自主开发了 MEMS 振动传感器、MEMS 声响传感器、MEMS 红外传感器、光纤传感器、声阵列传感器，形成了以 MEMS 传感器为主、复合多信息探测的小型化传感器系列。

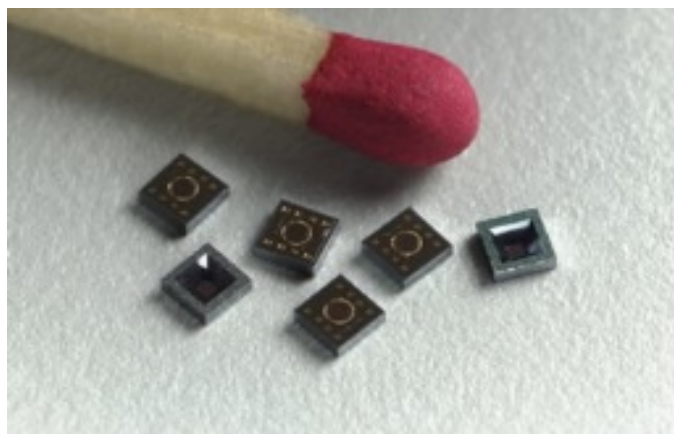


图 6.2 传感器和火柴头比较

MEMS 工艺的发展可以追溯到 1954 年, Smith 在贝尔实验室发现了压阻效应。随后, 基于硅材料的微机械加工工艺和微机电系统 (MEMS) 引起了科研人员的广泛注意并得到了迅速的发展。Feynman 在 1959 年的美国物理年会上首次提出了微计算机、微机械和微器件的设想。1962 年, 硅微型压力传感器问世, 1977 年, 由斯坦福大学研制的电容式压力传感器出现, 随后又研制出电容式加速度传感器。1979 年, Rolyance 和 Angell 开始研制压阻式微加速计。1987 年, 加州大学伯克利分校率先研制出直径为 $50\sim 500\ \mu\text{m}$ 的硅微马达, 标志着微电子机械系统雏形的出现。1989 年 MEMS 这一名称正式被提出, 90 年代后期出现了 NEMS, 其特征尺寸在几纳米至几百纳米。

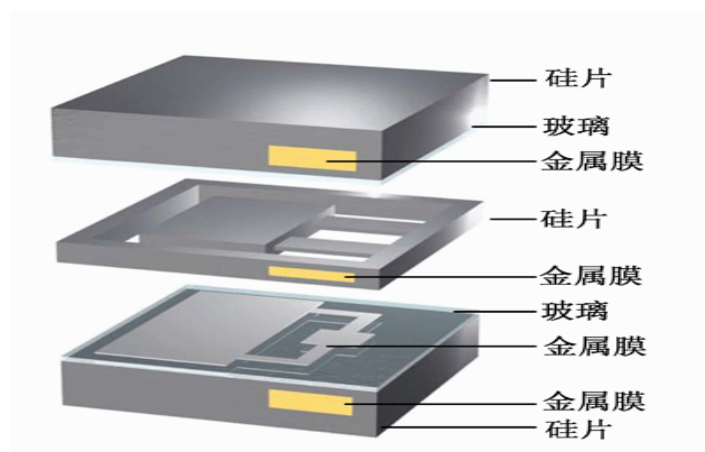


图 6.3 MEMS 传感器硬件结构

MEMS 传感器在国际上比较通用的分类中一共经历了 4 个发展阶段。第一代 MEMS 传感器元件主要基于硅芯片的结构，也有部分传感器在芯片上加入了模拟放大器；第二代 MEMS 传感器元件将模拟放大器和模数转换器集成在同一个芯片上；第三代 MEMS 传感器在一个芯片上同时集成了模拟放大器、模数转换器和数字智能温度补偿技术；第四代 MEMS 传感器是在第三代的基础上增加了校准记忆芯片以及温度补偿数据。

4.1.2、MEMS 传感器类型

MEMS 传感器的类别很多，按传感器的工作原理可以大致地分为物理型、化学型和生物型三类。按照被测的量又可分为加速度、角速度、压力、位移、流量、电量、磁场、红外、温度、气体成分、湿度、PH 值、离子浓度、生物浓度等类型的传感器。综合两种分类方法的分类体系如下示意图所示。

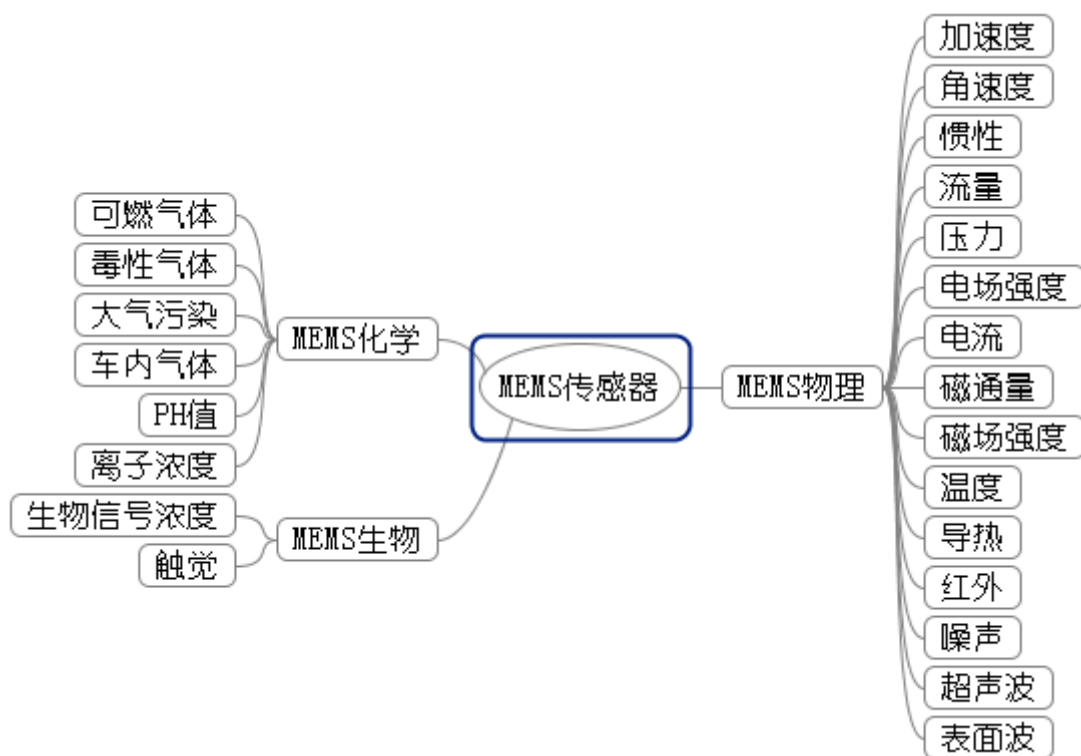


图 6.4 MEMS 传感器分类

4.1.3、MEMS 传感器的几个比较有影响力的生产厂家

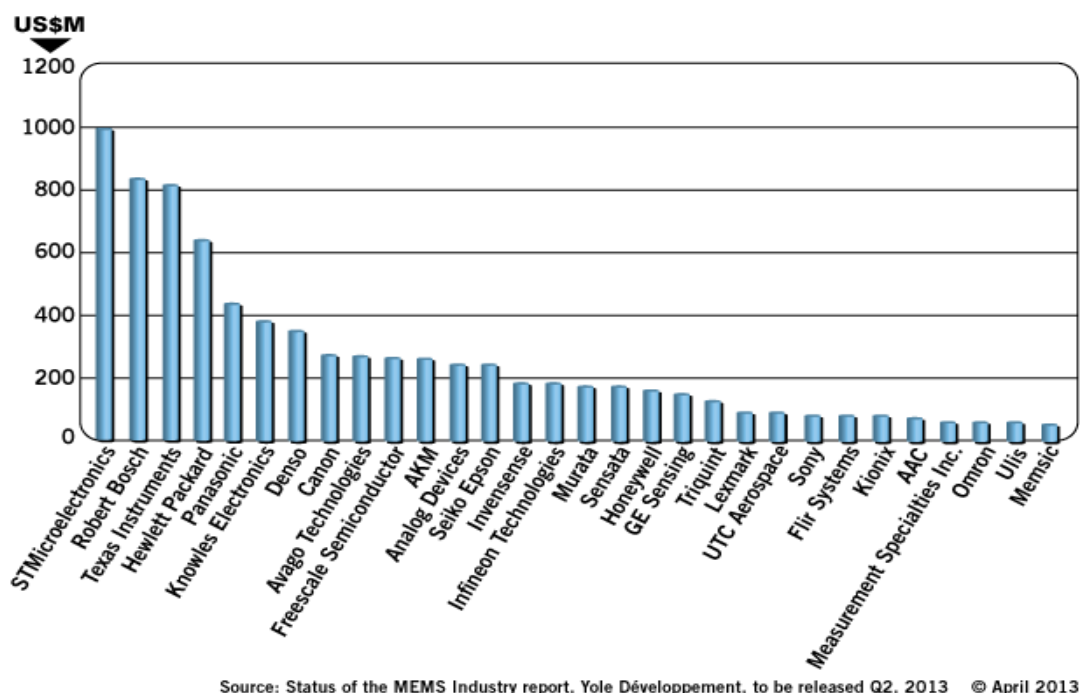


图 6.5 MEMS 传感器厂商

根据法国咨询公司 Yole Development 公司在 2012 年的数据，意法半导体（STMicroelectronics）是全球最大的 MEMS 传感器制造商；罗伯特博世公司（Robert Bosch GmbH）和德州仪器（Texas Instruments）在 MEMS 传感器出货量及销售额上不分伯仲，紧随意法半导体之后；再之后是惠普（Hewlett-Packard）、松下（Panasonic）和株式会社电装（DENSO）。而中国排名靠前的 MEMS 传感器厂商普遍在台湾，其中台积电（台湾积体电路制造公司）是全球最大的 MEMS 传感器代工厂商。

4.2、MEMS 传感器的应用

MEMS 传感器的操作范围在微米范围内，是一个智能并且独立的系统。由于 MEMS 传感器具有可微型化、可集成化、可批量化和多学科交叉的特点以及优良的物理性能，其在航空航天、汽车制造、电子设备控制设计、生物医疗等军用民用领域都有着重要的作用。在这里我们将以 MEMS 传感器在移动通讯设备生产、

游戏娱乐设备制造和多旋翼飞行器控制系统中的作用为例，了解 MEMS 在日常生活中的应用。

4.2.1、MEMS 传感器与移动通讯设备

如今移动通讯的迅速发展离不开 MEMS 技术的成熟。智能手机中众多软件功能的实现除了基于它功能强大的操作系统之外，也要归功于各种 MEMS 传感器的应用。为了增强智能手机的人机交互和众多软件功能的实现，MEMS 加速度计、触控面板、MEMS 陀螺仪、MEMS 磁力计和气压高度计等传感器早已集成在通讯设备的芯片中。

智能手机中的导航功能。MEMS 传感器是可以随时随地工作的自导式器件。其成本低、尺寸小、功耗低，是行人行驶方位推算应用的首选传感器。与 GPS 接收机集成在一起可以随时获取精确的地理信息位置。



图 6.6 MEMS 传感器应用

智能手机中的重力感应应用的实现。在智能手机和平板电脑中，屏幕旋转功能的实现就是基于 MEMS 重力加速度传感器来实现的。它可以精确地检测设备角度的变化，同时满足高精度移动设备苛刻的体积和功耗限定。同样，基于 MEMS 三轴加速度计、三轴陀螺仪和三轴磁力计对线加速度、角加速度和地磁场的测量可以实现依靠重力感应控制游戏画面中“汽车”转向或者依靠屏幕上的罗盘认清方向等等。



图 6.7 MEMS 传感器开发的应用

4.2.2、MEMS 传感器与游戏娱乐设备

例如世界上一款著名的游戏机——任天堂游戏机，依靠 MEMS 得天独厚的功能与性能在巨大的游戏机市场占据一席之地。意法半导体公司（ST）在 2006 年宣布将在任天堂游戏机中集成 MEMS 三轴加速度传感器来提供“动作驱动”的用户界面。

依靠在控制器内置 ST 高性能加速度传感器，任天堂游戏机能够在三维空间范围内检测玩家的动作，并将其转化成在游戏界面中的人物动作。凭借 MEMS 传感器高响应速度和高精确度，玩家能够在游戏中更好的互动，并且高精度的传感器可以将玩家手中的控制器变成一把虚拟的宝剑或者架子鼓棒。



图 6.8 MEMS 传感器在游戏交互中的应用

4.2.3、MEMS 传感器与多旋翼飞行器控制设计

多旋翼飞行器是依靠四个不同的出入量控制六个自由度的欠驱动系统。如果我们需稳定控制多旋翼飞行器,那么依靠人眼的视觉判断飞行器姿态并依靠经验用无线遥控装置来调整飞行器姿态是远远不够的。并且根据前文提到的多旋翼飞行器原理,若想用人工调整四个旋翼的功率来改变飞行器的飞行方向是不现实的。若使用传统模拟传感器收集飞行器在飞行状态下的数据来进行自动调整,其机械重量会导致飞行器的重量急剧上升同时飞行器的续航时间会大大缩短。而且模拟传感器的响应速度低,并不能满足多旋翼飞行器的高效率运转。

因此,我们可以想到利用微型化的 MEMS 传感器和其高速的响应速度来收集足够的而且精确的实时飞行姿态数据作为控制器处理的数据来源。这也是近些年来 MEMS 技术的成熟带来的多旋翼飞行器的崛起。

这里我们需要介绍一种 10 自由度传感器(10DOF)。10DOF 模块集成了 MEMS 技术制造的三轴加速度传感器、三轴数字输出陀螺仪、三轴磁场强度传感器和气压高度计。是纯粹的传感器集成模块。该模块可以直接应用于机器人设计、无人机设计、自动汽车和多旋翼飞行器控制系统的硬件模块。

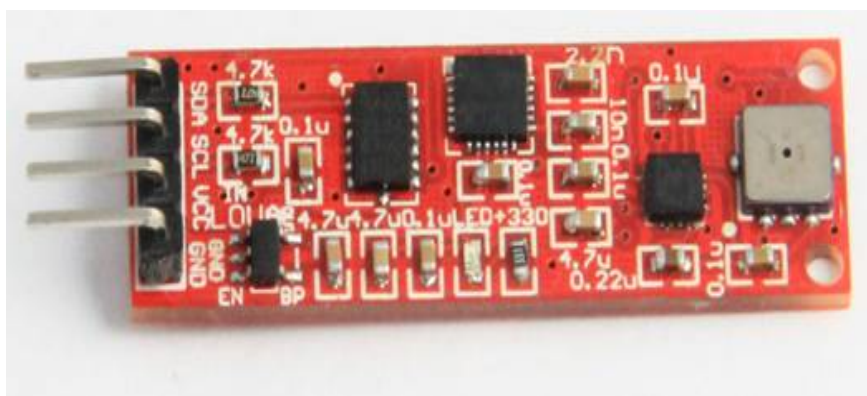


图 6.9 飞行器上使用的 MEMS 传感器

常见的 10DOF 模块有 GY-86(87) 10DOF 模块和 Mini IMU AHRS 10DOF 模块。其中 GY-86 10DOF 是 MWC 开源飞控的传感器部件。下面的图解为 IMU AHRS 10DOF 模块的硬件组成和实际应用。

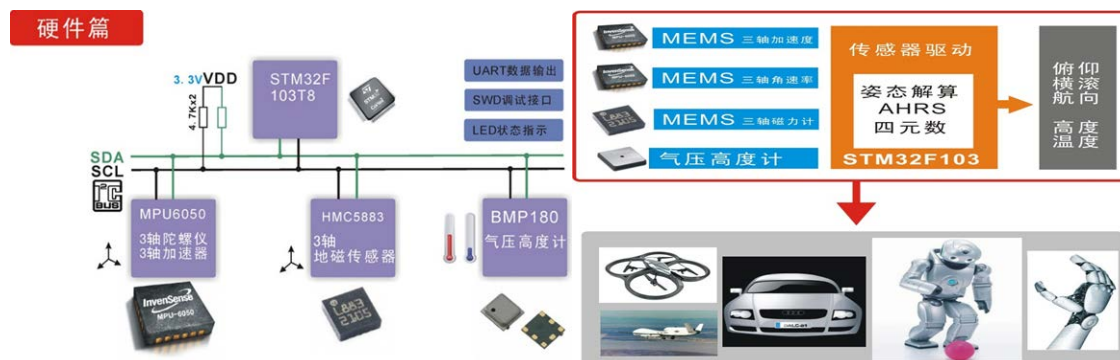


图 6.10 MEMS 传感器系统

4.3、常用的 MEMS 传感器

4.3.1、MPU-6000 系列及其扩展型号

4.3.1.1、用途



图 6.11 MPU6050

MPU-60X0（包括 MPU-6000 和 MPU-6050）是全球首例集成 9 轴的运动处理传感器。MPU-60X0 集成 3 轴 MEMS 陀螺仪，3 轴 MEMS 加速度计以及利用可扩展的数字运动处理器（Digital MotionProcessor（DMP））的 I²C 接口外接的数字传感器。比如当 MPU-60X0 外接一个 3 轴的磁力计，就可以通过其 I²C 接口或者 SPI 接口输出一个完整的 9 轴信号（SPI 接口仅在 MPU-6000 可用）。MPU-60X0 主要可应用于清晰稳定成像、身份认证、姿态控制、姿态识别、可便携式导航设备、手机与平板电脑游戏、动态游戏控制、数字电视的 3D 远程遥控、人体健康监测、玩具等领域。

4.3.1.2、特征

- 以数字输出 6 轴或 9 轴的旋转矩阵、四元数、欧拉角格式的融合演算数据。
- 具有可程式控制范围为 ± 250 , ± 500 , ± 1000 , ± 2000 /秒 (dps) 的陀螺仪和可程式控制范围为 $\pm 2g$, $\pm 4g$, $\pm 8g$, $\pm 16g$ 的加速度计
- 拥有集成的 16 位模拟数字转换器 (ADCs), 可以同步将陀螺仪和加速度计测量的模拟量转化为数字量
- 可以通过 I²C 接口读取外接传感器数据
- 在之前的基础上降低了加速度计与陀螺仪轴间的敏感度, 降低了设定给予的影响和感测器的漂移
- 数字运动处理器 (DMP) 减少了复杂的融合演算数据、感测同步化、姿态感应等的负荷
- 内置运作时间偏差与磁力感测器校正演算技术, 免除了客户须另外进行校正的需求
- 具有以数位输出的温度传感器
- 以数位输入的同步引脚支持视频电子影像稳定技术与 GPS
- 可程式控制的中断支持姿势识别、摇摄、画面放大缩小、滚动、快速下降中断、high-G 中断、零动作感应、摇动感应功能
- 具有高达 400kHz 快速模式的 I²C 接口, 或最高至 20MHz 的 SPI 串行主机接口
- 内建频率产生器在所有温度范围仅有 $\pm 1\%$ 频率变化

4.3.1.3、传感器参数

■ 陀螺仪参数

参数	条件	最小	标准	最大	单位
陀螺仪零率输出	初始条件 25° C		± 20		$^{\circ}/s$
随温度变化	-40° C 到 +85° C		± 20		$^{\circ}/s$

非线性度	理想直线 25° C		0.2		%
交叉轴灵敏度			±2		%
总功率输出噪声	DLPFCFG=2 (100Hz)		0.05		°/s-rms
低频功率噪声	1Hz -10Hz		0.033		°/s-rms
噪声率谱密度	10Hz		0.005		°/s/√Hz

表 6.1 陀螺仪传感器参数

注：Y 轴自检的时候会收到来自 X 轴和 Z 轴的反极性信号。

■ 加速度计参数

参数	条件		最小	标准	最大	单位
零加速度输出 随温度变化	初始校准	X 轴与 Y 轴		±50		mg
		Z 轴		±80		mg
	温度变化 0° C ~+70° C	X 轴与 Y 轴		±35		mg
		Z 轴		±60		mg
非线性度	理想直线			0.5		%
交叉轴灵敏度				±2		%
功率谱密度	10Hz, AFS_SEL=0 & ODR=1kHz			400		g/√Hz

表 6.2 加速度计传感器参数

■ 电气指标

参数	条件	最小	标准	最大	单位
----	----	----	----	----	----

参数	条件	最小	标准	最大	单位
VDD 供电电源 (工作电压)		2.375		3.46	V
功耗	陀螺仪、加速度计和 DMP 均工作		3.9		mA
温度范围 指定温度范围	性能参数在指定温度范围之外 不适用	-40		+85	°C

表 6.3 MEMS 传感器电器特性

■ 绝对最大额定值

参数值超过下表中绝对最大额定值可能会对设备造成永久性的损害。

传感器	额定值
供电电压, VDD	-0.5V ~ +6V
加速度 (所有轴)	10000g 在 0.2ms 内
工作温度范围	-40°C ~ 105°C
储存温度范围	-40°C ~ 125°C
静电放电保护	2kV (HBM); 200V (MM)

表 6.4 极限参数

4.3.1.4、封装

首先我们定义轴的正方向与轴的旋转方向, 标记引脚 PIN1 如下图所示。

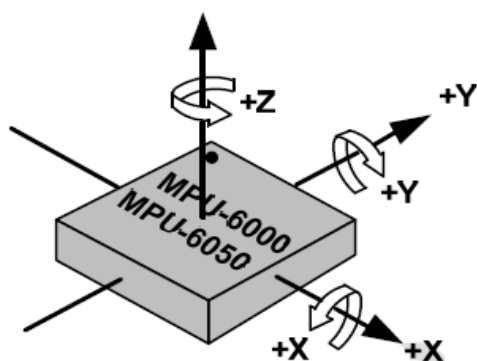


图 6.12 MPU6050 各轴的正方向与旋转方向

MPU-6000 采用的 QFN 封装技术（方形扁平无引脚封装），其封装尺寸为 $(4*4*0.9) \text{ mm}^3$ ，详细尺寸如下图所示。

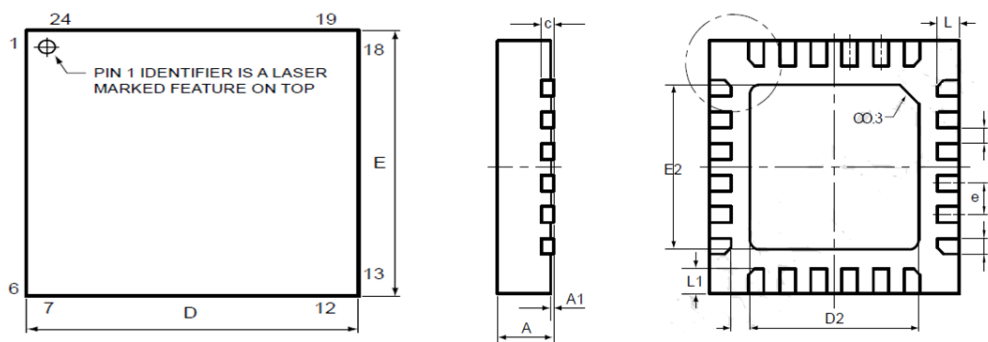


图 6.13 MPU6050 封装

符号	均以毫米（mm）为单位		
	最小尺寸	中间尺寸	最大尺寸
A	0.85	0.90	0.95
A1	0.00	0.02	0.05
b	0.18	0.25	0.30
c	—	0.20 REF	—

符号	均以毫米（mm）为单位		
	最小尺寸	中间尺寸	最大尺寸
D	3.90	4.00	4.10
D2	2.65	2.70	2.75
E	3.90	4.00	4.10
E2	2.55	2.60	2.65
e	—	0.50	—
f(e-b)	—	0.25	—
K	0.25	0.30	0.35
L	0.30	0.35	0.40
L1	0.35	0.40	0.45
s	0.05	—	0.15

表 6.5 MPU6050 封装尺寸

4.3.2、其他系列

■ MPU-9150

MPU-9150 为全球首例九轴运动感测追踪组件，专为如智能型手机、平板电脑、配戴式传感器等需要低功耗、低成本、高性能的消费性电子产品设备设计。

■ MPU-9250

InvenSense 第二代 9 轴组合传感器将 6 轴惯性测量单元(加速度计+陀螺仪)和 3 轴磁力计集成于 3 mm x 3 mm QFN 封装中，相比上一代产品，面积减小了 45%。

■ L3GD20

应用于体感游戏输入设备。

应用于人体运动识别设备。

应用于家用电器和机器人生产制造。

■ 特征

L3GD20 三轴数字输出陀螺仪是一种低功耗三轴角速率传感器。该传感器通过集成在传感器上的微机械敏感元件和 IC 端口收集测量的偏转角速率，并通过 I²C /SPI 数字串口输出到外部电路。

传感器上的敏感元件是借助于意法半导体公司（ST）的微机械制造技术刻蚀在硅晶片上的惯性测量元件。IC 端口是用 COMS 集成电路制造工艺集成在传感器内部的电路，COMS 集成电路制造工艺能够实现 IC 端口与机械敏感元件匹配的高度集成电路。

L3GD20 三轴数字输出陀螺仪有三种量程（ ± 250 / ± 500 / ± 2000 dps），因此用户可以根据自己的需求设定量程。该传感器采用触点阵列封装技术（LGA），即在底面制作有阵列状态钽电极触点的封装技术。装配时插入插座即可。L3GD20 三轴数字输出陀螺仪有很宽的工作温度范围，可以在 -40°C 到 $+85^{\circ}\text{C}$ 的温度环境中保持很高的可靠性。

下面是 L3GD20 三轴数字输出陀螺仪的总体特征。

- I²C /SPI 数字输出接口
- 16 bit 速率的数据输出，8 bit 温度数据输出。
- 2 条数字输出线路。
- 集成低-高频滤波器，滤波带宽可调。
- 嵌入温度传感器
- 嵌入先进先出的数据缓存器（FIFO）
- 高抗震性能
- 符合 RoHS 标准

4.3.2.1、传感器参数（供电电压 3.0V，25 摄氏度条件下测试）

机械参数

参数	测试条件	最小值	标准值	最大值	单位
测量范围	可选范围		±250		Dps
			±500		
			±2000		
灵敏度	FS = 250 dps		8.75		mdps/digit
	FS = 500 dps		17.50		
	FS = 2000 dps		70		
灵敏度随温度变化率	从-40℃到 85℃		±2		%
零飘	FS = 250 dps		±10		dps
	FS = 500 dps		±15		
	FS = 2000 dps		±75		
温飘	FS = 250 dps		±0.03		dps/℃
	FS = 2000 dps		±0.04		dps/℃
非线性度	理想直线		0.2		%FS
噪声密度率			0.03		dps / (Hz) ^{0.5}
输出速率			95/190/ 380/760		Hz
工作温度范围		-40		+85	℃

表 6.6 传感器参数

这些数据是在 3.0V 电压下测量的，在表中有该元件的适应电压范围。

图 6.14 传感器轴向特性

引脚	名称	功能
1	Vdd_IO (1)	为 I/O 串口供电
2	SCL	I ² C 串行时钟 (SCL)
	SPC	SPI 串口时钟 (SPC)
3	SDA	I ² C 串行数据 (SDA)
	SDI	SPI 串行数据输入端口 (SDI)
	SD0	3 线串行数据输出端口 (SD0)
4	SD0	SPI 串行数据输出端口 (SD0)
	SA0	I ² C 数据地址的低有效位 (SA0)
5	CS	I ² C / SPI 模式选择 (1: SPI 闲置模式 / I ² C 连接模式; 0: SPI 连接模式 / I ² C 不可用)
6	DRDY/INT2	数据准备 / FIFO 中断 (水印/ 溢出/ 空闲)
7	INT1	可编程中断端口
8	Reserved	连接到 GND 的端口
9	Reserved	连接到 GND 的端口
10	Reserved	连接到 GND 的端口
11	Reserved	连接到 GND 的端口
12	Reserved	连接到 GND 的端口
13	GND	提供标准电平 (0V)
14	Reserved	通过一个陶瓷电容连接到 GND 的端口 (2)

引脚	名称	功能
15	Reserved	连接到 Vdd 的端口
16	Vdd (3)	供电端口

表 6.8 传感器引脚功能

要求 100 nF 滤波电路电容。

在 11V 的电压偏置条件下必须保证至少 1nF 的电容。

要求 100 nF - 110nF 的电容。

■ 封装描述

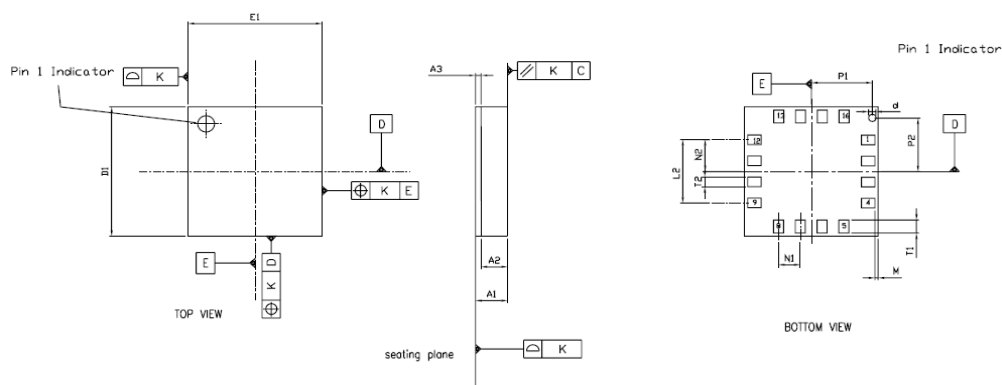


图 6.15 传感器封装图

尺寸						
标注	毫米			英寸		
	Min	Typ	Max	Min	Typ	Max
A1			1.000			0.0394
A2		0.785			0.0309	
A3		0.200			0.0079	

尺寸						
标注	毫米			英寸		
	Min	Typ	Max	Min	Typ	Max
D		0.300			0.0118	
D1	3.850	4.000	4.150	0.1516	0.1575	0.1634
E1	3.850	4.000	4.150	0.1516	0.1575	0.1634
L2		1.950			0.0768	
M		0.100			0.0039	
N1		0.650			0.0256	
N2		0.975			0.0384	
P1		1.750			0.0689	
P2		1.525			0.0600	
T1		0.400			0.0157	
T2		0.300			0.0118	
K		0.050			0.0020	

表 6.9 传感器封装尺寸

4.3.3、MS5611

用途：MS5611 是由瑞士的 MEAS 公司推出的具有 SPI 和 I²C 总线接口的新一代高分辨率气压传感器，可以输出 24 位的气压值和温度值。MS5611 的分辨率为 10cm，能够用于制作高度计和气压计，它可以搭载在手机、手表等移动设备上，还能应用于气压系统和室内导航等领域。

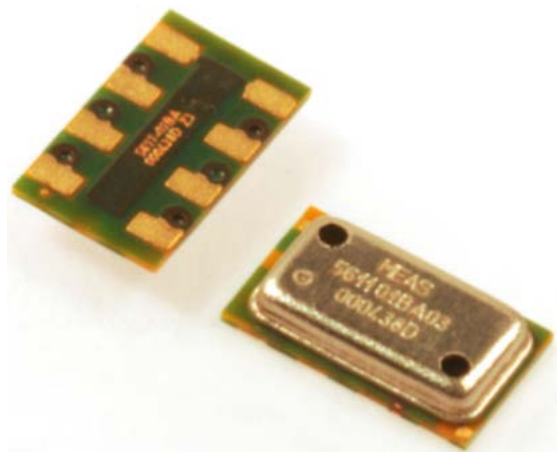


图 6.16 MS5611 气压计传感器

■ 特征

高分辨率，分辨率为 10cm。

响应时间快，为 1ms。

功耗低，工作电流为 1 μ A （待机状态下小于 0.15 μ A）。

工作范围为：气压是 10 至 1200mbar。

具有 SPI 和 I²C 总线接口。

■ 传感器参数

项	环境	最小	标准	最大	单位
供电电压		1.8	3.0	3.6	V
工作温度		-40	+25	+85	°C
存储温度		-40		+125	°C
过压				6	bar

项	环境	最小	标准	最大	单位
最大焊接温度	最长 时间 40s			250	°C
VDD 电容	VDD 对 GND	100			nF

表 6.10 传感器电气参数

■ 封装

MS5611 芯片采用的是表面贴装型 QFN（方形扁平无引脚封装）封装技术，芯片大小为 $(5.0 \times 3.0 \times 1.0) \text{ mm}^3$ 。其封装方框图如下所示：

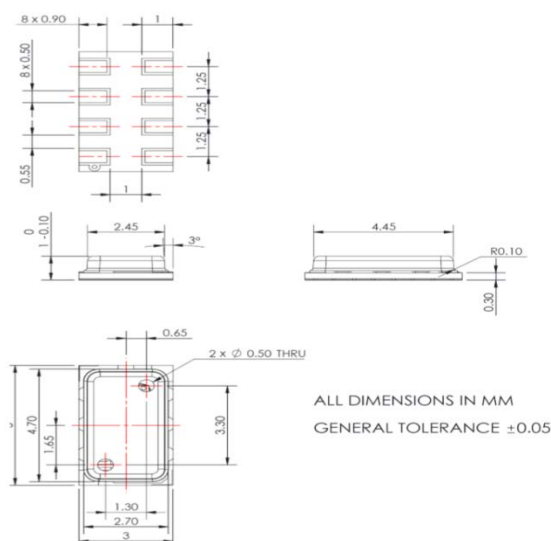


图 6.17 传感器封装

■ 引脚定义

引脚	名称	类型	功能
1	VDD	P	电源正极
2	PS	I	协议选择

引脚	名称	类型	功能
			PS 为高时接入 I ² C 总线 PS 为低时接入 SPI 总线
3	GND	G	接地
4	CSB	I	芯片选择（低电平有效），内部连接
5			
6	SDO	O	串行数据输出
7	SDI/SDA	I/I ² O	串行数据输入/I ² C 总线数据输出输入
8	SCLK	I	串行数据时钟信号

表 6.11 引脚定义

引脚示意图如下

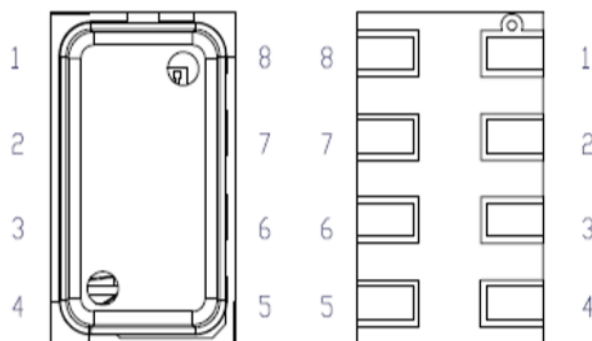


图 6.18 传感器引脚顺序

4.3.4、HMC5883

■ 用途：

霍尼韦尔 HMC5883L 是一种表面贴装的高集成模块，并带有数字接口的弱磁传感器芯片，应用于低成本罗盘和磁场检测领域。HMC5833L 使用了霍尼韦尔的各向异性磁阻技术，它具有为了测量方向和地球磁场而设计的有着非常低的十字

轴灵敏度的固态结构。HMC5883L 可以应用在包括手机、上网本、消费类电子产品、汽车导航系统和个人导航设备等上面。



■ 特征：

- 12-bit ADC 与低干扰 AMR 传感器，能在±8 高斯的磁场中实现 2 毫高斯的分辨率
- 内置自检功能
- 低电压工作 (2.16~3.6V) 和超低功耗 (100uA)
- 内置驱动电路
- I²C 数字接口
- 能让罗盘航向精度精确到 1° ~2°

■ 电气指标：

在 25°C 的条件下测试，另有说明除外

特性	条件	最小	标准	最大	单位
供电电源					
供电	VDD 参考 AGND	2.16	2.5	3.6	V

特性	条件	最小	标准	最大	单位
电压	VDDIO 参考 DGND	1.71	1.8	VDD+0.1	V
平均 电流 损耗	闲置模式				
	测量模式 (7.5Hz ODR)				
	没有应用测量平均值, 即 设置 MA1:MA0=00)	-	2	-	μA
	VDD=2.5V, VDDIO=1.8V (双 电源模式)	-	100	-	μA
	VDD=VDDIO=2.5V (单电源 模式)				
性能					
磁场 量程	满量程 (FS) - 全部施加磁 场 (典型)	-8		+8	高斯 (gauss)
磁动 态范 围	3-bit 增益控制	± 1		± 8	高斯
灵敏 度 (增 益)	VDD=3.0, GN=0 到 7, 12-bit ADC	230		1370	LSb/gauss
数字 解析 度	VDD=3.0, GN=0 到 7, 1-LSb, 12-bit ADC	0.73		4.35	毫高斯
噪声 基数	VDD=3.0V, GN=0 无测量平 均值, 标准偏差 100 个样 本		2		毫高斯

特性	条件	最小	标准	最大	单位
线性	±2.0 高斯输入范围			0.1	±%满量程
磁滞	±2.0 高斯输入范围		±25		ppm
横轴 灵敏 性	测试条件：横向区域 =0.5gauss Happlied=±3gauss		±20		%/FS/gauss
输出 速率	连续测量模式	0.75		75	Hz
	单一测量模式			160	Hz
测量 周期	从接受指令到数据准备		6		Ms
启动 时间	I ² C 指令准备时间		200		μs
	测量的相似回路准备		50		ms
增益 公差	所有增益/动态范围设置		±5		%
I ² C 地址	8-bit 读取地址		0x3D		hex
	8-bit 写入地址		0X3C		hex
I ² C 率	由I ² C 主机控制			400	kHz
I ² C 滞后 性	施密特触发器脉冲输入在 SCL 和 SDA 上的滞后性-下 降 (VDDIO=1.8V)		0.2*VDDIO		Volts
	上升 (VDDIO=1.8V)		0.8*VDDIO		Volts
自测	X&Y 轴		±1.16		高斯
	Z 轴		±1.08		高斯

特性	条件	最小	标准	最大	单位
	X&Y&Z 轴 (GN=5) 正偏置	243		575	LSb
	X&Y&Z 轴 (GN=5) 负偏置	-575		-243	LSb
常规					
ESD 电压	人体模式 (所有引脚)			2000	V
	机器模式 (所有引脚)			750	V
工作温度	一般环境	-30		85	°C
存储温度	常规环境, 无偏置	-40		125	°C

表 6.12 2 传感器参数

4.3.4.1、封装

三轴磁阻传感器和 ASIC 都被封装在 $3.0 \times 3.0 \times 0.9\text{mm}$ LCC 表面装配中。

HMC5883L 封装图 (16 引脚 LPCC, 单位: 毫米) 如下图所示

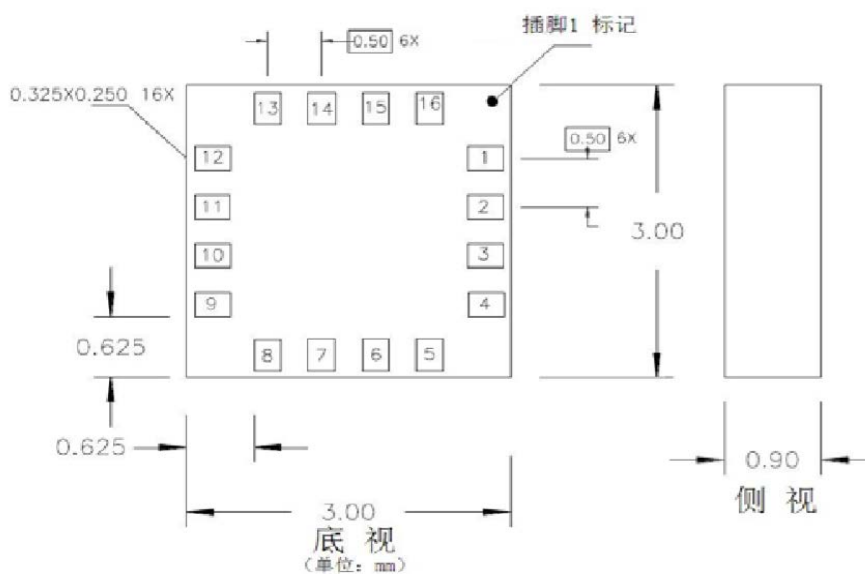


图 6.19 传感器封装

4.3.4.2、引脚定义

引脚	名称	描述
1	SCL	串行时钟-I ² C 总线主/从时钟
2	VDD	电源 (2.16V-3.6V)
3	NC	无连接
4	S1	连接 VDDIO
5	NC	无连接
6	NC	无连接
7	NC	无连接
8	SETP	置位/复位带正-S/R 电容 (C2) 连接
9	GND	接地
10	C1	存储电容器 (C1) 连接
11	GND	接地
12	SETC	S/R (C2) 连接-驱动端
13	VDDIO	IO 电源供应 (1.7V-VDD)
14	NC	无连接
15	DRDY	数据准备, 中断引脚。内部被拉高。选项为连接, 当数据位于输出寄存器上时会在低电位上停 250 μs
16	SDA	串行数据-I ² C 总线主/从数据

表 6.13 传感器引脚功能

4.3.4.3、引脚图

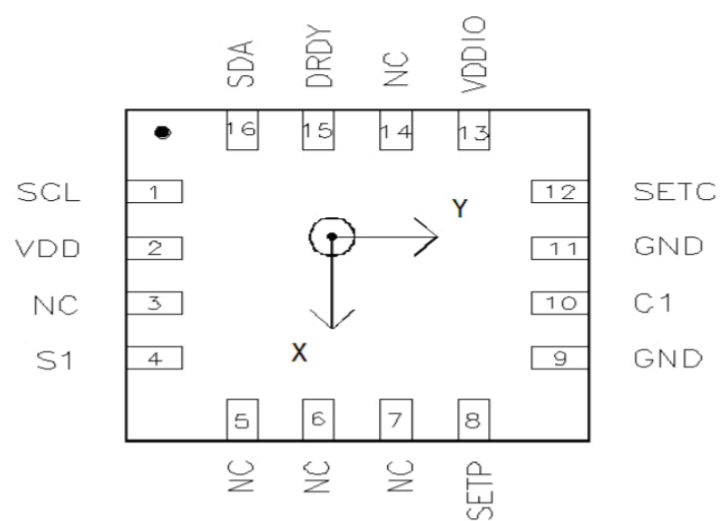


图 6.20 传感器引脚顺序和功能

第二部分

E360 开发

要点提示： PIXHAWK 整体架构的认识

PIXHAWK 原生码 RCS 分析

PIXHAWK 硬件构架

PIXHAWK PX4FMU 和 PX4IO 最底层启动过程分析

PIXHAWK 姿态与控制

飞行模式切换

DEMO 程序讲解



学前提示：

PX4 是一个软、硬件开源项目（遵守 BSD 协议），目的在于为学术、爱好和工业团体提供一款低成本高性能的高端自驾仪。这个项目源于 ETH Zurich（苏黎世联邦理工大学）的计算机视觉与几何实验室、自主系统实验室和 自动控制实验室 的 PIXHAWK 项目，一些出色的个人(Contact and Credits)也参与其中，包括 3D Robotics 和 international 3DR distributors 的成员。

电气连接说明：

1、超声波控制板与飞控连接

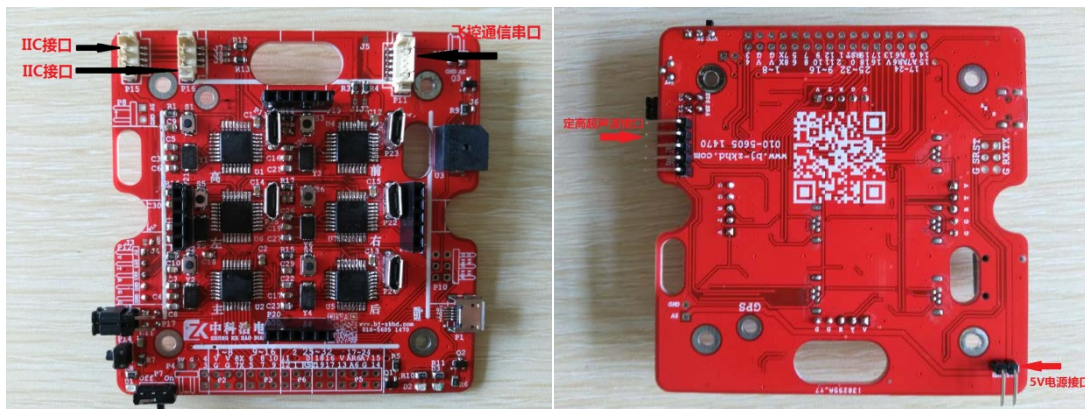


图 1.1 超声波控制板接口

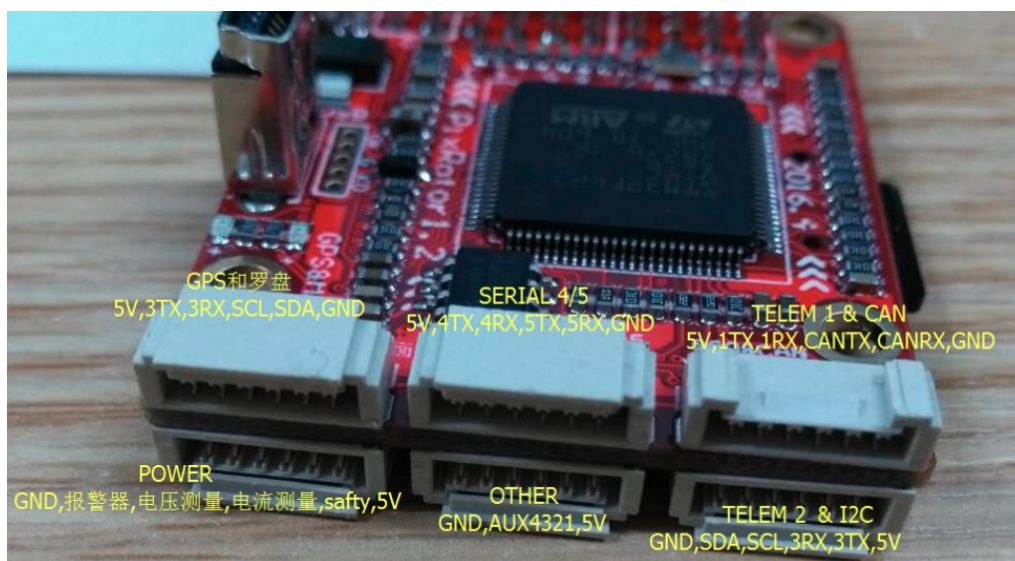


图 1.2 飞控接口定义

2、定高超声波与超声波控制板连接：将定高超声波的 VCC 与控制板定高超声波接口的 V 连接，Trig/Tx 与 T，Echo/Rx 与 R，GND 与 G，GND 与 G 相连接。

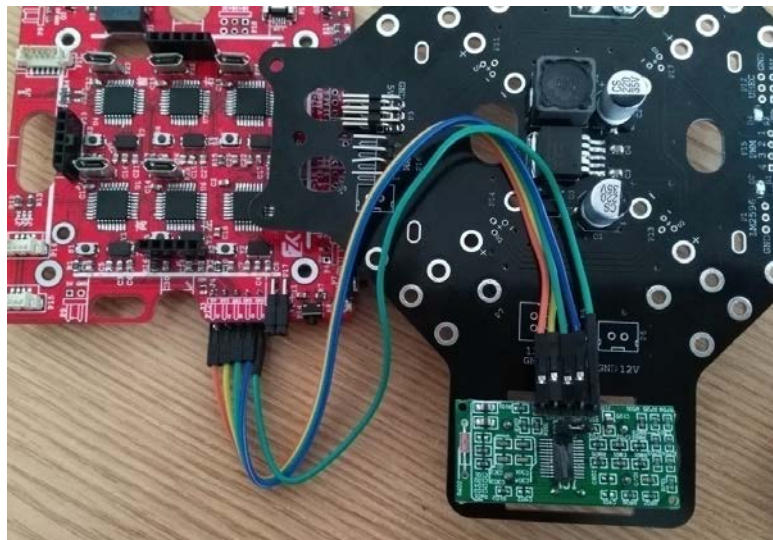


图 1.3 定高超声波与控制板连线

3、控制板与飞控连线：先将光流与 2 个 IIC 接口的任意一个相连接，将另一个 IIC 接口与飞控的 TELEM2&I2C 接口相连。

控制板 IIC 线序：由上向下为 VCC、SCL、SDA、GND

飞控 TELEM2&I2C 线序：从左向右为 GND、SDA、SCL，最右侧为 VCC 其中 VCC 左侧两根线没用上。

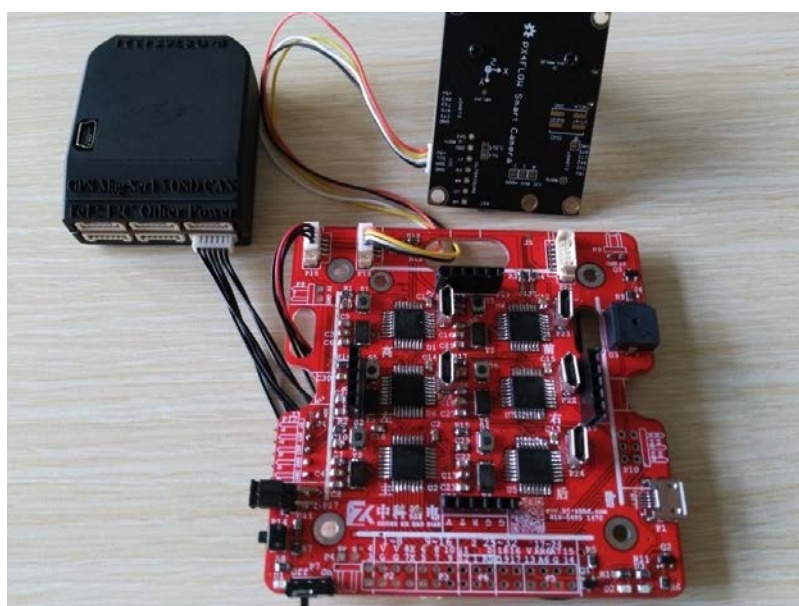


图 1.4 控制板与飞控连线-1

4、将控制板的飞控通信串口与飞控的 SRRIAL4/5 连接

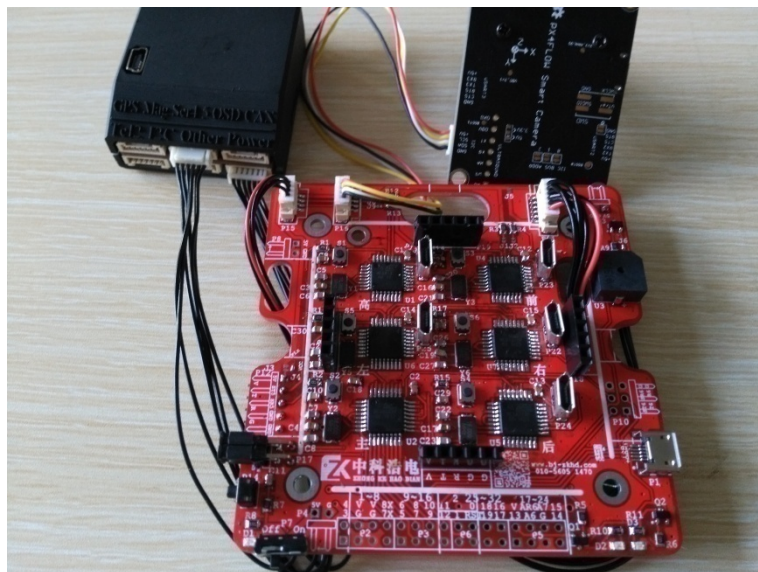


图 1.5 控制板与飞控连线-2

5、飞控与接收机连接

将接收机最左侧排针与飞控 RC 连接，接收机最左侧从上到下依次为信号、电源正、电源负。



图 1.6 飞控与接收机连接

第一章 Pixhawk 整体架构的认识

Pixhawk 自动驾驶仪软件可分为三大部分：实时操作系统、中间件和飞行控制栈。

1.1、NuttX 实时操作系统

NuttX 是一个小型实时嵌入式开源操作系统 (RTOS)，它有一个小巧的内核可以在小型 (8 位) 至中型 (32 位) 嵌入式微控制器上运行。NuttX 遵循 POSIX 和 ANSI 标准运行。在 pixhawk 中 NuttX 操作系统负责底层的任务调度和提供可移植操作系统接口 (如 `printf()`, `threads`, `/dev/ttyS1`, `open()`, `write()`, `poll()`, `ioctl()`) 等。

1.2、Pixhawk 飞行控制栈

PX4 中间件运行于操作系统之上，提供设备驱动和一个微对象请求代理 (micro object request broker, uORB) 用于驾驶仪上运行的单个任务之间的异步通信。Px4 被 3DR 开源后，整个代码结构被大改，原先的系统被摒弃，进而采用 NuttX，但是核心思想没变——为了简化开发而采用牺牲部分效率的消息传递机制，这是 Px4 与 ArduPilot 最本质的差别。

1.3、Pixhawk 飞行控制栈

飞行控制栈可以使用 PX4 的控制软件栈，也可以使用其他的控制软件，如 APM:Plane、APM:Copter，但必须运行于 PX4 中间件之上。

此部分又可分为

◆ **决策导航部分：**根据飞行器自身安全状态和接收到的命令，决定工作于什么模式，下一步应该怎么做。

◆ **位置姿态估计部分：**根据传感器得到自身的位置和姿态信息，此部分算法含金量最高，算法也相当多。

图 1.2 飞控控制详细流程



图 1.3 飞控整体架构

Pixhawk 的算法在 Firmware/src/modules，驱动在 Firmware/src/drivers

px4 原生固件模块列表：

◆ 系统命令程序

- mavlink - 通过串口发送和接收 mavlink 信息
- sdlog2 - 保存系统日志/飞行数据到 SD 卡
- tests - 测试系统中的测试程序
- top - 列出当前的进程和 CPU 负载
- uORB - 微对象请求代理器-分发其他应用程序之间的信息

◆ 驱动

- mkbldctrl --- bldctrl 电子模块驱动
- esc_calib --- ESC 的校准工具
- fmu --- FMU 引脚输入输出定义
- gpio_led --- GPIO LED 驱动
- gps --- GPS 接收器驱动
- pwm --- PWM 的更新速率命令
- sensors --- 传感器应用
- px4io --- px4io 驱动
- uavcan --- uavcan 驱动
- ◆ 飞行控制的程序
- ◆ 飞行安全和导航
 - commander --- 主要飞行安全状态机
 - navigator --- 任务，失效保护和 RTL 导航仪
- ◆ 估计姿态和位置
 - attitude_estimator_ekf - 基于 EKF 的姿态估计
 - ekf_att_pos_estimator - 基于 EKF 的姿态和位置估计
 - position_estimator_inav - 惯性导航的位置估计
- ◆ multirotor 姿态和位置控制器
 - mc_att_control - multirotor 姿态控制器
 - mc_pos_control - multirotor 位置控制器

◆ fixedwing 姿态和位置控制器

● fw_att_control - 固定翼飞机的姿态控制

● fw_pos_control_l1 - 固定翼位置控制器

◆ 垂直起降姿态控制器

● vtol_att_control - 垂直起降姿态控制器

1.3.1、决策

1.3.1.1、任务决策

以任务为导向，任务决策主要决定多旋翼下一步“去哪儿”，进一步，需要规划路径，使得整个过程能满足诸如：飞向航路点并沿航线飞，以及飞向航路点并避障等要求。

1.3.1.2 健康管理和失效保护

以安全为导向，失效保护主要决定多旋翼下一步“去哪儿”。多旋翼飞行器在飞行前或飞行中，可能会发生通信故障、传感器失效和动力系统异常等，这些意外会直接导致控制任务无法完成。这一部分包括安全问题的介绍、机载设备的健康评估、机载设备的健康监测、失效后的保护建议。完成任务模式切换，同时考虑电池电量、GPS 等传感器是否正常工作等信息。

传感器校正正在 commander 里面。

对应的程序在：Firmware/src/modules/commander

Firmware/src/modules/navigator

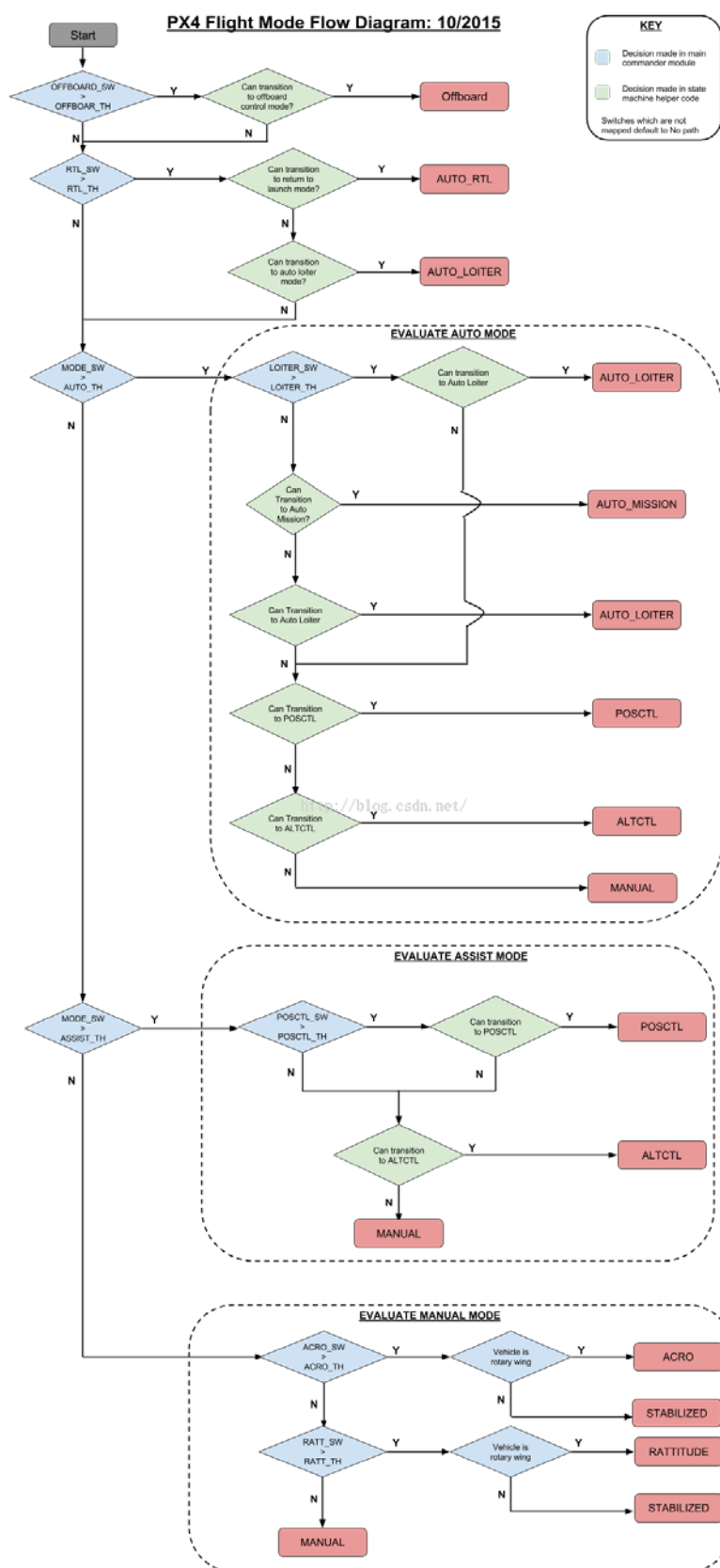
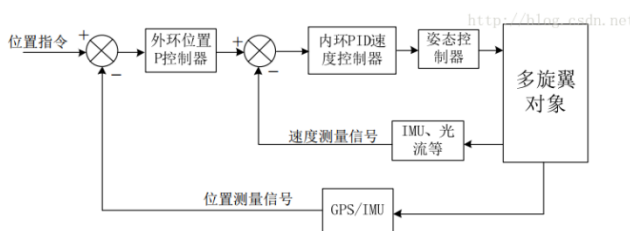
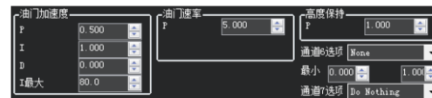
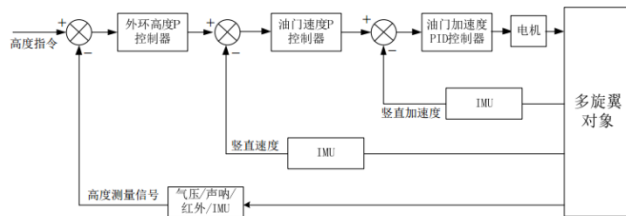
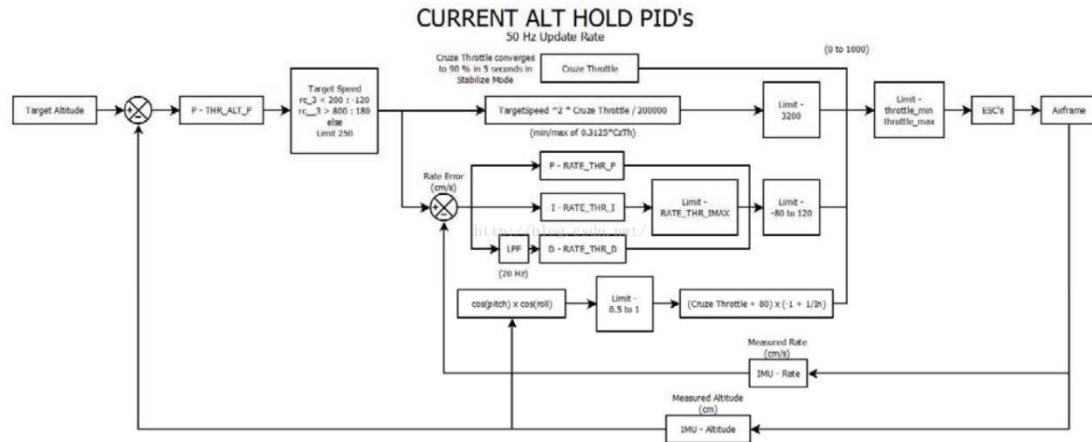
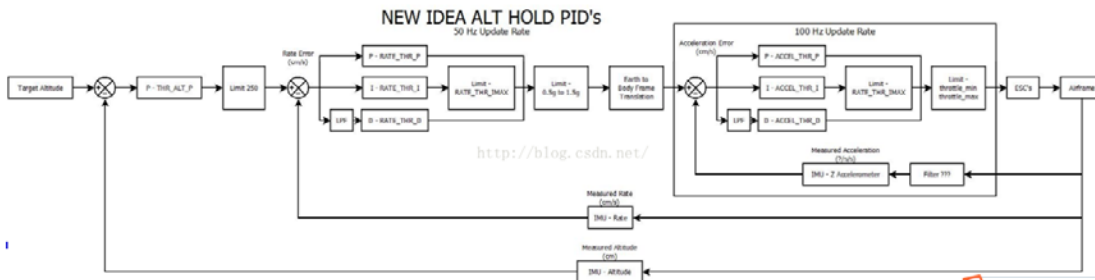


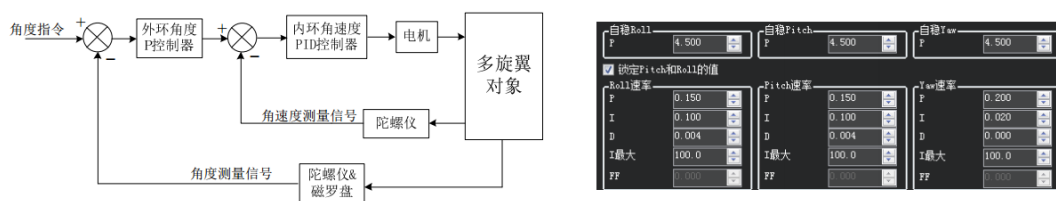
图 1.4 飞行模式图解

1.3.2、位置控制和姿态控制



高度调参在定高模式下，
位置调参在悬停模式下，
调整原则大同小异，从分
理解PID各个参数的作用，
分析飞行现象与响应曲线，
做有针对性地调整





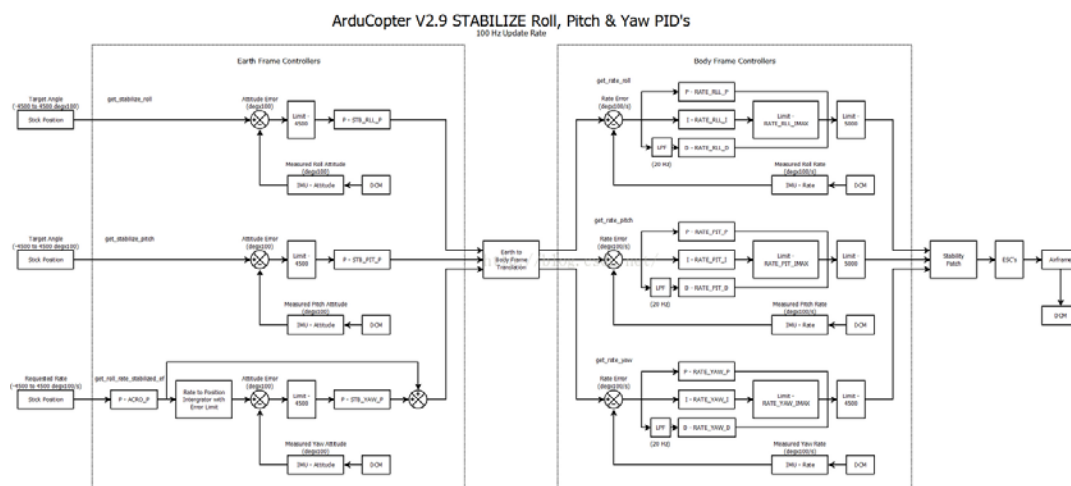
(1) 用飞控自带的参数进行简单试飞（做好安全防护措施），记录飞行操作现象，分析飞控log文件，获取姿态响应曲线
<http://blog.csdn.net/>

(2) 调整内环角速度P增益，使得响应快速，无震荡，无超调。

(3) 略微调整内环角速度I和D增益，进一步优化响应，适当返回去调整速度P增益

(4) 同理调整偏航内环角速度P增益，接着调整内环I和D增益，最后调整外环P增益

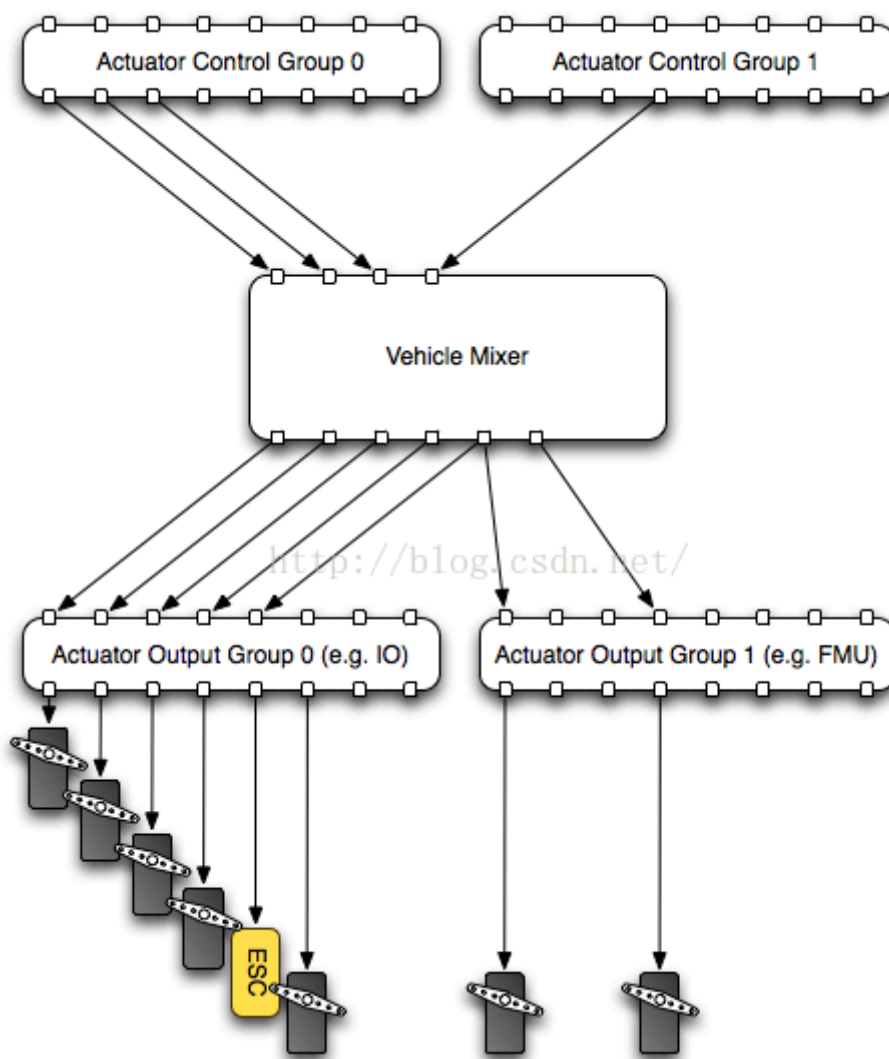
(5) 根据飞行效果，针对性地微调各个系数



1.3.3、PWM 输出

1.3.3.1、Mixer

混合器输出定义控制器的输出是如何映射到电机和伺服输出。所有内置 mixer 文件位于 ROM 文件系统的/etc/mixers 目录，并编译成固件。Mixer 是一组独立的映射器，从控制读入写入，写入执行器输出。一个模块，结合一组根据预先定义的规则和参数的输入，产生一组输出。



(1) 语法:

Mixer 的定义是文本文件，用一个大写字母后跟一个冒号开头的行。所有其他的行都会被忽略，这意味着解释性文本可与定义相混合。每个文件可以定义多个 Mixer。由一个 Mixer 产生的执行器输出的数量是特定的。

起始形式如下:

`<tag>:<mixer arguments>`

标签选择 Mixer 的类型，“M”为一种简单求和 Mixer，“R”为多转子（多旋翼）Mixer 等。

一个空的 Mixer 不消耗控制，并产生一个执行机构的输出，其值始终为零。通常一个空的 Mixer 被用作占位符 Mixer 的集合中，以实现制动器输出的特定模式。

空的 Mixer 定义形式：

Z:

(2) 简单的 Mixer

一个简单的 Mixer 的开始：

M: <control count>

O: <-ve scale> <+ve scale> <offset> <lower limit> <upper limit>

如果为零，则输出就为零，Mixer 输出的固定值是受限制的。

第二行定义了输出输出定标器和标量参数。虽然计算作为浮点运算进行的，存储在所述定义文件中的值是由 10000 倍缩放，-0.5 偏移被编码为-5000.

该定义继续描述控制输入和缩放项：

S: <group> <index> <-ve scale> <+ve scale> <offset> <lower limit>
<upper limit>

这些值定义了控制组从那些定标器中读取数据，和该组内的值的偏移。是特定于设备读取 Mixer 的定义。

当用于混合车辆的控制，混合器组 0 是车辆姿态控制组，索引值从 0 到 3 通常分别是：roll, pitch, yaw, thrust。

具体详解见：[https://pixhawk.org/dev/mixing?s\[\]=mixer](https://pixhawk.org/dev/mixing?s[]=mixer)

(3) 多旋翼 Mixer

在多旋翼混合器结合四个控制输入（roll, pitch, yaw, thrust）为一组用于驱动电机速度控制器制动输出。

Mixer 定义的形式：

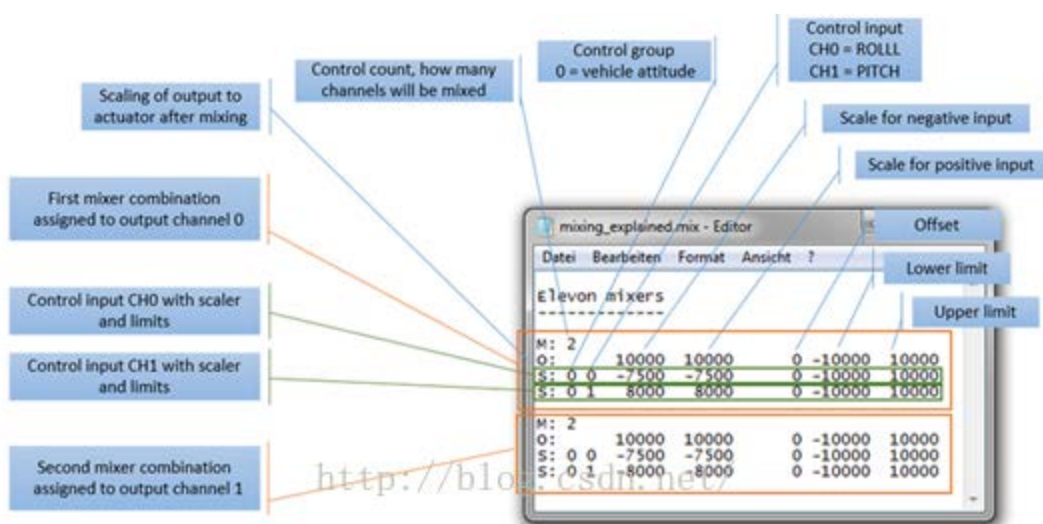
R: <geometry> <roll scale> <pitch scale> <yaw scale> <deadband>

参数解释：

机械：包括 quad（4x, 4+），hex（6x, 6+），octo（8x, 8+）

单独的 roll, pitch 和 yaw 控制因子

电机输出死区



If you wanted to change only the second output, channel-1 (Pixhawk pin 2), you still need an entry for first output, channel-0 (Pixhawk pin 1)

EXAMPLE:

mixers						
M: 1	10000	10000	0	-10000	10000	
O: 0	0	0	0	0	0	
S: 0 0	10000	10000	0	-10000	10000	
M: 2	10000	10000	0	-10000	10000	
O: 0	0	0	0	0	0	
S: 0 0	-7500	-7500	0	-10000	10000	
S: 0 1	-8000	-8000	0	-10000	10000	

Roll has no mix here, but is needed to

1.4、源码框架介绍

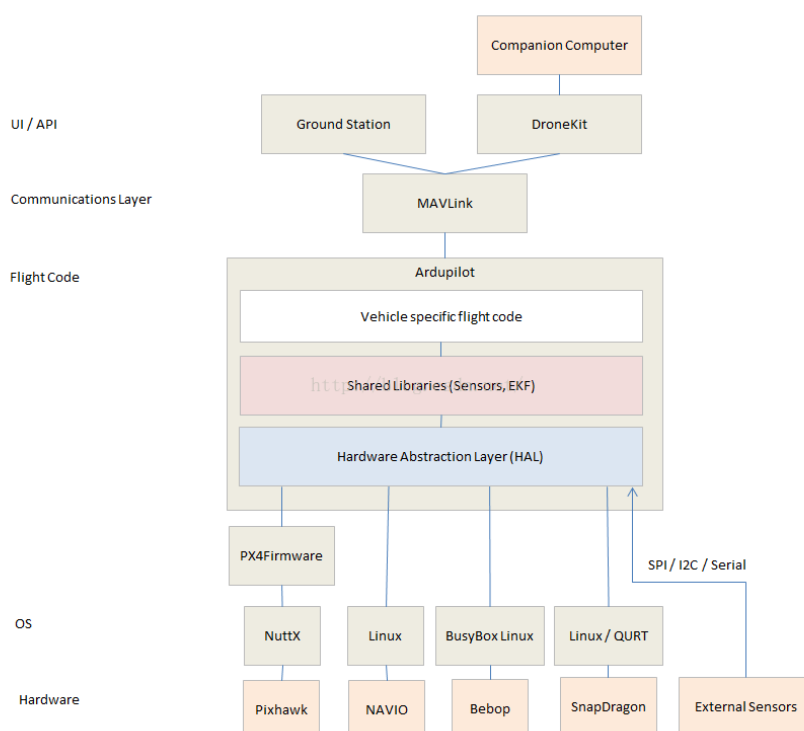


图 1.5 Pixhawk 代码框架

1.4.1、Libraries 介绍

1、核心库

AP_AHRS: 采用 DCM（方向余弦矩阵方法）或 EKF（扩展卡尔曼滤波方法）预估飞行器姿态。

AP_Common: 所有执行文件（sketch 格式，arduino IDE 的文件）和其他库都需要的基础核心库。

AP_Math: 包含了许多数学函数，特别对于矢量运算。

AC_PID: PID 控制器库。

AP_InertialNav: 扩展带有 gps 和气压计数据的惯性导航库。

AC_AttitudeControl: 姿态控制相关库。

AP_WPNav: 航点相关的导航库。

AP_Motors: 多旋翼和传统直升机混合的电机库。

RC_Channel: 更多的关于从 APM_RC 的 PWM 输入/输出数据转换到内部通用单位的库, 比如角度。

AP_HAL, AP_HAL_AVR, AP_HAL_PX4: 硬件抽象层库, 提供给其他高级控制代码一致的接口, 而不必担心底层不同的硬件。AP_HAL_PX4: GPIO、I2C、UART、RCinput/output、scheduler、semaphores、storage。

2、传感器相关库

AP_InertialSensor: 读取陀螺仪和加速度计数据, 并向主程序执行标准程序和提供标准单位数据 (deg/s, m/s)。

AP_RangerFinder: 声呐和红外测距传感器的交互库

AP_Baro: 气压计相关库

AP_GPS: GPS 相关库

AP_Compass: 三轴罗盘相关库

AP_OpticalFlow: 光流传感器相关库

3、其他库

AP_Mount, AP_Camera, AP_Relay: 相机安装控制库, 相机快门控制库

AP_Mission: 从 eeprom (电可擦只读存储器) 存储/读取飞行指令相关库

AP_Buffer: 惯性导航时所用到的一个简单的堆栈 (FIFO, 先进先出) 缓冲区

AP_AccelCal、AP_Declination、AP_RCMapper、AP_RPM、AP_RSSI

AP_ADC: Analog to Digital

APM_Control: pitch/roll/yaw controller

DataFlash: flash memory

GCS_Console/GCS_MAVLink: 地面站通信、飞行日志

第二章 Pixhawk 原生码 rcS 分析

2.1、代码执行流程

1. 编译时将 `cmake/configs/nutttx_px4fmu-v2_default.cmake`（或 `makefiles/config_px4fmu-v2_default.mk`）文件中配置的模块全部编译并烧写到固件中去。

2. 地面站的配置会在 flash 中生成 `/fs/mtd_params` 文件，该文件包含了飞行器的各类信息（机架，校准信息，飞行模式等）。

3. 启动 pixhawk，执行 `/Firmware/ROMFS/px4fmu_common/init.d/rcS`，该文件会读入之前生成的参数文件，进而选择执行哪一个脚本文件。（如选择 DJI450 机架会执行

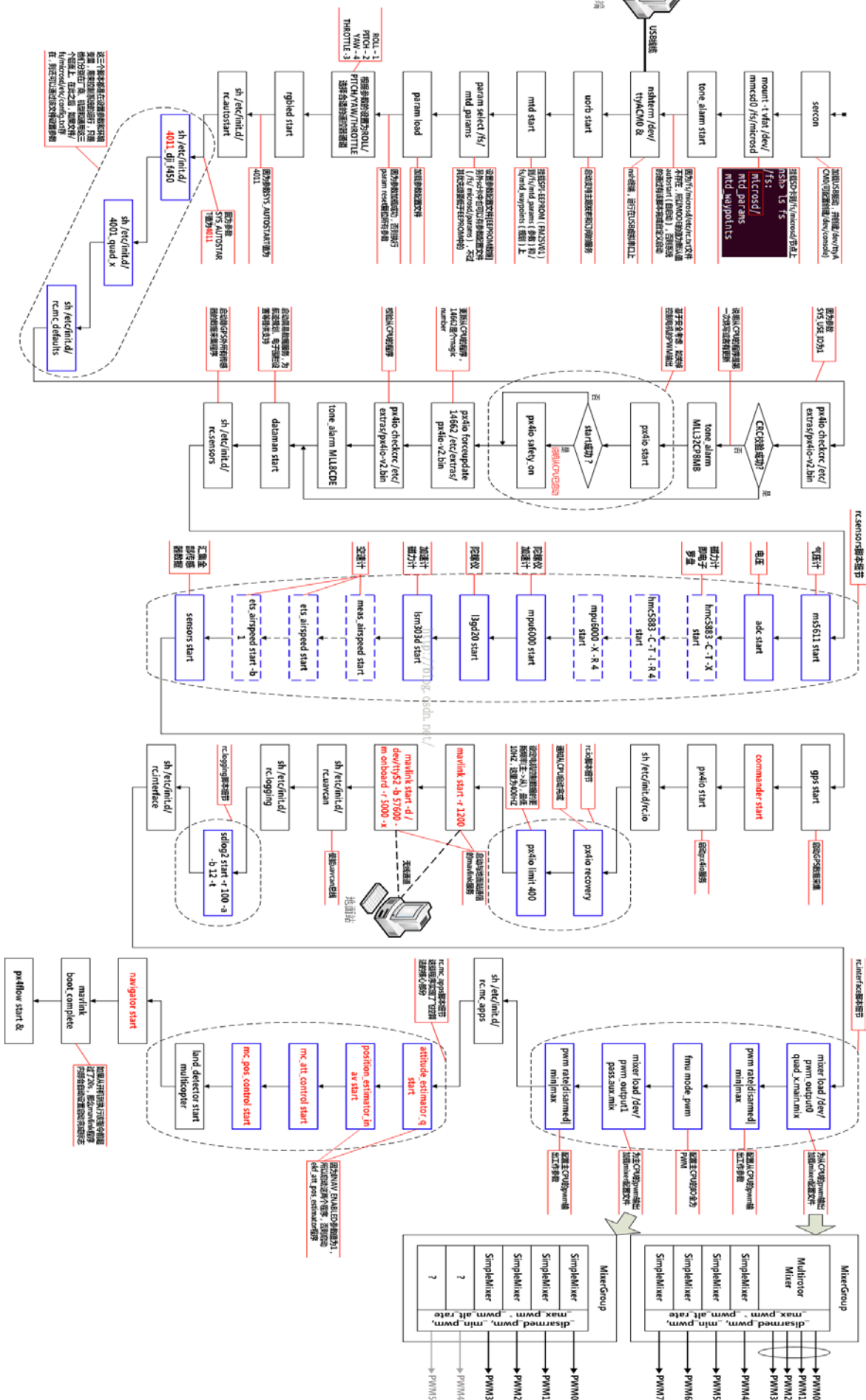
`/Firmware/ROMFS/px4fmu_common/init.d/rc.mc_apps`

和 `/Firmware/ROMFS/px4fmu_common/init.d/4011_dji_f450`），

它们的主要作用为启动飞控所需的各类软件。

4. 不同模块通过 uORB 通信。

下图形象说明启动流程：



2.2、代码注解

以下为代码做了部分注释：

rcS:

```
1  #!nsh
2  #
3  # PX4FMU startup script.
4  #
5  # NOTE: COMMENT LINES ARE REMOVED BEFORE STORED IN ROMFS.
6  #
7
8  #
9  # Start CDC/ACM serial driver
10 #
11 sercon
12
13 #
14 # Default to auto-start mode.    //一些默认设置
15 #
16 set MODE autostart
17
18 set TUNE_ERR ML<<CP4CP4CP4CP4
19 set LOG_FILE /fs/microsd/bootlog.txt
20
```

```
21 #
22 # Try to mount the microSD card. //挂载SD卡
23 #
24 # REBOOTWORK this needs to start after the flight control loop
25 if mount -t vfat /dev/mmcSD0 /fs/microsd
26 then
27     echo "[i] microSD mounted: /fs/microsd"
28     # Start playing the startup tune
29     tone_alarm start
30 else
31     tone_alarm MBAGP
32     if mkfatfs /dev/mmcSD0
33     then
34         if mount -t vfat /dev/mmcSD0 /fs/microsd
35         then
36             echo "[i] microSD card formatted"
37         else
38             echo "[i] format failed"
39             tone_alarm MNBG
40             set LOG_FILE /dev/null
41         fi
42     else
43         set LOG_FILE /dev/null
44     fi
```

```
45 fi
46
47 #
48 # Look for an init script on the microSD card.
49 # Disable autostart if the script found.
50 #
51 set FRC /fs/microsd/etc/rc.txt
52 if [ -f $FRC ] //如果找到初始化脚本，则不自动启动
53 then
54     echo "[i] Executing script: $FRC"
55     sh $FRC
56     set MODE custom
57 fi
58 unset FRC
59
60 if [ $MODE == autostart ] //自动启动过程
61 then
62
63     #
64     # Start the ORB (first app to start) //启动uORB
65     #
66     uorb start
67
68     #
```

```
69      # Load parameters          //下载上位机传到sd卡上的参数（机型什么的）
70      #
71      set PARAM_FILE /fs/microsd/params
72      if mtd start      //启动mtd
73      then
74          set PARAM_FILE /fs/mtd_params
75      fi
76
77      param select $PARAM_FILE
78      if param load      //下载参数
79      then
80          echo "[param] Loaded: $PARAM_FILE"
81      else
82          echo "[param] FAILED loading $PARAM_FILE"
83
84          if param reset
85          then
86              fi
87          fi
88      #
89      # Start system state indicator    //启动系统状态指示
90      #
91      if rgbled start      //启动rgbled
92      then
```

```
93     else
94         if blinkm start          //启动blinkm(闪光)
95             then
96                 blinkm systemstate
97             fi
98         fi
99
100     # Currently unused, but might be useful down the road
101     #if pca8574 start
102     #then
103     #fi
104
105     #
106     # Set AUTOCNF flag to use it in AUTOSTART scripts //设置AUTOCNF标志
107     #
108     if param compare SYS_AUTOCONFIG 1
109     then
110         # Wipe out params except RC*
111         param reset_nostart RC*
112         set AUTOCNF yes
113     else
114         set AUTOCNF no
115     fi
116
```

```
117  #
118  # Set default values //设置默认值
119  #
120  set HIL no
121  set VEHICLE_TYPE none
122  set MIXER none
123  set MIXER_AUX none
124  set OUTPUT_MODE none
125  set PWM_OUT none
126  set PWM_RATE none
127  set PWM_DISARMED none
128  set PWM_MIN none
129  set PWM_MAX none
130  set PWM_AUX_OUT none
131  set PWM_AUX_RATE none
132  set PWM_ACHDIS none
133  set PWM_AUX_DISARMED none
134  set PWM_AUX_MIN none
135  set PWM_AUX_MAX none
136  set FAILSAFE_AUX none
137  set MK_MODE none
138  set FMU_MODE pwm
139  set AUX_MODE pwm
140  set MAVLINK_F default
```

```
141     set EXIT_ON_END no
142
143     set MAV_TYPE none
144
145     set FAILSAFE none
146
147     set USE_I0 yes
148
149     #
150
151     # Set USE_I0 flag          //设置USE_I0标志
152
153     #
154
155     if param compare SYS_USE_I0 1
156
157     then
158
159         if ver hwcmp PX4FMU_V4          //比较px4版本
160
161         then
162
163             set USE_I0 no
164
165         fi
166
167     fi
168
169     if ver hwcmp MINDPX_V2
170
171     then
172
173         set USE_I0 no
174
175     fi
176
177     else
178
179         set USE_I0 no
180
181     fi
182
183
184     # should set to 0.8 for mindpx-v2 board.
```



```
165     if param compare INAV_LIDAR_ERR 0.5
166 then
167     if ver hwcmp MINDPX_V2
168 then
169         param set INAV_LIDAR_ERR 0.8
170         param save
171     fi
172 fi
173
174 #
175 # Set parameters and env variables for selected AUTOSTART
176 #
177 if param compare SYS_AUTOSTART 0
178 then
179     echo "[i] No autostart"
180 else
181     sh /etc/init.d/rc.autostart //执行rc.autostart
182 fi
183 unset MODE
184
185 #
186 # Wipe incompatible settings for boards not having two outputs
187 if ver hwcmp PX4FMU_V4
188 then
```

```
189     set MIXER_AUX none
190 fi
191
192 #
193 # Override parameters from user configuration file //根据用户配置文件
194                                     //重写参数
195 #
196 set FCONFIG /fs/microsd/etc/config.txt
197 if [ -f $FCONFIG ]
198 then
199     echo "[i] Custom config: $FCONFIG"
200     sh $FCONFIG
201 fi
202 unset FCONFIG
203
204 #
205 # If autoconfig parameter was set, reset it and save parameters
206 #
207 if [ $AUTOCNF == yes ] //如果AUTOCNF == yes, 那么复位并保存参数
208 then
209     param set SYS_AUTOCONFIG 0
210     param save
211 fi
212 unset AUTOCNF
```

```
213
214     set IO_PRESENT no
215
216     if [ $USE_IO == yes ]      //px4的版本为v2，所以条件满足
217     then
218         #
219         # Check if PX4IO present and update firmware if needed //比较PX4IO，
220                                         //判断是否升级
221         #
222         if [ -f /etc/extras/px4io-v2.bin ]    //选择PX4IO-v2固件
223         then
224             set IO_FILE /etc/extras/px4io-v2.bin
225         else
226             set IO_FILE /etc/extras/px4io-v1.bin
227         fi
228
229         if px4io checkcrc ${IO_FILE} //crc检验是否是最新版本
230         then
231             echo "PX4IO CRC OK" >> $LOG_FILE
232
233             set IO_PRESENT yes
234         else
235             echo "PX4IO Trying to update" >> $LOG_FILE
236
```

```
237     tone_alarm MLL32CP8MB
238
239     if px4io start    //启动px4io
240     then
241         # try to safe px4 io so motor outputs dont go crazy
242         if px4io safety_on    //判断px4io是否安全
243         then
244             # success! no-op
245         else
246             # px4io did not respond to the safety command
247             px4io stop
248         fi
249     fi
250
251     if px4io forceupdate 14662 ${IO_FILE}
252     then
253         usleep 500000
254         if px4io checkcrc $IO_FILE    //是否强制升级
255         then
256             echo "PX4IO CRC OK after updating" >> $LOG_FILE
257             tone_alarm MLL8CDE
258
259             set IO_PRESENT yes
260         else
```

```
261         echo "ERROR: PX4IO update failed" >> $LOG_FILE
262     tone_alarm $TUNE_ERR
263 fi
264 else
265     echo "ERROR: PX4IO update failed" >> $LOG_FILE
266     tone_alarm $TUNE_ERR
267 fi
268 fi
269 unset IO_FILE
270
271 if [ $IO_PRESENT == no ] //没有找到px4io
272 then
273     echo "[i] ERROR: PX4IO not found"
274     echo "ERROR: PX4IO not found" >> $LOG_FILE
275     tone_alarm $TUNE_ERR
276 fi
277 fi
278
279 #
280 # Set default output if not set //如果不设置，则为默认设置none
281 #
282 if [ $OUTPUT_MODE == none ]
283 then
284     if [ $USE_I0 == yes ] //px4的版本为v2，所以条件满足
```

```
285     then
286         set OUTPUT_MODE io
287     else
288         set OUTPUT_MODE fmu
289     fi
290 fi
291
292 if [ $OUTPUT_MODE == io -a $IO_PRESENT != yes ]
293 then
294 # Need IO for output but it not present, disable output //需要IO口输
295                                     //出，但是它不存在
296     set OUTPUT_MODE none
297     echo "[i] ERROR: PX4IO not found, disabling output"
298
299     # Avoid using ttyS0 for MAVLink on FMUv1
300     if ver hwcmp PX4FMU_V1 //比较px4的版本
301     then
302         set FMU_MODE serial //设置FMU_MODE为serial模式
303     fi
304 fi
305
306 if [ $OUTPUT_MODE == ardrone ]
307 then
308     set FMU_MODE gpio_serial
```

```
309     fi
310
311     if [ $HIL == yes ]      //比较HIL参数设置
312     then
313         set OUTPUT_MODE hil
314         if ver hwcmp PX4FMU_V1
315         then
316             set FMU_MODE serial
317         fi
318         unset HIL
319     else
320         unset HIL
321         gps start
322     fi
323
324     # waypoint storage      //存储路径
325     # REBOOTWORK this needs to start in parallel
326     if dataman start
327     then
328         fi
329
330     #
331     # Sensors System (start before Commander so Preflight checks are properly run)
332     #//起飞前传感器检查
```

```
333 sh /etc/init.d/rc.sensors //执行rc.sensors (标准的PX4FMU v1, v2, v3传感器
334 //启动脚本)
335
336 # Needs to be this early for in-air-restarts
337 commander start //启动commander, 类似于任务管理器
338
339 #
340 # Start primary output
341 #
342 set TTYS1_BUSY no
343
344 #
345 # Check if UAVCAN is enabled, default to it for ESCs//检查UAVCAN是否使能
346 #
347 if param greater UAVCAN_ENABLE 2 //如果UAVCAN_ENABLE大于2
348 then
349 set OUTPUT_MODE uavcan_esc
350 fi
351
352 # If OUTPUT_MODE == none then something is wrong with setup and we shouldn't try
353 to enable output
354 //如果OUTPUT_MODE == none, 那么有问题, 不能尝试使能输出
355 if [ $OUTPUT_MODE != none ]
356 then
```



```
357     if [ $OUTPUT_MODE == uavcan_esc ]
358     then
359         if param compare UAVCAN_ENABLE 0
360         then
361             echo "[i] OVERRIDING UAVCAN_ENABLE = 1"
362             param set UAVCAN_ENABLE 1
363         fi
364     fi
365
366     if [ $OUTPUT_MODE == io -o $OUTPUT_MODE == uavcan_esc ]
367     then
368         if px4io start
369         then
370             sh /etc/init.d/rc.io //执行rc.io
371         else
372             echo "ERROR: PX4IO start failed" >> $LOG_FILE
373             tone_alarm $TUNE_ERR
374         fi
375     fi
376
377     if [ $OUTPUT_MODE == fmu -o $OUTPUT_MODE == ardrone ]
378     then
379         if fmu mode_$FMU_MODE
380         then
```

```
381         echo "[i] FMU mode_${FMU_MODE} started"
382     else
383         echo "[i] ERROR: FMU mode_${FMU_MODE} start failed"
384         echo "ERROR: FMU start failed" >> $LOG_FILE
385         tone_alarm $TUNE_ERR
386     fi
387
388     if ver hwcmp PX4FMU_V1
389     then
390         if [ $FMU_MODE == pwm -o $FMU_MODE == gpio ]
391         then
392             set TTYS1_BUSY yes
393         fi
394         if [ $FMU_MODE == pwm_gpio -o $OUTPUT_MODE == ardrone ]
395         then
396             set TTYS1_BUSY yes
397         fi
398     fi
399 fi
400
401 if [ $OUTPUT_MODE == mkblctrl ]
402 then
403     set MKBLCTRL_ARG ""
404     if [ $MKBLCTRL_MODE == x ]
```

```
405         then
406             set MKBLCTRL_ARG "-mkmode x"
407         fi
408         if [ $MKBLCTRL_MODE == + ]
409             then
410                 set MKBLCTRL_ARG "-mkmode +"
411             fi
412
413         if mkblctrl $MKBLCTRL_ARG
414             then
415                 echo "[i] MK started"
416             else
417                 echo "[i] ERROR: MK start failed"
418                 echo "ERROR: MK start failed" >> $LOG_FILE
419                 tone_alarm $TUNE_ERR
420             fi
421         unset MKBLCTRL_ARG
422     fi
423     unset MK_MODE
424
425     if [ $OUTPUT_MODE == hil ]
426         then
427             if pwm_out_sim mode_port2_pwm8
428                 then
```

```
429         echo "[i] PWM SIM output started"
430     else
431         echo "[i] ERROR: PWM SIM output start failed"
432         echo "ERROR: PWM SIM output start failed" >> $LOG_FILE
433         tone_alarm $TUNE_ERR
434     fi
435 fi
436
437 #
438 # Start IO or FMU for RC PPM input if needed //如果需要，则启动IO和
439                                     //FMU接收PPM
440 #
441 if [ $IO_PRESENT == yes ]
442 then
443     if [ $OUTPUT_MODE != io ]
444     then
445         if px4io start
446         then
447             echo "[i] PX4IO started"
448             sh /etc/init.d/rc.io
449         else
450             echo "[i] ERROR: PX4IO start failed"
451             echo "ERROR: PX4IO start failed" >> $LOG_FILE
452             tone_alarm $TUNE_ERR
```

```
453         fi
454     fi
455 else
456     if [ $OUTPUT_MODE != fmu -a $OUTPUT_MODE != ardrone ]
457     then
458         if fmu mode_$FMU_MODE
459         then
460             echo "[i] FMU mode_$FMU_MODE started"
461         else
462             echo "[i] ERROR: FMU mode_$FMU_MODE start failed"
463             echo "ERROR: FMU mode_$FMU_MODE start failed" >> $LOG_FILE
464             tone_alarm $TUNE_ERR
465         fi
466     fi
467     if ver hwcmp PX4FMU_V1
468     then
469         if [ $FMU_MODE == pwm -o $FMU_MODE == gpio ]
470         then
471             set TTYS1_BUSY yes
472         fi
473         if [ $FMU_MODE == pwm_gpio -o $OUTPUT_MODE == ardrone ]
474         then
475             set TTYS1_BUSY yes
476         fi
477     fi
```

```
477         fi
478     fi
479 fi
480 fi
481 //以下为mavlink的设置
482     if [ $MAVLINK_F == default ]
483     then
484         # Normal mode, use baudrate 57600 (default) and data rate 1000 bytes/s
485         if [ $TTYs1_BUSY == yes ]
486         then
487             # Start MAVLink on ttyS0, because FMU ttyS1 pins configured as something
488         else
489             set MAVLINK_F "-r 1200 -d /dev/ttyS0"
490
491             # Exit from nsh to free port for mavlink
492             set EXIT_ON_END yes
493         else
494             set MAVLINK_F "-r 1200"
495             # Avoid using ttyS1 for MAVLink on FMUv4
496             if ver hwcmp PX4FMU_V4
497             then
498                 set MAVLINK_F "-r 1200 -d /dev/ttyS1"
499             # Start MAVLink on Wifi (ESP8266 port)
500             mavlink start -r 40000 -m config -b 921600 -d /dev/ttyS0
```

```
501         fi
502     fi
503 fi
504
505     mavlink start $MAVLINK_F
506     unset MAVLINK_F
507
508     #
509     # MAVLink onboard / TELEM2
510     #
511     if ver hwcmp PX4FMU_V1
512     then
513     else
514         # XXX We need a better way for runtime eval of shell variables,
515         # but this works for now
516         if param compare SYS_COMPANION 921600
517         then
518             mavlink start -d /dev/ttyS2 -b 921600 -m onboard -r 80000 -x
519         fi
520         if param compare SYS_COMPANION 57600
521         then
522             mavlink start -d /dev/ttyS2 -b 57600 -m onboard -r 5000 -x
523         fi
524         if param compare SYS_COMPANION 157600
```

```
525     then
526         mavlink start -d /dev/ttyS2 -b 57600 -m osd -r 1000
527     fi
528     if param compare SYS_COMPANION 257600
529     then
530         mavlink start -d /dev/ttyS2 -b 57600 -m magic -r 5000 -x
531     fi
532     if param compare SYS_COMPANION 357600
533     then
534         mavlink start -d /dev/ttyS2 -b 57600 -r 1000
535     fi
536     # Sensors on the PWM interface bank
537     # clear pins 5 and 6
538     if param compare SENS_EN_LL40LS 1
539     then
540         set AUX_MODE pwm4
541     fi
542     if param greater TRIG_MODE 0
543     then
544         # Get FMU driver out of the way
545         set MIXER_AUX none
546         set AUX_MODE none
547         camera_trigger start
548     fi
```



```
549     fi
550
551     # Transitional support: Disable safety on all Pixracer boards
552     if ver hwcmp PX4FMU_V4
553     then
554         if param set CBRK_IO_SAFETY 22027
555         then
556             fi
557         fi
558
559         #
560         # Starting stuff according to UAVCAN_ENABLE value
561         #
562         if param greater UAVCAN_ENABLE 0
563         then
564             if uavcan start
565             then
566                 else
567                     tone_alarm $TUNE_ERR
568                 fi
569             fi
570
571             if param greater UAVCAN_ENABLE 1
572             then
```

```
573     if uavcan start fw
574     then
575     else
576         tone_alarm $TUNE_ERR
577     fi
578 fi
579
580 #
581 # Logging
582 #
583 sh /etc/init.d/rc.logging //执行rc.logging，也就是启动rc.logging
584
585 #
586 # Start up ARDrone Motor interface
587 #
588 if [ $OUTPUT_MODE == ardrone ]
589 then
590     ardrone_interface start -d /dev/ttyS1
591 fi
592
593 #
594 # Fixed wing setup //固定翼的设置
595 #
596 if [ $VEHICLE_TYPE == fw ]
```

```
597     then
598         echo "[i] FIXED WING"
599
600     if [ $MIXER == none ]
601     then
602         # Set default mixer for fixed wing if not defined
603         set MIXER AERT
604     fi
605
606     if [ $MAV_TYPE == none ]
607     then
608         # Use MAV_TYPE = 1 (fixed wing) if not defined
609         set MAV_TYPE 1
610     fi
611
612     param set MAV_TYPE $MAV_TYPE
613
614     # Load mixer and configure outputs
615     sh /etc/init.d/rc.interface //执行rc.interface, 配置控制接口
616
617     # Start standard fixedwing apps
618     sh /etc/init.d/rc.fw_apps//执行rc.fw_apps, 开启姿态和位置估算
619 fi
620
```

```
621 #
622 # Multicopters setup //多旋翼的设置
623 #
624 if [ $VEHICLE_TYPE == mc ]
625 then
626     echo "[i] MULTICOPTER"
627
628     if [ $MIXER == none ]
629     then
630         echo "Mixer undefined"
631     fi
632
633     if [ $MAV_TYPE == none ] //根据用户的选择，设置MAV_TYPE
634     then
635         # Use mixer to detect vehicle type
636         if [ $MIXER == quad_x -o $MIXER == quad_+ ]
637         then
638             set MAV_TYPE 2
639         fi
640         if [ $MIXER == quad_w -o $MIXER == sk450_deadcat ]
641         then
642             set MAV_TYPE 2
643         fi
644         if [ $MIXER == quad_h ]
```

```
645         then
646             set MAV_TYPE 2
647         fi
648         if [ $MIXER == tri_y_yaw- -o $MIXER == tri_y_yaw+ ]
649             then
650                 set MAV_TYPE 15
651             fi
652             if [ $MIXER == hexa_x -o $MIXER == hexa_+ ]
653                 then
654                     set MAV_TYPE 13
655                 fi
656                 if [ $MIXER == hexa_cox ]
657                     then
658                         set MAV_TYPE 13
659                     fi
660                 if [ $MIXER == octo_x -o $MIXER == octo_+ ]
661                     then
662                         set MAV_TYPE 14
663                     fi
664                 if [ $MIXER == octo_cox -o $MIXER == octo_cox_w ]
665                     then
666                         set MAV_TYPE 14
667                     fi
668             fi
```

```
669
670     # Still no MAV_TYPE foundmav的类型没找到
671     if [ $MAV_TYPE == none ]
672     then
673         echo "Unknown MAV_TYPE"
674         param set MAV_TYPE 2
675     else
676         param set MAV_TYPE $MAV_TYPE
677     fi
678
679     # Load mixer and configure outputs
680     sh /etc/init.d/rc.interface //执行rc.interface, 配置控制接口
681
682     # Start standard multicopter apps
683     sh /etc/init.d/rc.mc_apps// 执行rc.mc_apps, 启动姿态位置的估计和控制
684 fi
685
686 #
687 # VTOL setup //工具类型设置
688 #
689 if [ $VEHICLE_TYPE == vtol ]
690 then
691     echo "[init] Vehicle type: VTOL"
692
```

```
693     if [ $MIXER == none ]
694 then
695     echo "Default mixer for vtol not defined"
696 fi
697
698     if [ $MAV_TYPE == none ]
699 then
700     # Use mixer to detect vehicle type
701     if [ $MIXER == caipirinha_vtol ]
702 then
703     set MAV_TYPE 19
704 fi
705     if [ $MIXER == firefly6 ]
706 then
707     set MAV_TYPE 21
708 fi
709     if [ $MIXER == quad_x_pusher_vtol ]
710 then
711     set MAV_TYPE 22
712 fi
713 fi
714
715     # Still no MAV_TYPE found
716     if [ $MAV_TYPE == none ]
```

```
717     then
718         echo "Unknown MAV_TYPE"
719         param set MAV_TYPE 19
720     else
721         param set MAV_TYPE $MAV_TYPE
722     fi
723
724     # Load mixer and configure outputs
725     sh /etc/init.d/rc.interface //执行rc.interface, 下载mixer并配置输出
726
727     # Start standard vtol apps
728     sh /etc/init.d/rc.vtol_apps //执行rc.vtol_apps,
729                                     //启动attitude_estimator_q start、
730                                     //position_estimator_inav start、
731                                     //vtol_att_control start、
732                                     //mc_att_control start、
733                                     //mc_pos_control start、
734                                     //fw_att_control start、
735                                     //fw_pos_control_l1 start
736     fi
737
738     #
739     # Rover setup //漫游设置
740     #
```



```
741     if [ $VEHICLE_TYPE == rover ]
742     then
743         # 10 is MAV_TYPE_GROUND_ROVER
744         set MAV_TYPE 10
745
746         # Load mixer and configure outputs
747         sh /etc/init.d/rc.interface      //执行rc.interface, 下载mixer并配置
748                                         //输出
749
750         # Start standard rover apps
751         sh /etc/init.d/rc.axialracing_ax10_apps  //ekf_att_pos_estimator start
752
753         param set MAV_TYPE 10
754     fi
755
756     #
757     # For snapdragon, we need a passthrough mode
758     # Do not run any mavlink instances since we need the serial port for
759     # communication with Snapdragon.
760     #
761     if [ $VEHICLE_TYPE == passthrough ]
762     then
763         mavlink stop-all
764
```

```
765         # Stop multicopter attitude controller if it is running, the controls come
766         # from Snapdragon.
767         if mc_att_control stop
768         then
769         fi
770
771         # Start snapdragon interface on serial port.
772         if ver hwcmp PX4FMU_V2
773         then
774             # On Pixfalcon use the standard telemetry port (Telem 1).
775             snapdragon_rc_pwm start -d /dev/ttyS1
776             px4io start
777         fi
778
779         if ver hwcmp PX4FMU_V4
780         then
781             # On Pixracer use Telem 2 port (TL2).
782             snapdragon_rc_pwm start -d /dev/ttyS2
783             fmu mode_pwm4
784         fi
785
786         # Arm straightaway.
787         pwm arm
788         # Use 400 Hz PWM on all channels.
```

```
789     pwm rate -a -r 400
790 fi
791
792 unset MIXER
793 unset MAV_TYPE
794 unset OUTPUT_MODE
795
796 #
797 # Start the navigator          //启动导航
798 #
799 navigator start
800
801 #
802 # Generic setup (autostart ID not found) //常规设置
803 #
804 if [ $VEHICLE_TYPE == none ]
805 then
806     echo "[i] No autostart ID found"
807
808 fi
809
810 # Start any custom addons      //启动自定义插件
811 set FEXTRAS /fs/microsd/etc/extras.txt
812 if [ -f $FEXTRAS ]
```

```
813     then
814         echo "[i] Addons script: $FEXTRAS"
815         sh $FEXTRAS
816     fi
817     unset FEXTRAS
818
819     # Run no SD alarm      //运行没有sd卡的警告
820     if [ $LOG_FILE == /dev/null ]
821     then
822         echo "[i] No microSD card found"
823         # Play SOS
824         tone_alarm error
825     fi
826
827 # End of autostart
828 fi
829
830 # There is no further script processing, so we can free some RAM
831 # XXX potentially unset all script variables.
832 unset TUNE_ERR
833
834 # Boot is complete, inform MAVLink app(s) that the system is now fully up and running
835 mavlink boot_complete
836
```

```
837 # Sensors on the PWM interface bank
838 if param compare SENS_EN_LL40LS 1
839 then
840     if pwm_input start //启动pwm输入设置
841     then
842         if ll40ls start pwm
843         then
844             fi
845         fi
846     fi
847
848 if ver hwcmp PX4FMU_V4
849 then
850     frsky_telemetry start -d /dev/ttyS6
851 fi
852
853 if ver hwcmp PX4FMU_V2
854 then
855     # Check for flow sensor - as it is a background task, launch it last
856     px4flow start & //如果检测到光流，则启动
857 fi
858
859 if ver hwcmp MINDPX_V2
860 then
```

```
861     #mindexp also need flow
862     px4flow start &
863 fi
864
865 # Start USB shell if no microSD present, MAVLink else //如果没找到sd卡，则启动
866                                     //usb shell,
867 if [ $LOG_FILE == /dev/null ]                                     //找到了则启动mavlink
868 then
869     # Try to get an USB console
870     nshterm /dev/ttyACM0 &
871 else
872     mavlink start -r 800000 -d /dev/ttyACM0 -m config -x
873 fi
874
875
876 if [ $EXIT_ON_END == yes ]
877 then
878     echo "Exit from nsh"
879     exit
880 fi
881 unset EXIT_ON_EN
```

rc. sensors:

```
1 #!nsh
2 #
3 # Standard startup script for PX4FMU v1, v2, v3 onboard sensor drivers.
4 #
5
6 if ver hwcmp PX4FMU_V1 //比较px4版本
7 then
8     if ms5611 start //气压计
9     then
10     fi
11 else
12     # Configure all I2C buses to 100 KHz as they
13     # are all external or slow
14     fmu i2c 1 100000
15     fmu i2c 2 100000
16
17     if ms5611 -s start
18     then
19     fi
20
21     # Blacksheep telemetry
22     if bst start
```

```
23     then
24     fi
25 fi
26
27 if adc start
28 then
29 fi
30
31 if ver hwcmp PX4FMU_V2    //比较px4版本
32 then
33     # External I2C bus
34     if hmc5883 -C -T -X start//启动电子罗盘hmc5883
35     then
36     fi
37
38     # Internal I2C bus
39     if hmc5883 -C -T -I -R 4 start
40     then
41     fi
42
43     # external MPU6K is rotated 180 degrees yaw
44     if mpu6000 -X -R 4 start //加速度计mpu6000
45     then
46     set BOARD_FMUUV3 true
```



```
47     else
48         set BOARD_FMOV3 false
49     fi
50
51     if [ $BOARD_FMOV3 == true ]
52     then
53         # external L3GD20H is rotated 180 degrees yaw
54         if l3gd20 -X -R 4 start //陀螺仪l3gd20
55         then
56         fi
57
58         # external LSM303D is rotated 270 degrees yaw
59         if lsm303d -X -R 6 start
60         then
61         fi
62
63         # internal MPU6000 is rotated 180 deg roll, 270 deg yaw
64         if mpu6000 -R 14 start
65         then
66         fi
67
68         if hmc5883 -C -T -S -R 8 start
69         then
70         fi
```

```
71
72     if meas_airspeed start -b 2
73     then
74     fi
75
76     else
77     # FMUv2
78     if mpu6000 start
79     then
80     fi
81
82     if l3gd20 start
83     then
84     fi
85
86     if lsm303d start
87     then
88     fi
89     fi
90 fi
91
92 if ver hwcmp PX4FMU_V4
93 then
94     # External I2C bus
```

```
95     if hmc5883 -C -T -X start
96 then
97     fi
98
99     # Internal SPI bus is rotated 90 deg yaw
100    if hmc5883 -C -T -S -R 2 start
101 then
102     fi
103
104    # Internal SPI bus ICM-20608-G is rotated 90 deg yaw
105    if mpu6000 -R 2 -T 20608 start
106 then
107     fi
108
109    # Internal SPI bus mpu9250 is rotated 90 deg yaw
110    if mpu9250 -R 2 start
111 then
112     fi
113 fi
114
115 if ver hwcmp PX4FMU_V1
116 then
117     # FMUv1
118     if mpu6000 start
```

```
119     then
120   fi
121
122   if l3gd20 start
123   then
124   fi
125
126   # MAG selection
127   if param compare SENS_EXT_MAG 2
128   then
129     if hmc5883 -C -I start
130     then
131     fi
132   else
133     # Use only external as primary
134     if param compare SENS_EXT_MAG 1
135     then
136       if hmc5883 -C -X start
137       then
138       fi
139     else
140       # auto-detect the primary, prefer external
141       if hmc5883 start
142       then
```

```
143         fi
144     fi
145 fi
146 fi
147
148 if ver hwcmp MINDPX_V2
149 then
150     if mpu6500 start
151     then
152     fi
153
154     if lsm303d start
155     then
156     fi
157
158     if l3gd20 start
159     then
160     fi
161
162     # External I2C bus
163     if hmc5883 -C -T -X start
164     then
165     fi
166 fi
```

```
167
168 if meas_airspeed start
169 then
170 else
171     if ets_airspeed start
172     then
173     else
174         if ets_airspeed start -b 1
175         then
176         fi
177     fi
178 fi
179
180 if sf10a start
181 then
182 fi
183
184 # Wait 20 ms for sensors (because we need to wait for the HRT and work queue callbacks
185 to fire)
186 usleep 20000
187 if sensors start
188 then
189 fi
```


rc.mc_apps:

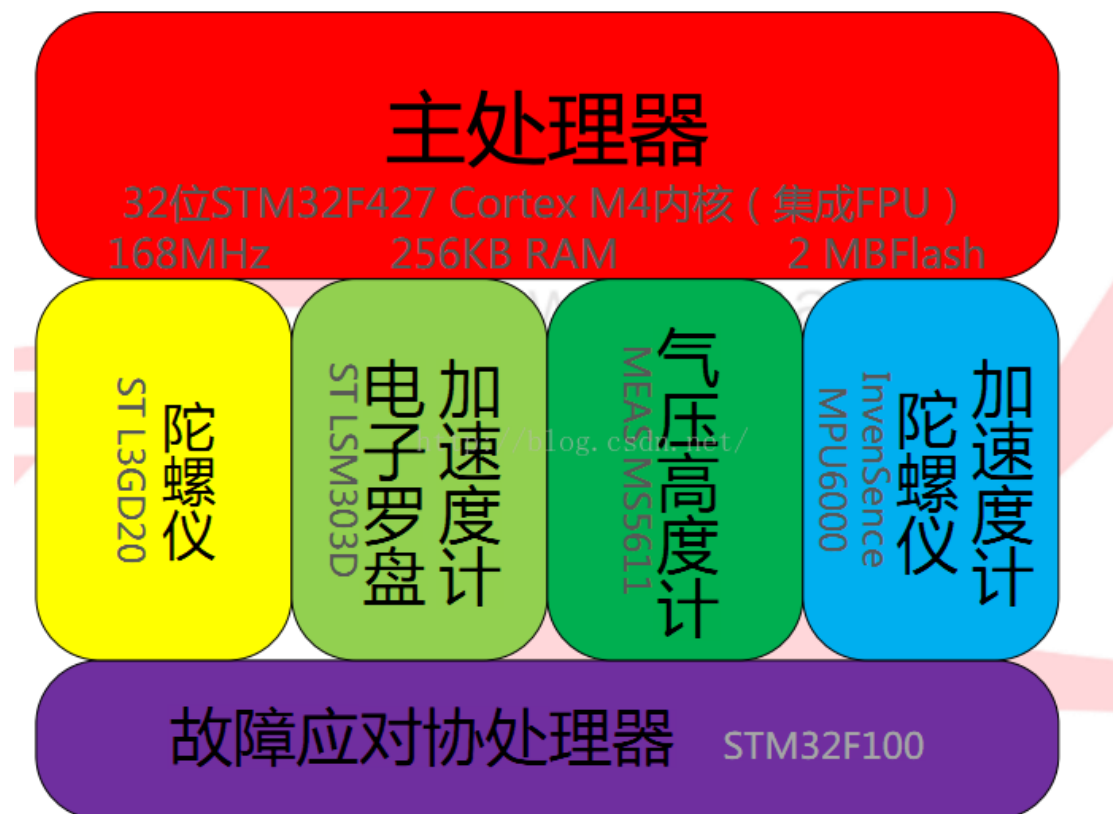
```
1  #!nsh
2  #
3  # Standard apps for multirotors:
4  # att & pos estimator, att & pos control.
5  #
6
7  # The system is defaulting to INAV_ENABLED = 1
8  # but users can alternatively try the EKF-based
9  # filter by setting INAV_ENABLED = 0
10 //选择姿态位置估计和控制使用的算法，默认 INAV_ENABLED = 1，那么
11 //姿态估计 Attitude_estimator_q
12 //位置估计 position_estimator_inav
13 //姿态控制 mc_att_control
14 //位置控制 mc_pos_control
15
16 if param compare INAV_ENABLED 1
17 then
18     attitude_estimator_q start
19     position_estimator_inav start
20 else
21     if param compare LPE_ENABLED 1
22     then
23         attitude_estimator_q start
```



```
24     local_position_estimator start
25
26     else
27         ekf2 start
28     fi
29
30     if mc_att_control start
31     then
32     else
33         # try the multiplatform version
34         mc_att_control_m start
35     fi
36
37     if mc_pos_control start
38     then
39     else
40         # try the multiplatform version
41         mc_pos_control_m start
42     fi
43
44     #
45     # Start Land Detector
46     #
47     land_detector start multicopter
```


第三章 Pixhawk 硬件构架

3.1、Pixhawk 硬件芯片列表



● 处理器

- ◆ STM32F427 VIT6 (168 Mhz/256 KB RAM/2 MB 闪存 100Pin)
- ◆ 32 位 STM32F100C8T6 (48Pin) 故障保护协处理器，平常用来控制输入信号采集 (“Big-Little” 架构)
- ◆ 晶振 24MHz 精度 15ppm 3325 封装

● 传感器

- ◆ InvenSense MPU6000 三轴加速度计/陀螺仪
- ◆ ST Micro L3GD20 16 位陀螺仪

◆ ST Micro LSM303D 14 位加速度计/磁力计

◆ MEAS MS5611 气压计

● 处理

◆ 双 加速度计 MPU6000 1KHz LSM303D 800Hz（消除 1KHz 共振）

◆ 双 陀螺仪 取平均

◆ 双 罗盘 一个内置一个外置，备用

◆ 双 GPS 基于锁定状态和星数选择切换（或更复杂的切换机制、并可对比测试性能）

● 其它

◆ FM25V01 128K 非易失闪存

◆ TXS0108 通用电平驱动芯片（Usart 、舵机输出），隔离/增强驱动，无 TTL 转 232 功能

◆ LTC4417 电源管理芯片

◆ MIC5332 双输出超低压降 LDO

◆ BQ24315 电池管理芯片

◆ TCA62724 三色 LED 灯专用芯片

◆ LT3469 信号采集运放

◆ 可外接 MS4522D 数字差压空速计（6.6V ADC 输入）

3.2、Pixhawk 传感器列表

空速计——IIC 接口

电源管理 Smbus-----IIC

航灯接口-----IIC

BMA180 角速度传感器-----SPI

地磁 HMC5883-----IIC/SPI

L3GD20 陀螺仪-----SPI

LSM303D 角速度/地磁-----SPI

超声波 MB12xx-----IIC

MD25-----IIC

MPU6000-----SPI

MS5611 气压计-----SPI

板载灯控制器 TCA62724FMG-----IIC

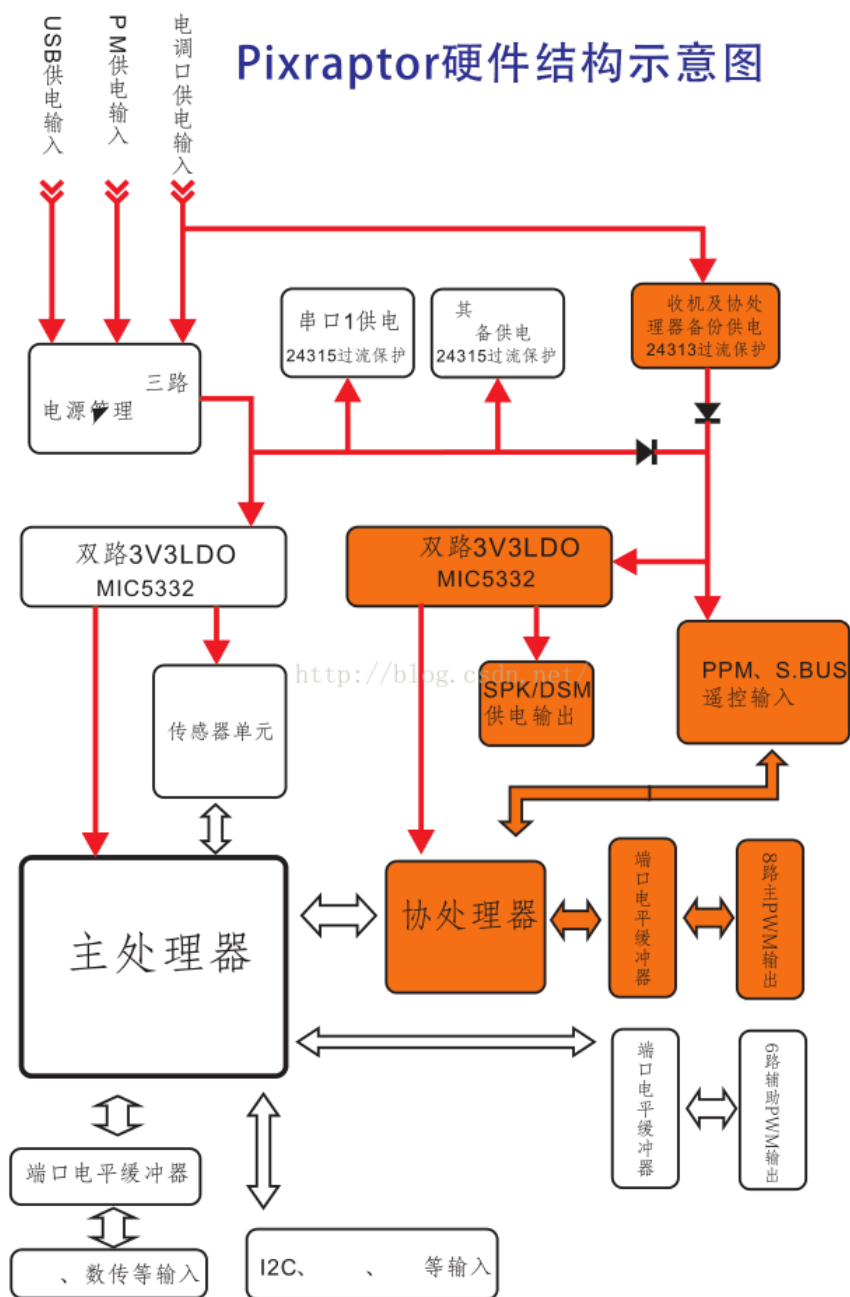
PC8574 (IIC 转并) -----IIC

PCA9685 (IIC/PWM) -----IIC

光流模块-----IIC

3.3、pixhawk 硬件构架

Pixraptor硬件结构示意图



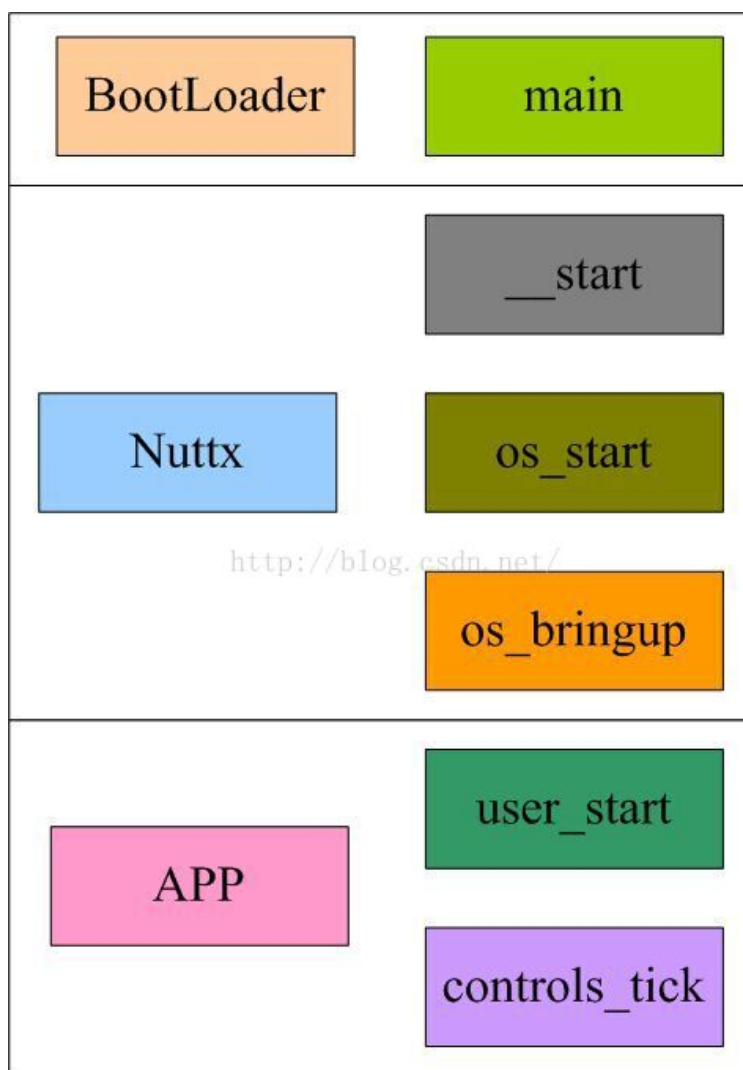
第四章 Pixhawk PX4FMU 和 PX4IO 最底层启动过程分析

4.1、PX4IO 与 PX4FMU

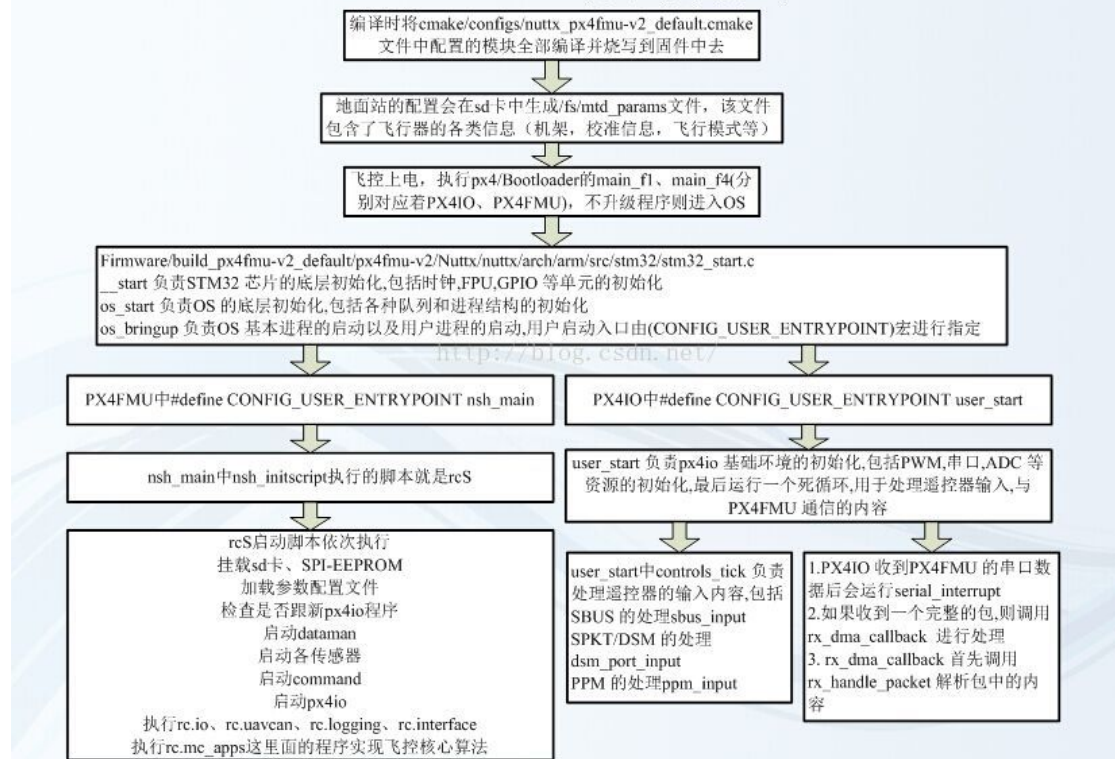
PX4IO (STM32F100)：为 PIXHAWK 中专用于处理输入输出的部分，输入为支持的各类遥控器 (PPM, SPKT/DSM, SBUS)，输出为电调的 PWM 驱动信号，它与 PX4FMU (STM32F427) 通过串口进行通信。

PX4FMU：各种传感器数据读取、姿态解算、PWM 控制量的计算、与 PX4IO 通信。

PX4IO 与 PX4FMU 一样，在大的软件架构上分为 Bootloader, OS 和 APP 三个部分，OS 为 Nuttx，如下图



Pixhawk 启动流程



4.2、Nuttx

代码位置:

PX4Nuttx/nuttx/arch/arm/src/stm32 /stm32_start.c

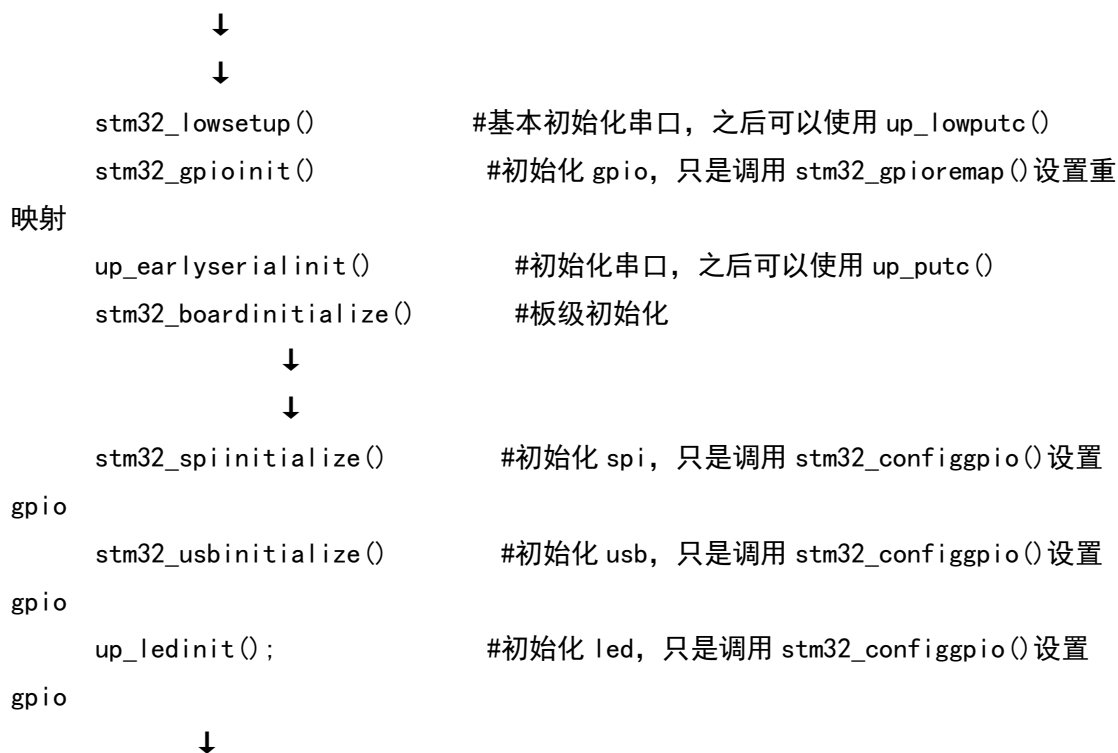
__start-- #处理器执行的第一条指令 (px4 使用的是 stm32, 入口在 stm32_start.c 中)

```

↓
↓
stm32_clockconfig() ----- #初始化时钟
↓
↓
rcc_reset()                  #复位 rcc
stm32_stdclockconfig()       #初始化标准时钟
rcc_enableperipherals()      #使能外设时钟
  
```

```

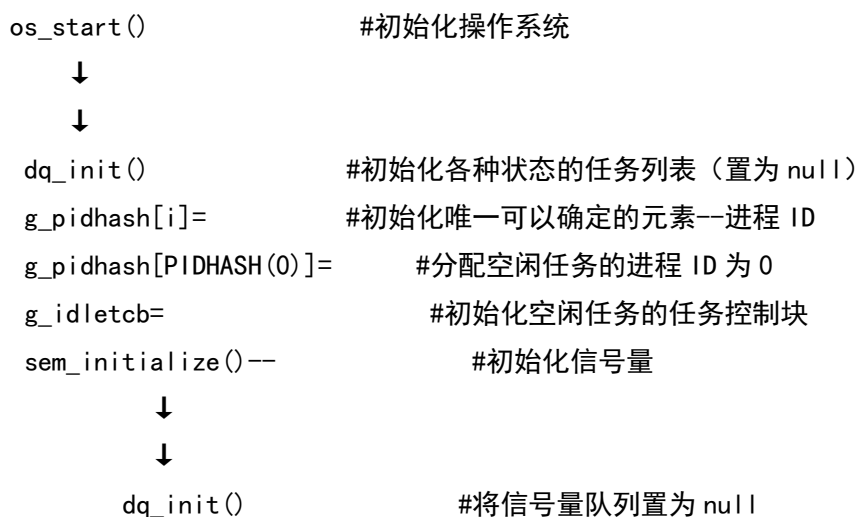
-----
stm32_fpuconfig()            #配置 fpu
  
```

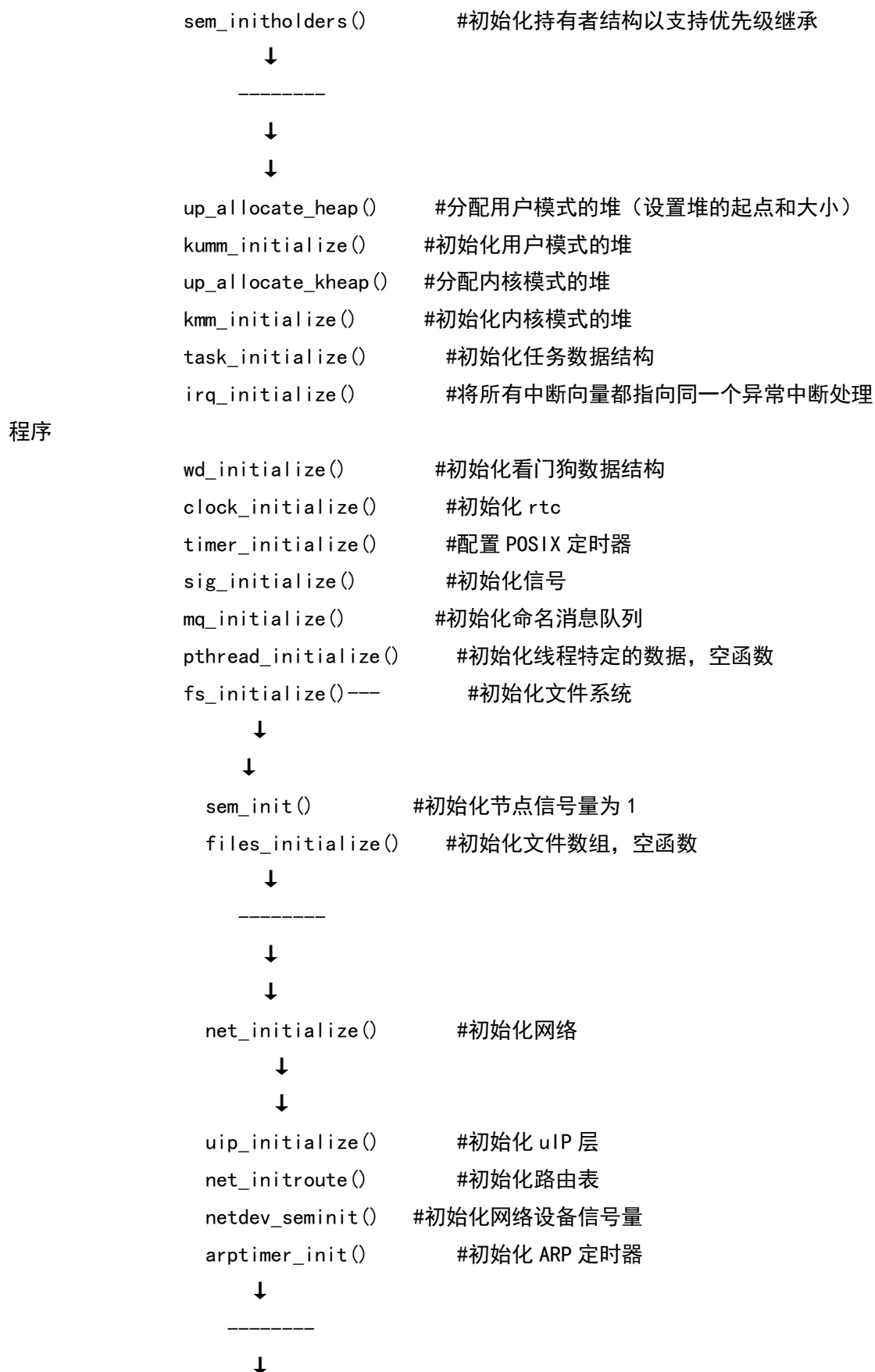


在 stm32_start.c 文件中会看到这么一句话：

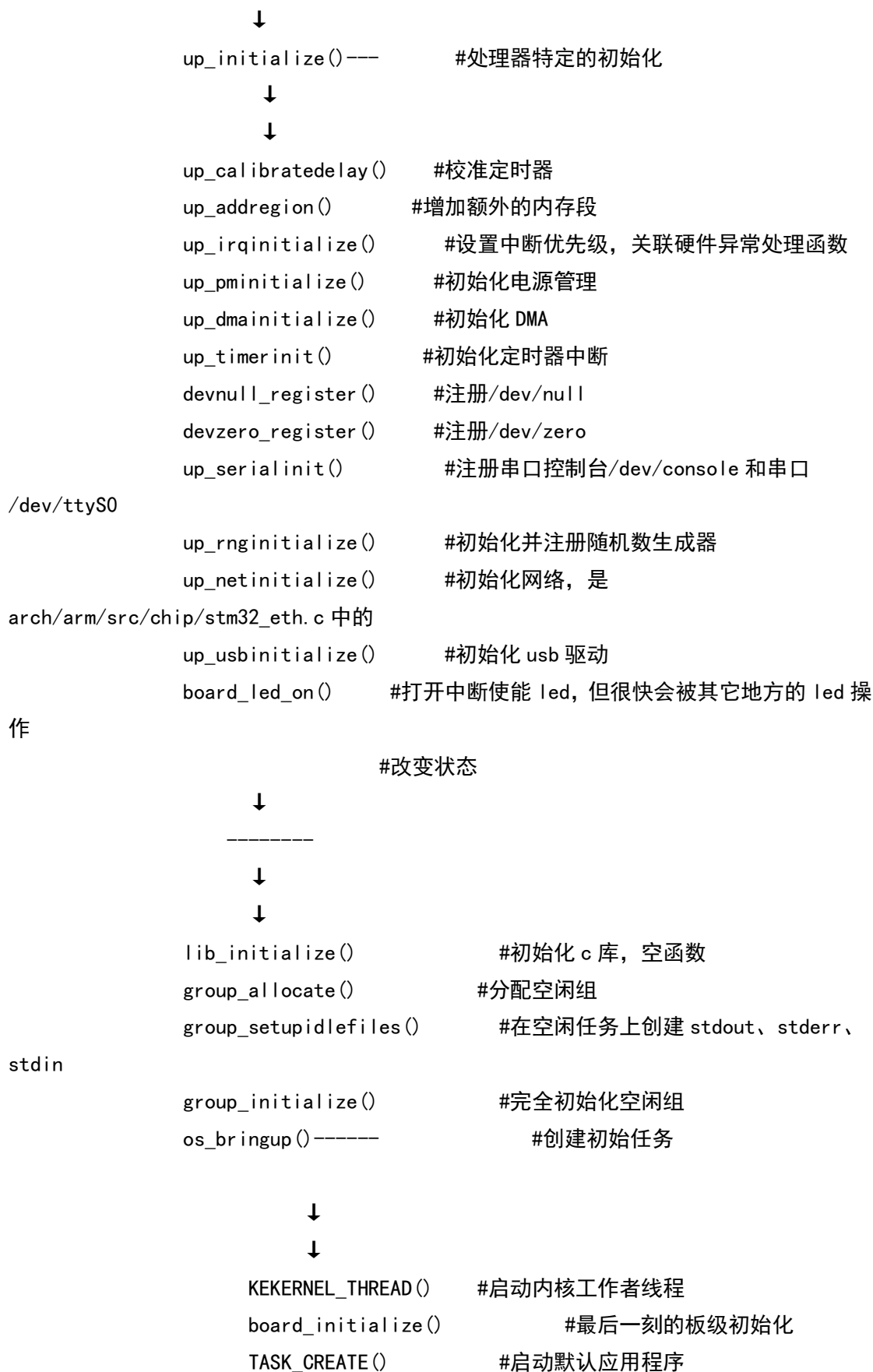
```
/* Then start NuttX */

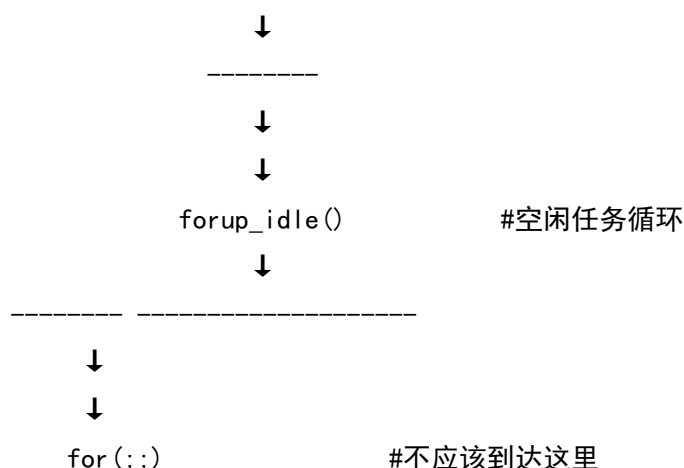
showprogress('\r');
showprogress('\n');
os_start();
```





程序





__start 负责 STM32 芯片的底层初始化, 包括时钟, FPU, GPIO 等单元的初始化;

os_start 负责 OS 的底层初始化, 包括各种队列和进程结构的初始化;

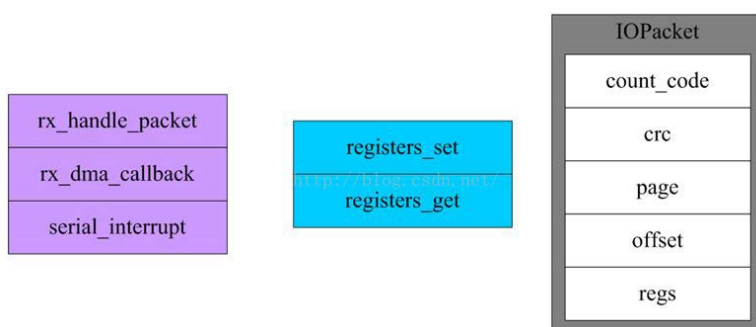
os_bringup 负责 OS 基本进程的启动以及用户进程的启动, 用户启动入口由 (CONFIG_USER_ENTRYPOINT) 宏进行指定。

4.3、APP

user_start (PX4Firmware/src/modules/px4iofirmware/ px4io.c) 负责 px4io 基础环境的初始化, 包括 PWM, 串口, ADC 等资源的初始化, 最后运行一个死循环, 用于处理遥控器输入, 与 PX4FMU 通信的内容。

controls_tick (PX4Firmware/src/modules/px4iofirmware/ controls.c) 负责处理遥控器的输入内容, 包括 SBUS 的处理 sbus_input、 SPKT/DSM 的处理 dsm_port_input、 PPM 的处理 ppm_input。

PX4IO 底层中断处理的内容如下图



紫色为 PX4IO 的底层串口 IO 操作, 流程为当 PX4IO 收到 PX4FMU 的串口数据后会运行 `serial_interrupt`, `serial_interrupt` 负责收发 DMA 的操作, 如果收到一个完整的包, 则调用 `rx_dma_callback` 进行处理, `rx_dma_callback` 首先调用 `rx_handle_packet` 解析包中的内容, 判断为写寄存器还是读寄存器, 处理完成后由 `rx_dma_callback` 发送回包给 PX4FMU。

蓝色为包操作, 只提供 `registers_set` 写操作和 `registers_get` 读操作。

`IOPacket` 为协议包, 包括以下几部分

`count_code` 标记包的读写, 错误, 长度等信息

`crc` 为包的效验码

`page` 为数据页

`offset` 为数据偏移量

`regs` 为数据内容

数据表格如下:

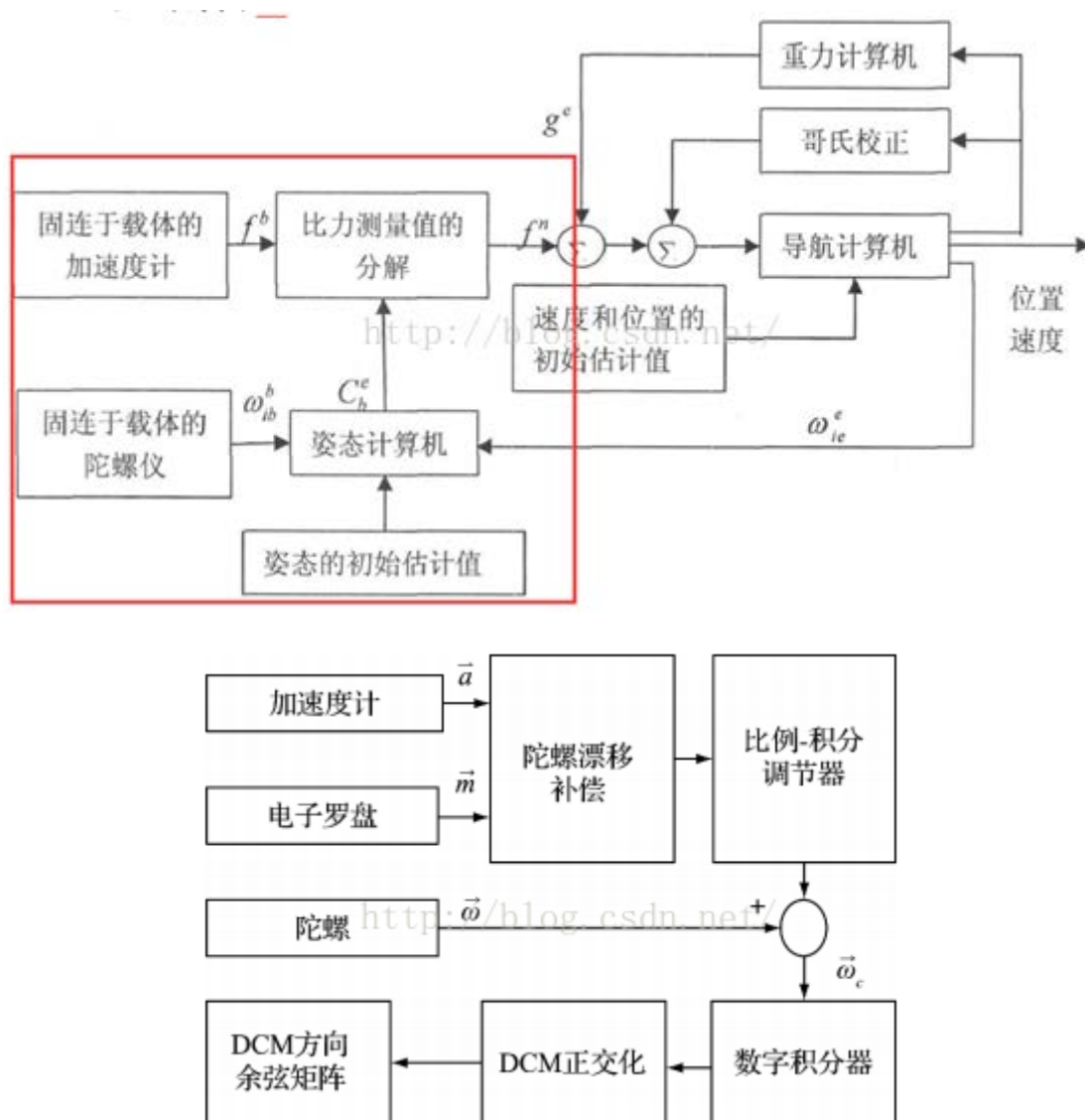
PAGE	定义	描述
0	PX4IO_PAGE_CONFIG	配置信息,包含协议版本,硬件版本,最大 RC 数量,最大 ADC 数量等信息
1	PX4IO_PAGE_STATUS	状态信息,包含 PPM,SBUS 等通道是否有效等信息
2	PX4IO_PAGE_ACTUATORS	

3	PX4IO_PAGE_SERVOS	
4	PX4IO_PAGE_RAW_RC_INPUT	RC 输入状态,包括信号强度等
5	PX4IO_PAGE_RC_INPUT	RC 输入值
6	PX4IO_PAGE_RAW_ADC_INPUT	ADC 采样值
7	PX4IO_PAGE_PWM_INFO	
50	PX4IO_PAGE_SETUP	
51	PX4IO_PAGE_CONTROLS	
52	PX4IO_PAGE_MIXERLOAD	
53	PX4IO_PAGE_RC_CONFIG	RC 配置信息,例如最大值,最小值,死区等
54	PX4IO_PAGE_DIRECT_PWM	
55	PX4IO_PAGE_FAILSAFE_PWM	
56	PX4IO_PAGE_SENSORS	
57	PX4IO_PAGE_TEST	
106	PX4IO_PAGE_CONTROL_MIN_PWM	
107	PX4IO_PAGE_CONTROL_MAX_PWM	
108	PX4IO_PAGE_DISARMED_PWM	

第五章 Pixhawk 姿态与控制

5.1、姿态解算

无人机控制中，主要就是姿态解算和姿态控制部分，该部分主要介绍姿态解算，下面是一张比较好理解的框图。



在姿态解算过程中，姿态表示的方法有很多种，比如欧拉角、四元数、DCM，各有的各的优势，比较常用的就是四元数，方便计算。下面介绍一个它们三个的优缺点。

姿态解算方法的比较：

算法	优点	优点	说明
欧拉角法（3 参数）	1、通过欧拉角微分方程直接解算出 pitch、roll、yaw 2、概念直观，易于理解 3、解算结果永远是正交的，无需再次正交化处理	1、方程中有三角函数的运算，接超越函数有一定的困难 2、当俯仰角接近 90° 时出现退化现象	1、适应于水平姿态角变化不大的情况 2、不适用于全姿态运载体
方向余弦法（9 参数）	1、对姿态矩阵微分方程的求解，避免了欧拉角法中出现的退化现象 2、可以全姿态运行	1、参数量过多，计算量大，实时性不好	很少在工程中使用
四元数法（4 参数）	1、直接求解四元数微分方程 2、只需要求解四个参数，计算量小 3、算法简单，易于操	漂移比较严重	可以在实现过程中修正漂移，应用比较广泛

	作		
--	---	--	--

首先，东北天坐标系是 n 系（地理坐标系，参考坐标系），载体坐标系是 b 系，就是我们飞行器的坐标系。对于四元数法的姿态解算，要求取的就是四元数的值；方向余弦矩阵（用于表示 n 系和 b 系的相对关系）中的元素本来应该是三角函数，这里由于使用四元数法，所以矩阵中的元素就成了四元数。所以姿态解算的任务就是求解由四元数构成的方向余弦矩阵 nCb （ nCb 表示从 b 系到 n 的转换矩阵，同理， bCn 表示从 n 系到 b 的转换矩阵，它们的关系是转置）。

显然，上述矩阵是有误差存在的。对于一个确定的向量 n ，用不同的坐标系表示时，他们所表示的大小和方向一定是相同的。但是由于这两个坐标系的转换矩阵存在误差，那么当一个向量经过这么一个有误差存在的旋转矩阵变换后，在另一个坐标系中肯定和理论值是有偏差的，我们通过这个偏差来修正这个旋转矩阵。这个旋转矩阵的元素是四元数，这就是说修正的就是四元数，于是乎姿态就这样被修正了，这才是姿态解算的原理。姿态解算求的是四元数，是通过修正旋转矩阵中的四元数来达到姿态解算的目的，而不要以为通过加速度计和地磁计来修正姿态，加速度计和地磁计只是测量工具和载体，通过这两个器件表征旋转矩阵的误差存在，然后通过算法修正误差，修正四元数，修正姿态。

加速度计修正 pitch_roll

加速度计可以修正 pitch_roll，但是我们必须要考虑离心加速度，离心加速度就等于旋转率向量（即 gyro vector）和速度向量的叉积（没有原因，平均得来的并且还相当准确，速度可以用 GPS 获取）。我们假设飞机方向和 X 轴平行。

$$A_{\text{centrifugal}} = \omega_{\text{gyro}} \times V$$

$$V = \begin{bmatrix} \text{velocity} \\ 0 \\ 0 \end{bmatrix}$$

Eqn. 25

在机体上测得的重力的为：

$$\mathbf{g}_{\text{reference}} = \text{Accelerometer} + \boldsymbol{\omega}_{\text{gyro}} \times \mathbf{V}$$

$$\text{Accelerometer} = \begin{bmatrix} \text{Accelerometer}_x \\ \text{Accelerometer}_y \\ \text{Accelerometer}_z \end{bmatrix} \quad \text{Eqn. 26}$$

Pitch_roll 的旋转修正向量是由 DCM 的 Z 行与归一化以后的重力参考向量的叉积。

$$\text{RollPitchCorrectionPlane} = \begin{bmatrix} r_{zx} \\ r_{zy} \\ r_{zz} \end{bmatrix} \times \mathbf{g}_{\text{reference}} \quad \text{Eqn. 27}$$

在 n 系中，加速度计输出为，经过 bCn（用四元数表示的转换矩阵）转换之后到 b 系中的值为；在 b 系中，加速度计的测量值为，现在和均表示在 b 系中的竖直向下的向量，由此，我们来做向量积（叉积），得到误差，利用这个误差来修正 bCn 矩阵，于是四元数就在这样一个过程中被修正了。但是，由于加速度计无法感知 z 轴上的旋转运动，所以还需要用地磁计来进一步补偿。

介绍一下 $[vx, vy, vz]^T$ 的计算过程：

$$C_b^n = \begin{bmatrix} q_0^2 + q_1^2 - q_2^2 - q_3^2 & 2(q_1q_2 - q_0q_3) & 2(q_1q_3 + q_0q_2) \\ 2(q_1q_2 + q_0q_3) & q_0^2 - q_1^2 + q_2^2 - q_3^2 & 2(q_2q_3 - q_0q_1) \\ 2(q_1q_3 - q_0q_2) & 2(q_2q_3 + q_0q_1) & q_0^2 - q_1^2 - q_2^2 + q_3^2 \end{bmatrix}$$

重力加速度归一化： $g_n = [0, 0, 1]^T$

转换到 b 系之后的三轴分量为： $g_b = [vx, vy, vz]^T = C_b^n \cdot g_n = C_b^n \cdot [0, 0, 1]^T$

最后计算结果为： $g_b = [2(q_1q_3 - q_0q_2), 2(q_2q_3 + q_0q_1), q_0^2 - q_1^2 - q_2^2 + q_3^2]^T$ ；即用四元数表示的 DCM 的第三列元素。

然后就是 $[ex, ey, ez]^T$ 的求取过程，这个主要就靠向量的叉积，例如两个向量 A 和 B，它们的叉积计算公式为：

$$\begin{aligned} (\mathbf{A} \times \mathbf{B})_x &= A_y B_z - A_z B_y \\ (\mathbf{A} \times \mathbf{B})_y &= A_z B_x - A_x B_z \\ (\mathbf{A} \times \mathbf{B})_z &= A_x B_y - A_y B_x \end{aligned} \quad \text{Eqn. 10}$$

加速度计在静止时测量的是重力加速度，是有大小和方向的；同理，地磁计同样测量的是地球磁场的大小和方向，只不过这个方向不再是竖直向下，而是与 x 轴（或者 y 轴）呈一个角度，与 z 轴呈一个角度。记作，假设 x 轴对准北边，所以 $b_y=0$ ，即。倘若知道 b_x 和 b_z 的精确值，那么就可以采用和加速度计一样的修正方法来修正。只不过在加速度计中，在 n 系中的参考向量是，变成了地磁计的。如果我们知道 b_x 和 b_z 的精确值，那么就可以摆脱掉加速度计的补偿，直接用地磁计和陀螺仪进行姿态解算，但是你看过谁只用陀螺仪和地磁计进行姿态解算吗？没有，因为没人会去测量当地的地磁场相对于东北天坐标的夹角，也就是 b_x 和 b_z （插曲：关于这个 b_x 和 b_z 的理解：可以对比重力加速度的理解，就像 v_x v_y v_z 似的，因为在每一处的归一化以后的重力加速度都是 0 0 1 然后旋转到机体坐标系，而地球每一处的地磁大小都不一样的，不能像重力加速度那样直接旋转得到了，只能用磁力计测量到的数据去强制拟合。）。那么现在怎么办？前面已经讲了，姿态解算就是求解旋转矩阵，这个矩阵的作用就是将 b 系和 n 正确的转化直到重合。现在我们假设 nCb 旋转矩阵是经过加速度计校正后的矩阵，当某个确定的向量（ b 系中）经过这个矩阵旋转之后（到 n 系），这两个坐标系在 XOY 平面上重合（参考 DCM IMU:Theory 的 Drift cancellation 部分），只是在 z 轴旋转上会存在一个偏航角的误差。下图表示的是经过 nCb 旋转之后的 b 系和 n 系的相对关系。可以明显发现加速度计可以把 b 系通过四元数法从任意角度拉到与 n 系水平的位置上，此时只剩下一个偏航角误差。这也是为什么加速度计无法修正偏航的原因。

现在我们反过来从 b 系推往 n 系：设地磁计在 b 系中的输出为 $[mx, my, mz]^T$ ，经过 nCb1 旋转之后得到 $[hx, hy, hz]^T$ (n 系)。在这个 XOY 平面上 (n 系)， $[bx, 0, bz]^T$ 的投影为 $\sqrt{bx*bx}$ ， $[hx, hy, hz]^T$ 的投影为 $\sqrt{hx*hx + hy*hy}$ 。显然，地磁计在 XOY 平面上 (n 系) 的向量的大小必定相同，所以有 $bx = \sqrt{hx*hx + hy*hy}$ 。而对于 bz 的处理，我们不做变动，令 $bz = hz$ 即可。经过这样处理之后的 $[bx, 0, bz]^T$ ，经过 bCn1 旋转回到 b 系中，得到 $[wx, wy, wz]^T$ ，这个值再和 b 系中的地磁计输出 $[mx, my, mz]^T$ 做向量积求误差，再次修正 bCn1 (或者 nCb1)，得到 bCn2 (或者 nCb2)。这样就完成了一次地磁计的补偿。

将加速度计没能做到的 z 轴上的旋转修正，通过地磁计在 XOY 平面上的地磁力相同原理，得到了修正。于是乎，pitch 和 roll 通过加速度计修正，然后在这个基础之上 (该地磁计补偿方法必须依靠加速度计修正提供一致的 XOY 平面，才会有 $bx = \sqrt{hx*hx + hy*hy}$ 等式成立)，yaw 通过地磁计来补偿，最终得到了没有偏差的实时姿态 (也就是由四元数组成的旋转矩阵)。

对 Gyro 修正补偿以后为：

$$\begin{aligned} \omega(t) &= \omega_{\text{gyro}}(t) + \omega_{\text{correction}}(t) \\ \omega_{\text{gyro}}(t) &= \text{three axis gyro measurements} \\ \omega_{\text{correction}}(t) &= \text{gyro correction} \end{aligned} \quad \text{Eqn. 16}$$

并把通过 PI 反馈控制器修正过的 gyro vector 作为公式 17 的输入更新 DCM：

$$\begin{aligned} \mathbf{R}(t+dt) &= \mathbf{R}(t) \begin{bmatrix} 1 & -d\theta_z & d\theta_y \\ d\theta_z & 1 & -d\theta_x \\ -d\theta_y & d\theta_x & 1 \end{bmatrix} \\ d\theta_x &= \omega_x dt \\ d\theta_y &= \omega_y dt \\ d\theta_z &= \omega_z dt \end{aligned} \quad \text{Eqn. 17}$$

此方式在数学上没有任何问题，但是由于地磁传感器极易受到各种干扰，而此算法又将地磁传感器所指示的方向过度的融入到了姿态当中，导致实际使用中数据会非常不稳定。怪不得 crazypony 的算法中没有使用磁力计修正 YAW，这应该就是原因所在吧。

姿态解算过程中不仅需要修正陀螺仪的各种 errors，还需要实时的更新的 DCM。在 `libraries/AP_AHRS/AP_AHRS_DCM.cpp` 中的 `matrix_update(delta_t)`，就是实时更新 DCM 矩阵的，源码如下：

```
1 // update the DCM matrix using only the gyros
```

```
2 void AP_AHRS_DCM::matrix_update(float _G_Dt)
3 {
4     // note that we do not include the P terms in _omega. This is
5     // because the spin_rate is calculated from _omega.length(),
6     // and including the P terms would give positive feedback into
7     // the _P_gain() calculation, which can lead to a very large P
8     // value
9     _omega.zero();
10
11     // average across first two healthy gyros. This reduces noise on
12     // systems with more than one gyro. We don't use the 3rd gyro
13     // unless another is unhealthy as 3rd gyro on PH2 has a lot more
14     // noise
15     uint8_t healthy_count = 0;
16     Vector3f delta_angle;
17     for (uint8_t i=0; i<_ins.get_gyro_count(); i++) {
18         if (_ins.get_gyro_health(i) && healthy_count < 2) {
19             Vector3f dangle;
20             if (_ins.get_delta_angle(i, dangle)) {
21                 healthy_count++;
22                 delta_angle += dangle;
23             }
24         }
25     }
```

```
26     if (healthy_count > 1) {  
27         delta_angle /= healthy_count;  
28     }  
29     if (_G_Dt > 0) {  
30         _omega = delta_angle / _G_Dt;  
31         _omega += _omega_l;  
32         _dcm_matrix.rotate((_omega + _omega_P + _omega_yaw_P) * _G_Dt);  
33     }  
34 }
```

首先就是通过陀螺仪获取两次 gyro_vector，然后取平均以降低误差；然后就是比较专业的算法实现_dcm_matrix.rotate((_omega + _omega_P + _omega_yaw_P) * _G_Dt)了。进入到 libraries/AP_Math/matrix3.cpp 中有 rotate() 函数，该函数就是实现 DCM 更新的算法实现，算法主要是用陀螺仪的输出值与 DCM 矩阵的乘积再加回到 DCM 中去，处理过程中使用了离散化的概念，即 $dcm(k+1) = dcm(k) + \text{增量}$ ，因为公式里有求导，必须离散化后才能计算机处理。

```
1 // apply an additional rotation from a body frame gyro vector  
2 // to a rotation matrix.  
3 template <typename T>  
4 void Matrix3<T>::rotate(const Vector3<T> &g)  
5 {  
6     Matrix3<T> temp_matrix;  
7     temp_matrix.a.x = a.y * g.z - a.z * g.y;  
8     temp_matrix.a.y = a.z * g.x - a.x * g.z;  
9     temp_matrix.a.z = a.x * g.y - a.y * g.x;
```

```
10     temp_matrix.b.x = b.y * g.z - b.z * g.y;  
11     temp_matrix.b.y = b.z * g.x - b.x * g.z;  
12     temp_matrix.b.z = b.x * g.y - b.y * g.x;  
13     temp_matrix.c.x = c.y * g.z - c.z * g.y;  
14     temp_matrix.c.y = c.z * g.x - c.x * g.z;  
15     temp_matrix.c.z = c.x * g.y - c.y * g.x;  
16  
17     (*this) += temp_matrix; //增加累加到原始数据上  
18 }
```


主要分析一下该算法的实现，该部分如果直接参考 DCM IMU:Theory 中的 Computing direction cosines from gyro signals 部分介绍的公式 17 (15 页) 有些过于抽象，所以下面介绍一下推导部分。在载体坐标系中定义矢量 r^b ，可以通过该矢量左乘方向余弦矩阵 C_b^n 转换到参考系中： $r^n = C_b^n \cdot r^b$ 。

$$\text{其中 } C_b^n = \begin{bmatrix} c_{11} & c_{12} & c_{13} \\ c_{21} & c_{22} & c_{23} \\ c_{31} & c_{32} & c_{33} \end{bmatrix}$$

方向余弦矩阵随时间的传递的推导过程。 C_b^n 随时间的变化率(标红表示更新后的)如下：

$$\dot{C}_b^n = \lim_{\Delta t \rightarrow 0} C_b^n / \Delta t = \lim_{\Delta t \rightarrow 0} [C_b^n(t + \Delta t) - C_b^n(t)] / \Delta t$$

$C_b^n(t + \Delta t)$ 可以写成如下两个矩阵相乘的形式： $C_b^n(t + \Delta t) = C_b^n(t)A(t)$

式中 $A(t)$ 是一个联系 b 系从 t 时刻到 $t + \Delta t$ 时刻的方向余弦矩阵。对于小角度转动， $A(t)$ 可以表示为： $A(t) = [I + \Delta A]$ ，其中 I 是 3×3 的单位矩阵。且

$$\Delta A = \begin{bmatrix} 0 & -\Delta\theta_z & \Delta\theta_y \\ \Delta\theta_z & 0 & \Delta\theta_x \\ -\Delta\theta_y & \Delta\theta_x & 0 \end{bmatrix}$$

式中 $\theta_x, \theta_y, \theta_z$ 分别表示 b 系统绕其横滚轴、俯仰轴和偏航轴在 Δt 时间间隔内转动的小角度。在 Δt 趋近于 0 时，小角度近似是有效的。替换 $\dot{C}_b^n$ 公式后得到：

$$\dot{C}_b^n = C_b^n \lim_{\Delta t \rightarrow 0} \Delta A / \Delta t$$

在 Δt 趋近于 0 时， $\Delta A / \Delta t$ 是角速度矢量 $\mathbf{W}_{nb}^b = [w_x, w_y, w_z]^T$ 的斜对称形式，表示 b 系相对于 n 系在载体轴系的转动角速率。即：

$$\dot{C}_b^n = C_b^n \Omega \quad \text{其中 } \Omega = \begin{bmatrix} 0 & -w_z & w_y \\ w_z & 0 & w_x \\ -w_y & w_x & 0 \end{bmatrix}$$

上式可以在捷联惯性导航系统的计算机中解算，以便跟踪载体相对于参考系的姿态，它的分量形式如下。即使用 gyro_vector 更新以后的 DCM：

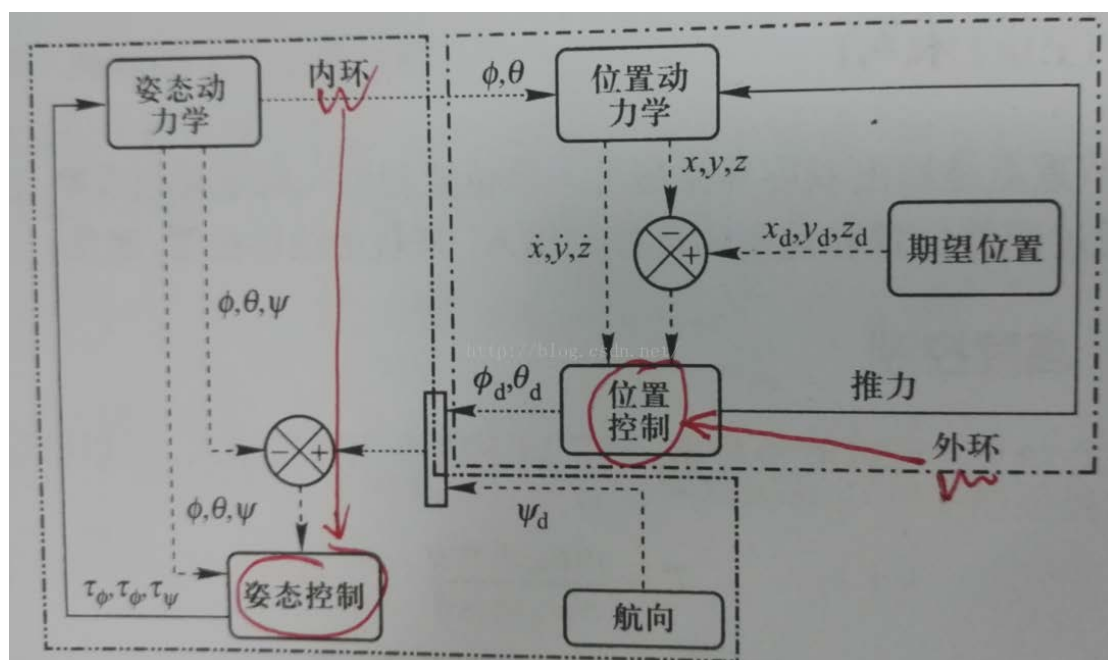
$$\begin{aligned} c_{11} &= c_{12}w_z - c_{13}w_y, & c_{12} &= c_{13}w_x - c_{11}w_x, & c_{13} &= c_{11}w_y - c_{12}w_x \\ c_{21} &= c_{22}w_z - c_{23}w_y, & c_{22} &= c_{23}w_x - c_{21}w_x, & c_{23} &= c_{21}w_y - c_{22}w_x \\ c_{31} &= c_{32}w_z - c_{33}w_y, & c_{32} &= c_{33}w_x - c_{31}w_x, & c_{33} &= c_{31}w_y - c_{32}w_x \end{aligned}$$

5.2、姿态控制

5.2.1、基础知识

无人机控制部分主要分为两个部分，姿态控制部分和位置控制部分；位置控制可用远程遥控控制，而姿态控制一般由无人机系统自动完成。姿态控制是非常重要的，因为无人机的位置变化都是由姿态变化引起的。

下图阐释了 PX4 源码中的两个环路控制，分为姿态控制和位置控制。



PX4Fireware 原生固件中的 modules 中姿态估计有多种：

Attitude_estimator_ekf、Attitude_estimator_q、ekf_att_pos_estimator。

位置估计有：ekf_att_pos_estimator、local_position_estimator、position_estimator_inav

姿态控制有：fw_att_control、mc_att_control、mc_att_control_multiplatform、vtol_att_control

位置控制有：fw_pos_control_l1、fw_pos_control_l1、mc_pos_control_multiplatform

四旋翼用到以上哪些估计和控制算法呢？这部分在启动代码 rc.mc_app 里面有详细的说明。

```

1 # The system is defaulting to INAV_ENABLED = 1
2 # but users can alternatively try the EKF-based
3 # filter by setting INAV_ENABLED = 0
4 if param compare INAV_ENABLED 1

```

```
5  then
6      attitude_estimator_q start
7      position_estimator_inav start
8  else
9      if param compare LPE_ENABLED 1
10     then
11         attitude_estimator_q start
12         local_position_estimator start
13     else
14         ekf2 start
15     fi
16 fi
17
18 if mc_att_control start
19 then
20 else
21     # try the multiplatform version
22     mc_att_control_m start
23 fi
24
25 if mc_pos_control start
26 then
27 else
28     # try the multiplatform version
```

29 `mc_pos_control_m start`

30 `fi`

默认的是：

姿态估计 `Attitude_estimator_q`

位置估计 `position_estimator_inav`

姿态控制 `mc_att_control`

位置控制 `mc_pos_control`

5.2.2、飞行控制（理论概述）

飞行控制分为姿态控制和位置控制，该文章主讲姿态控制。

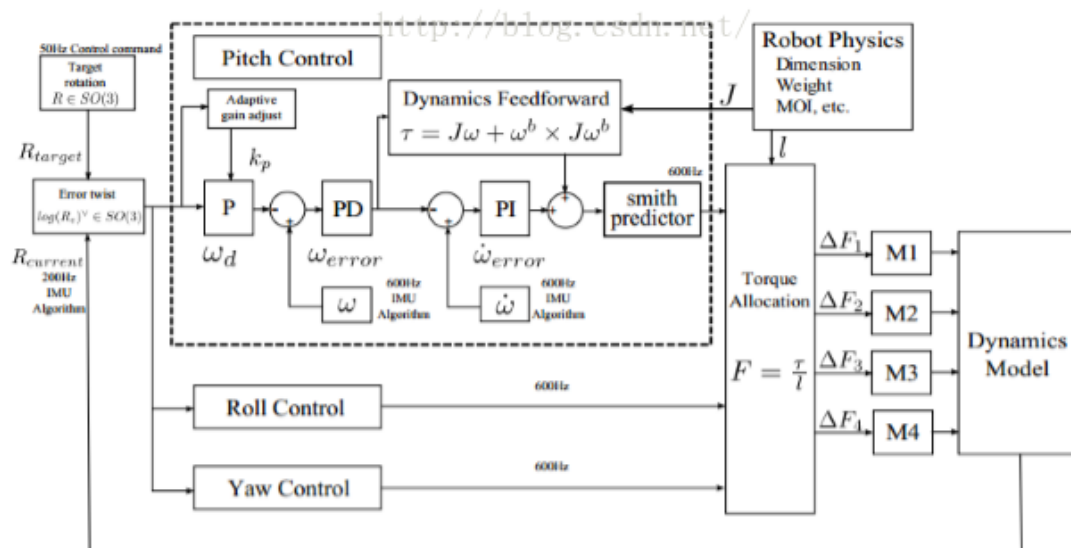
所谓姿态控制，主要就是在前期姿态解算的基础上对四旋翼飞行器进行有效的飞行控制，以达到所需要的控制效果。在这种情况下，算法要学会如何连续地做决策，并且算法的评价应该根据其所做选择的长期质量来进行。举一个具体的例子，想想无人机飞行所面临的难题：每不到一秒，算法都必须反复地选择最佳的行动控制。控制过程还是以经典的 PID 反馈控制器为主（在控制环路中可以添加 smith 预测器）。那么如何实现控制呢？

以四旋翼飞行器为例，主要就是通过改变旋翼的角速度来控制四旋翼无人机。每个旋翼产生一个推力（ F_1 、 F_2 、 F_3 、 F_4 ）和一个力矩，其共同作用构成四旋翼无人机的主推力、偏航力矩、俯仰力矩和滚转力矩。在四旋翼无人机中，正对的一对旋翼旋转方向一致，另外一对与之相反，来抵消静态平稳飞行时的回转效应和气动力矩。升降以及 RPY 的实现不在赘述。控制对象就是四旋翼无人机，其动力学模型可以描述为：将其视为有一个力和三个力矩的三维刚体。如下给出了小角度变化条件下的四旋翼无人机的近似动力学模型：

$$\begin{cases} m\ddot{x} = u(\sin \phi \sin \psi + \cos \phi \cos \psi \sin \theta) \\ m\ddot{y} = u(\cos \phi \sin \theta \sin \psi - \cos \psi \sin \phi) \\ m\ddot{z} = u \cos \theta \cos \phi - mg \\ \ddot{\psi} = \tau_{\psi} \\ \ddot{\theta} = \tau_{\theta} \\ \ddot{\phi} = \tau_{\phi} \end{cases}$$

式中：x、y 分别为水平面的坐标；z 为垂直位置； ψ 为绕 z 轴的偏航角， θ 为绕 y 轴的俯仰角， ϕ 为绕 x 轴的滚转角；u 为源自无人机底部向上的推力； τ_{ψ} 、 τ_{θ} 、 τ_{ϕ} 分别为偏航力矩、俯仰力矩、滚转力矩。

动态动力学模型： $J\dot{\omega}^b + \omega^b \times J\omega^b = \tau$



PX4 的姿态控制部分使用的是 roll-pitch 和 yaw 分开控制的（是为了解耦控制行为），即 tilt 和 torsion 两个环节。感性认识一下，如下图：

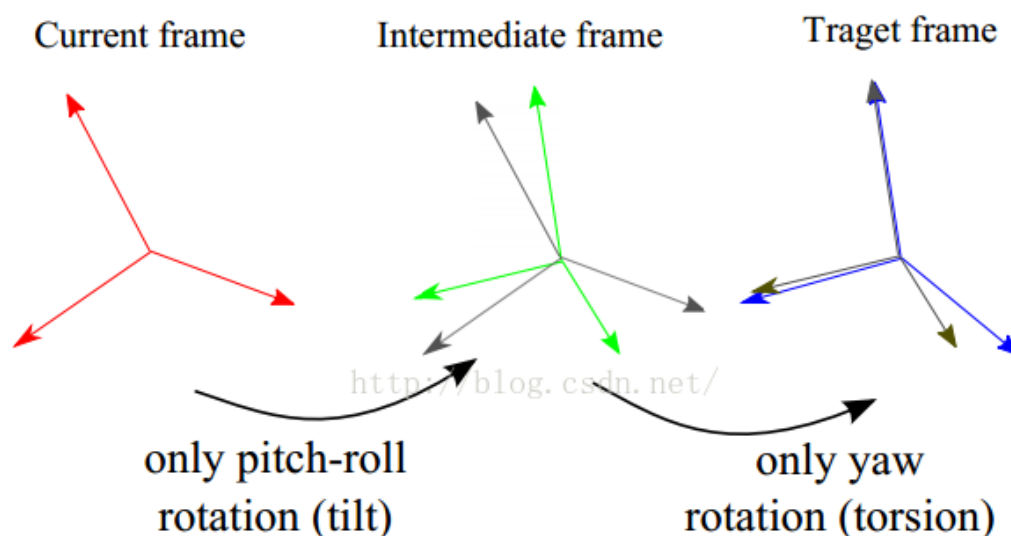


Fig. 4: Tilt-torsion decomposition. For any current (Red) and target (Blue) rotations, add an intermediate (Green) rotation. Then execute a two-stage control action.

根据经验所得,控制 roll-pitch 比控制 yaw 更容易实现。比如同样是实现 10° 的变化, roll-pitch 需要 60ms 左右; 但是 yaw 控制器却需要接近 150ms。

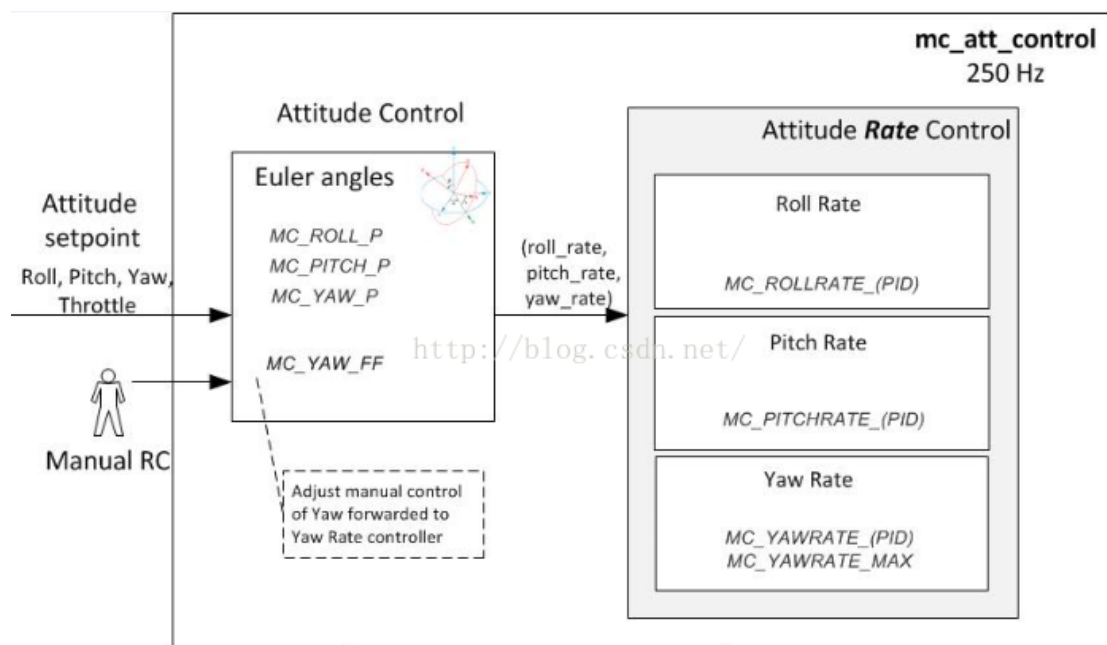
控制流程:

- 1)、预处理: 各参数的初始化。
- 2)、稳定 roll-pitch 的角速度。
- 3)、稳定 roll-pitch 的角度。
- 4)、稳定 yaw 的角速度。
- 5)、稳定 yaw 的角度。

5.2.3、进入姿态控制源码的前期过程

首先感性认识一下姿态控制部分的框架,控制部分分为内外环控制,内环控制角速度、外环控制角度。控制过程是先根据目标姿态 (target) 和当前姿态

(current) 求出偏差角，然后通过角速度来修正这个偏差角，最终到达目标姿态。



和姿态解算算法的流程几乎类似，主要的代码流程首先就是按照 C++ 语言的格式引用 C 语言的 main 函数，但是在该处变成了：

```
extern "C" __EXPORT int mc_att_control_main(int argc, char *argv[]).
```

然后：跳转到所谓的 main 函数，该部分有个要注意的点，如下代码所示：
即 `mc_att_control::g_control = new MulticopterAttitudeControl;` //重点 (934)，new 关键词类似于 C 语言中的 malloc，对变量进行内存分配的，即对姿态控制过程中使用到的变量赋初值。

/PX4Firmware/src/modules/mc_att_control/mc_att_control_main.cpp

```
1 int mc_att_control_main(int argc, char *argv[])
2 {
3     if (argc < 2) {
4         warnx("usage: mc_att_control {start|stop|status}");
```

```
5         return 1;
6     }
7
8     if (!strcmp(argv[1], "start")) {
9
10        if (mc_att_control::g_control != nullptr) {
11
12            warnx("already running");
13
14            return 1;
15        }
16
17        mc_att_control::g_control = new MulticopterAttitudeControl;
18
19        if (mc_att_control::g_control == nullptr) {
20
21            warnx("alloc failed");
22
23            return 1;
24        }
25
26        if (OK != mc_att_control::g_control->start()) {
27
28            delete mc_att_control::g_control;
```



```
29     return 0;  
30 }
```

然后：start 函数

/PX4Firmware/src/modules/mc_att_control/mc_att_control_main.cpp

```
1  int MulticopterAttitudeControl::start()  
2  {  
3      ASSERT(_control_task == -1);  
4  
5      /* start the task */  
6      _control_task = px4_task_spawn_cmd("mc_att_control",  
7                                         SCHED_DEFAULT,  
8                                         SCHED_PRIORITY_MAX - 5,  
9                                         1500,  
10  
11 (px4_main_t)&MulticopterAttitudeControl::task_main_trampoline,  
12  
13                                         nullptr);  
14  
15     if (_control_task < 0) {  
16         warn("task start failed");  
17         return -errno;  
18     }  
19     return OK;
```

1]

其中上面有个封装了 nuttx 自带的生成 task 的任务创建函数(他把优先级什么的做了重新的 define, 这么做是便于代码阅读): px4_task_spawn_cmd(), 注意它的用法。其函数原型是:

```
px4_task_t px4_task_spawn_cmd(const char *name, int scheduler, int priority,  
                               int stack_size, px4_main_t entry, char *const argv[])
```

第一个参数是 namespace, 第二个参数是选择调度策略, 第三个是任务优先级, 第四个是任务的栈空间大小, 第五个是任务的入口函数, 最后一个一般是 null。

然后:

```
1 /PX4Firmware/src/modules/mc_att_control/mc_att_control_main.cpp  
2  
3 void MulticopterAttitudeControl::task_main_trampoline(int argc, char *argv[])  
4 {  
5     mc_att_control::g_control->task_main();  
6 }
```

再然后终到本体。

```
/PX4Firmware/src/modules/mc_att_control/mc_att_control_main.cpp
```

```
void MulticopterAttitudeControl::task_main() {}
```

5.2.4、姿态控制源码——订阅

首先还是需要通过 IPC 模型 uORB 进行订阅所需要的数据。需要注意的一个细节就是在该算法处理过程中的有效数据的用途问题, 最后处理过的数据最后又被改进程自己订阅了, 然后再处理, 再订阅, 一直处于循环状态, 这就是所

谓的 PID 反馈控制器吧，最终达到所需求的控制效果，达到控制效果以后就把一系列的控制量置 0（类似于 idle），该任务一直在运行，随启动脚本启动的。

```
/PX4Firmware/src/modules/mc_att_control/mc_att_control_main.cpp
```

```
void MulticopterAttitudeControl::task_main()
```

```
1  /*
2
3  * do subscriptions
4
5  */
6  _v_att_sp_sub = orb_subscribe(ORB_ID(vehicle_attitude_setpoint));
7
8  _v_rates_sp_sub = orb_subscribe(ORB_ID(vehicle_rates_setpoint));
9
10 _ctrl_state_sub = orb_subscribe(ORB_ID(control_state));
11
12 _v_control_mode_sub = orb_subscribe(ORB_ID(vehicle_control_mode));
13
14 _params_sub = orb_subscribe(ORB_ID(parameter_update));
15
16 _manual_control_sp_sub = orb_subscribe(ORB_ID(manual_control_setpoint));
17
18 _armed_sub = orb_subscribe(ORB_ID(actuator_armed));
19
20 _vehicle_status_sub = orb_subscribe(ORB_ID(vehicle_status));
21
22 _motor_limits_sub = orb_subscribe(ORB_ID(multirotor_motor_limits));
```

上面这些订阅到底订阅了哪些东西呢，顾名思义，根据 ORB() 中的参数的名称就是知道订阅的到底用于做什么的了。这套开源代码中最优越的地方时变量的命名很好，通俗易懂。

5.2.5、参数初始化

紧随上面的代码就是参数数据的获取，parameters 主要就是前期定义的兴趣的数据，在姿态控制中的这些数据都是私有数据（private），比如 roll、

pitch、yaw 以及与它们对应的 PID 参数。注意区分 _params_handles 和 _params 这两种数据结构（struct 类型）。

```
/PX4Firmware/src/modules/mc_att_control/mc_att_control_main.cpp
```

```
1  /* initialize parameters cache */
2      parameters_update();
3  函数原型:
4  int MulticopterAttitudeControl::parameters_update()
5  {
6      float v;
7
8      float roll_tc, pitch_tc;
9
10     param_get(_params_handles.roll_tc, &roll_tc);
11     param_get(_params_handles.pitch_tc, &pitch_tc);
12
13     /* roll gains */
14     param_get(_params_handles.roll_p, &v);
15     _params.att_p(0) = v * (ATTITUDE_TC_DEFAULT / roll_tc);
16     param_get(_params_handles.roll_rate_p, &v);
17     _params.rate_p(0) = v * (ATTITUDE_TC_DEFAULT / roll_tc);
18     param_get(_params_handles.roll_rate_i, &v);
19     _params.rate_i(0) = v;
```

```
20 param_get(_params_handles.roll_rate_d, &v);
21 _params.rate_d(0) = v * (ATTITUDE_TC_DEFAULT / roll_tc);
22 param_get(_params_handles.roll_rate_ff, &v);
23 _params.rate_ff(0) = v;
24
25 /* pitch gains */
26
27 .....
28
29 /* yaw gains */
30 .....
31
32 /* angular rate limits */
33 param_get(_params_handles.roll_rate_max, &_params.roll_rate_max);
34 _params.mc_rate_max(0) = math::radians(_params.roll_rate_max);
35 param_get(_params_handles.pitch_rate_max, &_params.pitch_rate_max);
36 _params.mc_rate_max(1) = math::radians(_params.pitch_rate_max);
37 param_get(_params_handles.yaw_rate_max, &_params.yaw_rate_max);
38 _params.mc_rate_max(2) = math::radians(_params.yaw_rate_max);
39
40 /* auto angular rate limits */
41 param_get(_params_handles.roll_rate_max, &_params.roll_rate_max);
42 _params.auto_rate_max(0) = math::radians(_params.roll_rate_max);
43 param_get(_params_handles.pitch_rate_max, &_params.pitch_rate_max);
```

```
44     _params.auto_rate_max(1) = math::radians(_params.pitch_rate_max);
45     param_get(_params_handles.yaw_auto_max, &_amp;_params.yaw_auto_max);
46     _params.auto_rate_max(2) = math::radians(_params.yaw_auto_max);
47
48     /* manual rate control scale and auto mode roll/pitch rate limits */
49     param_get(_params_handles.acro_roll_max, &_amp;v);
50     _params.acro_rate_max(0) = math::radians(v);
51     param_get(_params_handles.acro_pitch_max, &_amp;v);
52     _params.acro_rate_max(1) = math::radians(v);
53     param_get(_params_handles.acro_yaw_max, &_amp;v);
54     _params.acro_rate_max(2) = math::radians(v);
55
56     /* stick deflection needed in rattitude mode to control rates not angles */
57     param_get(_params_handles.rattitude_thres, &_amp;_params.rattitude_thres);
58
59     param_get(_params_handles.vtol_type, &_amp;_params.vtol_type);
60
61     int tmp;
62     param_get(_params_handles.vtol_opt_recovery_enabled, &_amp;tmp);
63     _params.vtol_opt_recovery_enabled = (bool)tmp;
64
65     param_get(_params_handles.vtol_wv_yaw_rate_scale,
66     &_amp;_params.vtol_wv_yaw_rate_scale);
67
```

```
68     _actuators_0_circuit_breaker_enabled = circuit_breaker_enabled("CBRK_RATE_CTRL",
69     CBRK_RATE_CTRL_KEY);
70
71     return OK;
72 }
```

重点分析一下上述代码：其中 param_get() 函数比较重要，特别是内部使用的 lock 和 unlock 的使用（主要就是通过 sem 信号量控制对某一数据的互斥访问）。

/PX4Firmware/src/modules/systemlib/param

```
1  int param_get(param_t param, void *val)
2  {
3      int result = -1;
4      param_lock();
5      const void *v = param_get_value_ptr(param);
6      if (val != NULL) {
7          memcpy(val, v, param_size(param));
8          result = 0;
9      }
10     param_unlock();
11     return result;
12 }
```

上述使用的*lock 和*unlock 通过 sem 实现互斥访问（进临界区），源码如下。

```
/PX4Firmware/src/modules/systemlib/param
```

```
1  /** lock the parameter store */
2  static void param_lock(void)
3  {
4      //do {} while (px4_sem_wait(&param_sem) != 0);
5  }
6  /** unlock the parameter store */
7  static void param_unlock(void)
8  {
9      //px4_sem_post(&param_sem);
10 }
```

上面是开源代码中的，代码里面把 lock 和 unlock 函数都写成空函数了。应该是由于程序开发和版本控制不是一个人，有的程序开发到一半人走了，搞版本控制的，又找不到新的人来进行开发，搁置了忘记修改回来了吧；再或者别的什么意图。

经过上述分析，该 parameters_update() 函数主要就是获取 roll、pitch、yaw 的 PID 参数的。并对三种飞行模式（stabilize、auto、acro）下的最大姿态速度做了限制。

5.2.6、NuttX 任务使能

参数初始化结束即 parameters_update() 执行完继续执行：

```
/PX4Firmware/src/modules/mc_att_control/mc_att_control_main.cpp
```

```
1  /* wakeup source: vehicle attitude */
```



```
2     px4_pollfd_struct_t fds[1];  
3  
4     fds[0].fd = _ctrl_state_sub;  
5     fds[0].events = POLLIN;
```

注意上面的 fd 的赋值。随后进入任务的循环函数：while (!_task_should_exit) {}。都是一样的模式，在姿态解算时也是使用的该种方式。

5.2.7、阻塞等待数据

```
1 /PX4Firmware/src/modules/mc_att_control/mc_att_control_main.cpp  
2  
3 /* wait for up to 100ms for data */  
4     int pret = poll(&fds[0], (sizeof(fds) / sizeof(fds[0])), 100);  
5  
6     /* timed out - periodic check for _task_should_exit */  
7     if (pret == 0)  
8         continue;  
9  
10    /* this is undesirable but not much we can do - might want to flag unhappy status  
11    */  
12    if (pret < 0) {  
13        warn("poll error %d, %d", pret, errno);  
14        /* sleep a bit before next try */  
15        usleep(100000);  
16        continue;
```

```
17     }  
18  
19     perf_begin(_loop_perf);
```

1 首先是 `px4_poll()` 配置阻塞时间 100ms (uORB 模型的函数 API)。然后是打
2 开 MAVLINK 协议, 记录数据。如果 poll 失败, 直接使用关键词 `continue` 从头开
3 始运行 (注意 `while` 和 `continue` 的组合使用)。其中的 `usleep(10000)` 函数属
4 于线程级睡眠函数, 使当前线程挂起。

5 上面最后一个 `perf_begin(_loop_perf)`, 是一个空函数, 带 `perf` 开头的都
6 是空函数, 它的作用主要是 “Empty function calls for ros compatibility”。

7 5.2.8、获取当前姿态

```
8 /PX4Firmware/src/modules/mc_att_control/mc_att_control_main.cpp  
9  
10 /* run controller on attitude changes */  
11  
12     if (fds[0].revents & POLLIN) {  
13  
14         static uint64_t last_run = 0;  
15  
16         float dt = (hrt_absolute_time() - last_run) / 1000000.0f;  
17  
18         last_run = hrt_absolute_time();  
19  
20         /* guard against too small (< 2ms) and too large (> 20ms) dt's */  
21  
22         if (dt < 0.002f) {  
23  
24             dt = 0.002f;  
25  
26         } else if (dt > 0.02f) {  
27  
28             dt = 0.02f;  
29  
30         }  
31     }
```

```
22     }
23
24     /* copy attitude and control state topics */ //获取当前姿态数据
25     orb_copy(ORB_ID(control_state), _ctrl_state_sub, &_ctrl_state);
26
27     /* check for updates in other topics */
28     parameter_update_poll();
29     vehicle_control_mode_poll();
30     arming_status_poll();
31     vehicle_manual_poll();
32     vehicle_status_poll();
33     vehicle_motor_limits_poll();
```

注意上面的 revents，要与 events 区分开来，两者的区别如下：

```
pollevent_t events; /* The input event flags */
```

```
pollevent_t revents; /* The output event flags */
```

首先就是判断姿态控制器的控制任务是否已经使能，然后就是检测通过 hrt 获取时间精度的所需时间，并且约束在 2ms 至 20ms 以内。

PX4Firmware/src/modules/uORB/uORB.cpp

```
1  /**
2   * Fetch data from a topic.
3   *
4   * This is the only operation that will reset the internal marker that
```

```

5  * indicates that a topic has been updated for a subscriber. Once poll
6  * or check return indicating that an update is available, this call
7  * must be used to update the subscription.
8  *
9  * @param meta    The uORB metadata (usually from the ORB_ID() macro)
10 *               for the topic.
11 * @param handle  A handle returned from orb_subscribe.
12 * @param buffer  Pointer to the buffer receiving the data, or NULL
13 *               if the caller wants to clear the updated flag without
14 *               using the data.
15 * @return        OK on success, ERROR otherwise with errno set accordingly.
16 */
17 int orb_copy(const struct orb_metadata *meta, int handle, void *buffer)
18 {
19     return uORB::Manager::get_instance()->orb_copy(meta, handle, buffer);
20 }

```

第三个参数就是为了保存通过 orb_subscribe() 函数订阅获得的有效数据，该部分获取的是 ctrl_state，即控制姿态的数据，数据结构如下：（包含三轴加速度、三轴速度、三轴位置、空速、四元数、roll/pitch/yaw 的速率）。这个 copy 的内容，后面会用到多次。

然后就是检测数据是否已经更新。

```

/PX4Firmware/src/modules/mc_att_control/mc_att_control_main.cpp

```

```

1  /* check for updates in other topics */

```

```
2 parameter_update_poll();
```

parameter_update_poll() 的函数原型:

```
/PX4Firmware/src/modules/mc_att_control/mc_att_control_main.cpp
```

```
1 void MulticopterAttitudeControl::parameter_update_poll()
2 {
3     bool updated;
4
5     /* Check if parameters have changed */
6     orb_check(_params_sub, &updated);
7
8     if (updated) {
9         struct parameter_update_s param_update;
10        orb_copy(ORB_ID(parameter_update), _params_sub, &param_update);
11        parameters_update();
12    }
13 }
```

然后飞行模式判断是否是 MAIN_STATE_RATTITUD 模式，该模式是一种新的飞行模式，只控制角速度，不控制角度，俗称半自稳模式（小舵量自稳大舵量手动），主要用在 setpoint 中，航点飞行。根据介绍，这个模式只有在 pitch 和 roll 都设置为 Rattitude 模式时才有意义，如果 yaw 也设置了该模式，那么就会自动被手动模式替代了。所以代码中只做了 x、y 阈值的检测。

```
/PX4Firmware/src/modules/mc_att_control/mc_att_control_main.cpp
```

```
1 /PX4Firmware/src/modules/mc_att_control/mc_att_control_main.cpp
2
3 /* Check if we are in rattitude mode(新的飞行模式，角速度模式，没有角度控制)
4  *and the pilot is above the threshold on pitch or roll (yaw can rotate 360
5  *in normal att control). If both are true don't even bother running the
6  *attitude controllers */
7 if (_vehicle_status.main_state == vehicle_status_s::MAIN_STATE_RATTITUDE) {
8     if (fabsf(_manual_control_sp.y) > _params.rattitude_thres ||
9         fabsf(_manual_control_sp.x) > _params.rattitude_thres) {
10         _v_control_mode.flag_control_attitude_enabled = false;
11     }
12 }
```

5.2.9、进行姿态控制

根据前面的代码分析，在确定了飞行模式以后（判断当前飞行模式，通过最开始部分的*poll 函数获取），再进行姿态控制。

```
/PX4Firmware/src/modules/mc_att_control/mc_att_control_main.cpp
```

```
1 if (_v_control_mode.flag_control_attitude_enabled) {
2
3     if (_ts_opt_recovery == nullptr) {
4         // the tailsitter recovery instance has not been created, thus, the
5         //vehicle
6         // is not a tailsitter, do normal attitude control
```

```
7     control_attitude(dt);
8
9     } else {
10         vehicle_attitude_setpoint_poll();
11         _thrust_sp = _v_att_sp.thrust;
12         math::Quaternion q(_ctrl_state.q[0], _ctrl_state.q[1],
13                             _ctrl_state.q[2], _ctrl_state.q[3]);
14         math::Quaternion q_sp(&_v_att_sp.q_d[0]);
15         _ts_opt_recovery->setAttGains(_params.att_p, _params.yaw_ff);
16         _ts_opt_recovery->calcOptimalRates(q, q_sp,
17                                           _v_att_sp.yaw_sp_move_rate,
18                                           _rates_sp);
19
20         /* limit rates */
21         for (int i = 0; i < 3; i++) {
22             _rates_sp(i) = math::constrain(_rates_sp(i),
23                                           -_params.mc_rate_max(i), _params.mc_rate_max(i));
24         }
25     }
26
27     /* publish attitude rates setpoint */
28     _v_rates_sp.roll = _rates_sp(0);
29     _v_rates_sp.pitch = _rates_sp(1);
30     _v_rates_sp.yaw = _rates_sp(2);
```

```

31     _v_rates_sp.thrust = _thrust_sp;

32     _v_rates_sp.timestamp = hrt_absolute_time();

33

34     if (_v_rates_sp_pub != nullptr) {

35         orb_publish(_rates_sp_id, _v_rates_sp_pub, &_v_rates_sp);

36

37     } else if (_rates_sp_id) {

38         _v_rates_sp_pub = orb_advertise(_rates_sp_id, &_v_rates_sp);

39     }

40

41     //}

42

43 } else {

44     /* attitude controller disabled, poll rates setpoint topic */

45     if (_v_control_mode.flag_control_manual_enabled) {

46         /* manual rates control - ACRO mode */

47         _rates_sp = math::Vector<3>(_manual_control_sp.y,

48                                     _manual_control_sp.x,

49                                     _manual_control_sp.r).emult(_params.acro_rate_max);

50         _thrust_sp = math::min(_manual_control_sp.z,

51                                MANUAL_THROTTLE_MAX_MULTICOPTER);

52

53         /* publish attitude rates setpoint */

54         _v_rates_sp.roll = _rates_sp(0);
  
```



```
55     _v_rates_sp.pitch = _rates_sp(1);
56     _v_rates_sp.yaw = _rates_sp(2);
57     _v_rates_sp.thrust = _thrust_sp;
58     _v_rates_sp.timestamp = hrt_absolute_time();
59
60     if (_v_rates_sp_pub != nullptr) {
61         orb_publish(_rates_sp_id, _v_rates_sp_pub, &_v_rates_sp);
62
63     } else if (_rates_sp_id) {
64         _v_rates_sp_pub = orb_advertise(_rates_sp_id, &_v_rates_sp);
65     }
66
67     } else {
68         /* attitude controller disabled, poll rates setpoint topic */
69         vehicle_rates_setpoint_poll();
70         _rates_sp(0) = _v_rates_sp.roll;
71         _rates_sp(1) = _v_rates_sp.pitch;
72         _rates_sp(2) = _v_rates_sp.yaw;
73         _thrust_sp = _v_rates_sp.thrust;
74     }
75 }
```

上面的代码中，初始就是 `control_attitude(dt)`，控制数据都是由它来获取的。该函数内部做了很多的处理，控制理论基本都是在这个里面体现的，所以需要深入研究理解它才可以进一步的研究后续的算法。它的内部会通过算法

处理获得控制量（目标姿态 Target），即_rates_sp，一个 vector<3>变量，以便后续控制使用。

首先是姿态控制(control_attitude)，然后是速度控制(control_attitude_rates)。

5.3、角度控制环（control_attitude()函数）

获取目标姿态 Target

理论基础：先实现roll-pitch，然后yaw；即先tilt后torsion。如下是实现过程（ R_e 误差矩阵）：

$$R_e = R_{torsion} R_{tilt}$$

假设目标姿态矩阵的z轴 $Z_t = [0,0,0]^T$ ，当前姿态矩阵的z轴 $Z_c = [x,y,z]^T$ ，则 $Z_t = R_e Z_c$ 。反向推导得出 $Z_c = R_e^T Z_t$ 。旋转轴为 $r = Z_c \% R_e^T Z_t$ ，旋转角度为 θ ，则 $R_{tilt} = e^{r\theta}$ 。该旋转的目的主要就是为了使当前姿态的z轴和目标姿态的z轴对齐。然后再进行 $R_{torsion}$ 旋转对齐xy轴。分开控制的目的是为了解耦控制行为，即分别执行较快响应的动作和较慢效应的动作。

PX4Firmware/src/modules/mc_att_control/mc_att_control_main.cpp

```

1  /**
2   * Attitude controller.
3   * Input: 'vehicle_attitude_setpoint' topics (depending on mode)
4   * Output: '_rates_sp' vector, '_thrust_sp'
5   */
6  void MulticopterAttitudeControl::control_attitude(float dt)
7  {
8      vehicle_attitude_setpoint_poll();
9
10     _thrust_sp = _v_att_sp.thrust;
    
```

```
11
12     /* construct attitude setpoint rotation matrix */
13     math::Matrix<3, 3> R_sp;
14     R_sp.set(_v_att_sp.R_body);
15
16     /* get current rotation matrix from control state quaternions */
17     math::Quaternion q_att(_ctrl_state.q[0], _ctrl_state.q[1], _ctrl_state.q[2],
18 _ctrl_state.q[3]);
19     math::Matrix<3, 3> R = q_att.to_dcm();
20
21     /* all input data is ready, run controller itself */
22
23     /* try to move thrust vector shortest way, because yaw response is slower than
24 roll/pitch */
25     math::Vector<3> R_z(R(0, 2), R(1, 2), R(2, 2));
26     math::Vector<3> R_sp_z(R_sp(0, 2), R_sp(1, 2), R_sp(2, 2));
27
28     /* axis and sin(angle) of desired rotation */
29     math::Vector<3> e_R = R.transposed() * (R_z % R_sp_z);
30
31     /* calculate angle error */
32     float e_R_z_sin = e_R.length();
33     float e_R_z_cos = R_z * R_sp_z;
34
35     /* calculate weight for yaw control */
```

```

36     float yaw_w = R_sp(2, 2) * R_sp(2, 2);
37
38     /* calculate rotation matrix after roll/pitch only rotation */
39     math::Matrix<3, 3> R_rp;
40
41     if (e_R_z_sin > 0.0f) {
42         /* get axis-angle representation */
43         float e_R_z_angle = atan2f(e_R_z_sin, e_R_z_cos);
44         math::Vector<3> e_R_z_axis = e_R / e_R_z_sin;
45
46         e_R = e_R_z_axis * e_R_z_angle;
47
48         /* cross product matrix for e_R_axis */
49         math::Matrix<3, 3> e_R_cp;
50         e_R_cp.zero();
51         e_R_cp(0, 1) = -e_R_z_axis(2);
52         e_R_cp(0, 2) = e_R_z_axis(1);
53         e_R_cp(1, 0) = e_R_z_axis(2);
54         e_R_cp(1, 2) = -e_R_z_axis(0);
55         e_R_cp(2, 0) = -e_R_z_axis(1);
56         e_R_cp(2, 1) = e_R_z_axis(0);
57
58         /* rotation matrix for roll/pitch only rotation */
59         R_rp = R * (_I + e_R_cp * e_R_z_sin + e_R_cp * e_R_cp * (1.0f - e_R_z_cos));
  
```

```
60
61     } else {
62         /* zero roll/pitch rotation */
63         R_rp = R;
64     }
65
66     /* R_rp and R_sp has the same Z axis, calculate yaw error */
67     math::Vector<3> R_sp_x(R_sp(0, 0), R_sp(1, 0), R_sp(2, 0));
68     math::Vector<3> R_rp_x(R_rp(0, 0), R_rp(1, 0), R_rp(2, 0));
69     e_R(2) = atan2f((R_rp_x % R_sp_x) * R_sp_z, R_rp_x * R_sp_x) * yaw_w;
70
71     if (e_R_z_cos < 0.0f) {
72         /* for large thrust vector rotations use another rotation method:
73          * calculate angle and axis for R -> R_sp rotation directly */
74         math::Quaternion q;
75         q.from_dcm(R.transposed() * R_sp);
76         math::Vector<3> e_R_d = q.imag();
77         e_R_d.normalize();
78         e_R_d *= 2.0f * atan2f(e_R_d.length(), q(0));
79
80         /* use fusion of Z axis based rotation and direct rotation */
81         float direct_w = e_R_z_cos * e_R_z_cos * yaw_w;
82         e_R = e_R * (1.0f - direct_w) + e_R_d * direct_w;
83     }
```

```

84
85     /* calculate angular rates setpoint */
86     _rates_sp = _params.att_p.emult(e_R);
87
88     /* limit rates */
89     for (int i = 0; i < 3; i++) {
90         if (_v_control_mode.flag_control_velocity_enabled
91         && !_v_control_mode.flag_control_manual_enabled) {
92             _rates_sp(i) = math::constrain(_rates_sp(i), -_params.auto_rate_max(i),
93             _params.auto_rate_max(i));
94         } else {
95             _rates_sp(i) = math::constrain(_rates_sp(i), -_params.mc_rate_max(i),
96             _params.mc_rate_max(i));
97         }
98     }
99
100    /* weather-vane mode, dampen yaw rate */
101    if (_v_att_sp.disable_mc_yaw_control == true &&
102    _v_control_mode.flag_control_velocity_enabled
103    && !_v_control_mode.flag_control_manual_enabled) {
104        float wv_yaw_rate_max = _params.auto_rate_max(2) *
105        _params.vtol_wv_yaw_rate_scale;
106        _rates_sp(2) = math::constrain(_rates_sp(2), -wv_yaw_rate_max,
107        wv_yaw_rate_max);
108        // prevent integrator winding up in weathervane mode
109        _rates_int(2) = 0.0f;

```

```

110     }
111
112     /* feed forward yaw setpoint rate */
113     _rates_sp(2) += _v_att_sp.yaw_sp_move_rate * yaw_w * _params.yaw_ff;
114
115     /* weather-vane mode, scale down yaw rate */
116     if (_v_att_sp.disable_mc_yaw_control == true &&
117         _v_control_mode.flag_control_velocity_enabled
118         && !_v_control_mode.flag_control_manual_enabled) {
119         float wv_yaw_rate_max = _params.auto_rate_max(2) *
120             _params.vtol_wv_yaw_rate_scale;
121         _rates_sp(2) = math::constrain(_rates_sp(2), -wv_yaw_rate_max,
122             wv_yaw_rate_max);
123         // prevent integrator winding up in weathervane mode
124         _rates_int(2) = 0.0f;
125     }
126 }

```

首先就是通过 uORB 模型检测姿态数据是否已经更新。检测到更新数据以后，把数据拷贝到当前，并通过 `_thrust_sp = _v_att_sp.thrust` 把油门控制量赋值给控制变量。

然后构建姿态旋转矩阵（目标状态，所谓的 TargetRotation）。

```

1     /* construct attitude setpoint rotation matrix */
2
3     math::Matrix<3, 3> R_sp;
4
5     R_sp.set(_v_att_sp.R_body);

```

通过控制四元数获取当前状态的旋转矩阵 DCM，后面在计算误差以后旋转到 b 系时使用到了该处的 DCM。即由姿态解算得到的有效姿态信息。

```
/PX4Firmware/src/modules/mc_att_control/mc_att_control_main.cpp
```

```
1  /* get current rotation matrix from control state quaternions */
2  math::Quaternion q_att(_ctrl_state.q[0], _ctrl_state.q[1], _ctrl_state.q[2],
3  _ctrl_state.q[3]);
4  math::Matrix<3, 3> R = q_att.to_dcm(); //通过 math 库构建四元数
5
6  /*DCM 的函数原型：*/
7  PX4Firmware/src/lib/mathlib/math/Quaternion.hpp
8
9  /**
10   * create rotation matrix for the quaternion
11   */
12   Matrix<3, 3> to_dcm(void) const {
13       Matrix<3, 3> R;
14       float aSq = data[0] * data[0];
15       float bSq = data[1] * data[1];
16       float cSq = data[2] * data[2];
17       float dSq = data[3] * data[3];
18       R.data[0][0] = aSq + bSq - cSq - dSq;
19       R.data[0][1] = 2.0f * (data[1] * data[2] - data[0] * data[3]);
20       R.data[0][2] = 2.0f * (data[0] * data[2] + data[1] * data[3]);
```



```

21     R.data[1][0] = 2.0f * (data[1] * data[2] + data[0] * data[3]);
22     R.data[1][1] = aSq - bSq + cSq - dSq;
23     R.data[1][2] = 2.0f * (data[2] * data[3] - data[0] * data[1]);
24     R.data[2][0] = 2.0f * (data[1] * data[3] - data[0] * data[2]);
25     R.data[2][1] = 2.0f * (data[0] * data[1] + data[2] * data[3]);
26     R.data[2][2] = aSq - bSq - cSq + dSq;
27     return R;
28 }

```

然后取两个矩阵中的 Z 轴向量，即 YAW-axis。

```
/PX4Firmware/src/modules/mc_att_control/mc_att_control_main.cpp
```

```

1  /* all input data is ready, run controller itself */
2
3  /* try to move thrust vector shortest way, because yaw response is slower
4  than roll/pitch */
5  math::Vector<3> R_z(R(0, 2), R(1, 2), R(2, 2));
6  math::Vector<3> R_sp_z(R_sp(0, 2), R_sp(1, 2), R_sp(2, 2));

```

当前姿态的 z 轴和目标姿态的 z 轴的误差大小（即需要旋转的角度）并旋转到 b 系（即先对齐 Z 轴）

```
/PX4Firmware/src/modules/mc_att_control/mc_att_control_main.cpp
```

```

1  /* axis and sin(angle) of desired rotation */
2  math::Vector<3> e_R = R.transposed() * (R_z % R_sp_z);

```

$R_z R_{sp_z}$ 叉积，就是求取误差的，本来应该 z 轴相互重合的，如果不是 0 就作为误差项。然后再左乘旋转矩阵旋转到 b 系。

然后计算姿态角度误差（姿态误差）。

数学知识背景：由公式

$$a \times b = |a| |b| \sin \theta, \quad a \cdot b = |a| |b| \cos \theta$$

这里的 R_z 和 R_{sp_z} 都是单位向量，模值为 1，因此误差向量 e_R ($a \times b$ 叉积就是误差) 的模就是 $\sin \theta$ ，点积就是 $\cos \theta$ 。

```
/PX4Firmware/src/modules/mc_att_control/mc_att_control_main.cpp
```

```
1  /* calculate angle error */
2
   float e_R_z_sin = e_R.length();
3
   float e_R_z_cos = R_z * R_sp_z;
```

计算 yaw 的权重

```
/PX4Firmware/src/modules/mc_att_control/mc_att_control_main.cpp
```

```
1  /* calculate weight for yaw control */
2
   float yaw_w = R_sp(2, 2) * R_sp(2, 2);
```

因为多轴的 yaw 响应一般比 roll/pitch 慢了接近一倍，因此将两者解耦，先补偿 roll-pitch 的变化，计算 R_{rp} 。

```
/PX4Firmware/src/modules/mc_att_control/mc_att_control_main.cpp
```

```
1  /* calculate rotation matrix after roll/pitch only rotation */
2
   math::Matrix<3, 3> R_rp;
```

```
3
4     if (e_R_z_sin > 0.0f) {
5         /* get axis-angle representation */
6         float e_R_z_angle = atan2f(e_R_z_sin, e_R_z_cos);
7         math::Vector<3> e_R_z_axis = e_R / e_R_z_sin;
8
9         e_R = e_R_z_axis * e_R_z_angle;
10
11        /* cross product matrix for e_R_axis */
12        math::Matrix<3, 3> e_R_cp;
13        e_R_cp.zero();
14        e_R_cp(0, 1) = -e_R_z_axis(2);
15        e_R_cp(0, 2) = e_R_z_axis(1);
16        e_R_cp(1, 0) = e_R_z_axis(2);
17        e_R_cp(1, 2) = -e_R_z_axis(0);
18        e_R_cp(2, 0) = -e_R_z_axis(1);
19        e_R_cp(2, 1) = e_R_z_axis(0);
20
21        /* rotation matrix for roll/pitch only rotation */
22        R_rp = R * (_I + e_R_cp * e_R_z_sin + e_R_cp * e_R_cp * (1.0f - e_R_z_cos));
23
24    } else {
25        /* zero roll/pitch rotation */
26        R_rp = R;
```

首先需要明确的就是上述处理过程中的 DCM 量都是通过欧拉角来表示的，这个主要就是考虑在控制时需要明确具体的欧拉角的大小，还有就是算法的解算过程是通过矩阵微分方程推导得到的。再判断两个 z 轴是否存在误差

($e_{R_z_sin} > 0.0f$)，若存在误差则通过反正切求出该误差角度值

($\text{atan2f}(e_{R_z_sin}, e_{R_z_cos})$)；然后归一化 $e_{R_z_axis}(e_R / e_{R_z_sin}$

该步计算主要就是利用 $e_{R_z_sin} = e_R.\text{length}()$ 。然后就是 $e_R = e_{R_z_axis} * e_{R_z_angle}$ 了（主要就是为误差向量用角度量表示）。

然后就是构造 $\text{Matrix}(3, 3)$ e_{R_cp} $3*3$ 的反对称阵 (skew-symmetric matrix)，在此处叫做姿态误差矩阵比较容易理解，有时也叫叉乘算子， $\wedge$ 操作符如下：

$$\hat{w} = \begin{bmatrix} 0 & -w_3 & w_2 \\ w_3 & 0 & -w_1 \\ -w_2 & w_1 & 0 \end{bmatrix}$$

其中 $w = [w_1, w_2, w_3]^T$ ， $-\hat{w}^T \theta = \hat{w} \theta$ 。

到这了就介绍一些关于叉乘 (cross product) 的知识吧：

已经有两个向量 x 、 y ，其叉乘公式为： $x \times y = \hat{x} * y$

获取反对称阵以后，利用推导公式求取 rp 的旋转矩阵。公式如下（罗德里格旋转）：

$$C_b^E(t + \Delta t) = C_b^E(t) \left[I + \frac{\sin \Delta \theta_0}{\Delta \theta_0} \Delta \theta_0^b + \frac{1 - \cos \Delta \theta_0}{\Delta \theta_0^2} (\Delta \theta_0^b)^2 \right] \quad (7.2.17)$$

关于上述理论公式的一点理解：其实知道四元数的指数表达式就容易理解这个了，四元数的指数表达式 (an axis-angle rotation with axis $\omega \in SO(3)$ and angle $\theta \in [0, 2\pi)$)：

$$R = e^{\hat{w}\theta} = I + \hat{w}\theta + \frac{(\hat{w}\theta)^2}{2!} + \dots = [\hat{w} \sin \theta, \cos \theta] \quad (\text{欧拉公式})$$

通过泰勒展开 $\sin \theta$ ， $\cos \theta$ ：

$$\begin{aligned} \sin \theta &= \theta + o(\theta) \\ \cos \theta &= 1 - \frac{\theta^2}{2} + o(\theta^2) \end{aligned}$$

所以，通过替换得到公式： $e^{\hat{w}\theta} = I + \hat{w} \sin \theta + \hat{w}^2 (1 - \cos \theta)$ ，推导结束。该公式就是著名的罗德里格旋转公式 (Rodrigues' rotation formula)。

Wiki: https://en.wikipedia.org/wiki/Rodrigues%27_rotation_formula

上述是关于一维数据的推导情况，公式都成立，没有任何问题。但是如何把 w 扩展到矩阵上呢？这个还是需要一部分推导过程的，该过程不再赘述了，自己看书吧（美国人和瑞典人写的一本《自动控制—多科学视角》_P108. 矩阵指数）。有时会碰到 $[w]_X$ 的形式，其实是一样的，一般在李群相关的问题中会用 $\hat{w}$ 即 $\hat{w} \triangleq [w]_X$ 。

然后计算 yaw 的误差, 该误差是 roll_pitch 获取的 z 轴和目标姿态的 z 轴的误差。

```
/PX4Firmware/src/modules/mc_att_control/mc_att_control_main.cpp
```

```
1  /* R_rp and R_sp has the same Z axis, calculate yaw error */
2
3  math::Vector<3> R_sp_x(R_sp(0, 0), R_sp(1, 0), R_sp(2, 0));
4
5  math::Vector<3> R_rp_x(R_rp(0, 0), R_rp(1, 0), R_rp(2, 0));
6
7  e_R(2) = atan2f((R_rp_x % R_sp_x) * R_sp_z, R_rp_x * R_sp_x) * yaw_w;
```

该部分同样是根据向量的叉积和点积求出误差角度的正弦和余弦, 再反正切求出角度

上面介绍的是在小角度变化时, 如果是大角度变化时 (大于 90° , 可能性比较小) 使用如何方法处理。

```
/PX4Firmware/src/modules/mc_att_control/mc_att_control_main.cpp
```

```
1  if (e_R_z_cos < 0.0f) {
2
3      /* for large thrust vector rotations use another rotation method:
4
5       * calculate angle and axis for R -> R_sp rotation directly */
6
7      math::Quaternion q;
8
9      q.from_dcm(R.transposed() * R_sp);
10
11      math::Vector<3> e_R_d = q.imag();
12
13      e_R_d.normalize();
14
15      e_R_d *= 2.0f * atan2f(e_R_d.length(), q(0));
16
17      /* use fusion of Z axis based rotation and direct rotation */
```

```

11     float direct_w = e_R_z_cos * e_R_z_cos * yaw_w;
12     e_R = e_R * (1.0f - direct_w) + e_R_d * direct_w;
13 }

```

上面这段代码比较好理解，主要就是由 DCM 获取四元数；然后把四元数的虚部取出赋值给 `e_R_d` (`e_R_d = q.imag()`)；然后对其进行归一化处理；最后 2 行是先求出互补系数，再通过互补方式求取 `e_R`。

然后捏：计算角速度变化的大小，并对其进行约束（constrain）。

`/PX4Firmware/src/modules/mc_att_control/mc_att_control_main.cpp`

```

1  /* calculate angular rates setpoint */
2  _rates_sp = _params.att_p.emult(e_R);
3
4  /* limit rates */
5  for (int i = 0; i < 3; i++) {
6      if (_v_control_mode.flag_control_velocity_enabled
7          && !_v_control_mode.flag_control_manual_enabled) {
8          _rates_sp(i) = math::constrain(_rates_sp(i),
9                                         -_params.auto_rate_max(i),
10                                         _params.auto_rate_max(i));
11      } else {
12          _rates_sp(i) = math::constrain(_rates_sp(i),
13                                         -_params.mc_rate_max(i),
14                                         _params.mc_rate_max(i));
15      }

```

```
16     }
17
18     /* weather-vane mode, dampen yaw rate */
19     if (_v_att_sp.disable_mc_yaw_control == true &&
20         _v_control_mode.flag_control_velocity_enabled
21         && !_v_control_mode.flag_control_manual_enabled) {
22         float wv_yaw_rate_max = _params.auto_rate_max(2) *
23             _params.vtol_wv_yaw_rate_scale;
24         _rates_sp(2) = math::constrain(_rates_sp(2), -wv_yaw_rate_max,
25             wv_yaw_rate_max);
26         // prevent integrator winding up in weathervane mode
27         _rates_int(2) = 0.0f;
28     }
29
30     /* feed forward yaw setpoint rate 因为 yaw 响应较慢, 再加入一个前馈控制 */
31     _rates_sp(2) += _v_att_sp.yaw_sp_move_rate * yaw_w * _params.yaw_ff;
32
33     /* weather-vane mode, scale down yaw rate */
34     if (_v_att_sp.disable_mc_yaw_control == true &&
35         _v_control_mode.flag_control_velocity_enabled
36         && !_v_control_mode.flag_control_manual_enabled) {
37         float wv_yaw_rate_max = _params.auto_rate_max(2) *
38             _params.vtol_wv_yaw_rate_scale;
39         _rates_sp(2) = math::constrain(_rates_sp(2), -wv_yaw_rate_max,
```

```
40         wv_yaw_rate_max);  
41  
42         // prevent integrator winding up in weathervane mode  
43         _rates_int(2) = 0.0f;  
44     }  
45 }
```

上述代码中的一个 `emult(e_R)` 的函数原型

PX4Firmware/src/lib/matrix/matrix/Matrix.hpp

```
1     Matrix<Type, M, N> emult(const Matrix<Type, M, N> &other) const  
2     {  
3         Matrix<Type, M, N> res;  
4         const Matrix<Type, M, N> &self = *this;  
5  
6         for (size_t i = 0; i < M; i++) {  
7             for (size_t j = 0; j < N; j++) {  
8                 res(i, j) = self(i, j)*other(i, j);  
9             }  
10        }  
11  
12        return res;  
13    }
```

所以 `_rates_sp = _params.att_p.emult(e_R)` 这句话的意思就是用 `att_p` 的每一个元素和 `e_R` 中对应位置的每一个元素相乘，结果赋值给 `_rates_sp` 角速度变量

5.4、control_attitude(dt) 返回以后

/PX4Firmware/src/modules/mc_att_control/mc_att_control_main.cpp

```
1  /* publish attitude rates setpoint */
2      _v_rates_sp.roll = _rates_sp(0);
3      _v_rates_sp.pitch = _rates_sp(1);
4      _v_rates_sp.yaw = _rates_sp(2);
5      _v_rates_sp.thrust = _thrust_sp;
6      _v_rates_sp.timestamp = hrt_absolute_time();
7
8      if (_v_rates_sp_pub != nullptr) {
9          orb_publish(_rates_sp_id, _v_rates_sp_pub, &_v_rates_sp);
10
11      } else if (_rates_sp_id) {
12          _v_rates_sp_pub = orb_advertise(_rates_sp_id, &_v_rates_sp);
13      }
```

上面这部分代码就通过 control_attitude(dt) 经过一系列的算法处理过以后获取的目标内环角速度值, 并通过 uORB 模型发布出去, 包含 roll-pitch-yaw、油门量和时间戳。

该处正好可以再次深入理解一下 uORB 模型的一些理论。上述代码涉及了 orb_publish() 和 orb_advertise() 两个函数接口, 通常第一次发布有效数据之前需要使用 orb_advertise() 函数进行广播 (类似 topic register), 它发布成功以后会返回一个 handle 供 orb_publish() 发布时使用, 即广播之后可以使用 orb_publish() 进行发布新的数据。orb_advertise() 发布函数有第一个参数

类似 ID, 返回值作为 handle 以便区分再次使用 orb_publish() 时发布的是何种消息数据, 即再次说明 orb_publish() 需要在 orb_advertise() 函数接口之后使用。通过查看 orb_advertise() 函数的代码原型可以了解到, 该函数的作用就类似于把需要后续发布的主题 (topic) 注册一下, 然后才可以进行 orb_publish()。

在 task_main() 的开头处就是订阅各种 topics, 其中就有一个 _v_rates_sp_sub = orb_subscribe(ORB_ID(vehicle_rates_setpoint)) 订阅过程, 它就是在该算法执行到最后时发布的控制量数据 “_v_rates_sp”, 也就是按照前讲述的理论, 自己订阅自己发布的 topic, 以实现循环控制。其中需要注意的就是发布时用的 ID 和订阅时用的不一致所迷惑了, 其实它俩是一样的; 因为在上述处理过程中是把 ORB_ID(vehicle_rates_setpoint) 赋值给 _rates_sp_id 它的, 赋值过程在发布 topic 之前, 即在 vehicle_status_poll() 函数内部。

前面的算法都是在 flag_control_attitude_enabled 非零 (姿态控制) 的情况下实现的。紧接着, 是在 flag_control_attitude_enabled 为零时, 即转变为 flag_control_manual_enabled: 手动控制, 方法类似, 不在赘述。再接着, 连手动控制都为使能时, 只能 poll 了, 并把控制量都置 0。

姿态角度控制结束。

5.5、姿态速度控制 (角速度环)

```
/PX4Firmware/src/modules/mc_att_control/mc_att_control_main.cpp
```

```
1 if (_v_control_mode.flag_control_rates_enabled) {  
2     control_attitude_rates(dt);  
3  
4     /* publish actuator controls */
```

```
5  _actuators.control[0] = (PX4_ISFINITE(_att_control(0))) ?
6      _att_control(0) : 0.0f;
7  _actuators.control[1] = (PX4_ISFINITE(_att_control(1))) ?
8      _att_control(1) : 0.0f;
9  _actuators.control[2] = (PX4_ISFINITE(_att_control(2))) ?
10     _att_control(2) : 0.0f;
11 _actuators.control[3] = (PX4_ISFINITE(_thrust_sp)) ? _thrust_sp : 0.0f;
12 _actuators.timestamp = hrt_absolute_time();
13 _actuators.timestamp_sample = _ctrl_state.timestamp;
14
15 _controller_status.roll_rate_integ = _rates_int(0);
16 _controller_status.pitch_rate_integ = _rates_int(1);
17 _controller_status.yaw_rate_integ = _rates_int(2);
18 _controller_status.timestamp = hrt_absolute_time();
19
20 if (!_actuators_0_circuit_breaker_enabled) {
21     if (_actuators_0_pub != nullptr) {
22
23         orb_publish(_actuators_id, _actuators_0_pub, &_amp;_actuators);
24         perf_end(_controller_latency_perf);
25
26     } else if (_actuators_id) {
27         _actuators_0_pub = orb_advertise(_actuators_id, &_amp;_actuators);
28     }
```

```
29
30     }
31
32     /* publish controller status */
33     if (_controller_status_pub != nullptr) {
34         orb_publish(ORB_ID(mc_att_ctrl_status), _controller_status_pub,
35                   &_controller_status);
36
37     } else {
38         _controller_status_pub = orb_advertise(ORB_ID(mc_att_ctrl_status),
39                                               &_controller_status);
40     }
41 }
```

进入上述代码首先就是 `control_attitude_rates(dt)`，该函数的输入是前面算法处理得到的 `_rates_sp` 控制量（目标姿态），输出是 `_att_control` 控制量。其函数原型是：

```
/PX4Firmware/src/modules/mc_att_control/mc_att_control_main.cpp
```

```
1  /*
2   * Attitude rates controller.
3   * Input: '_rates_sp' vector, '_thrust_sp'
4   * Output: '_att_control' vector
5   */
6  void MulticopterAttitudeControl::control_attitude_rates(float dt)
```

```
7 {
8     /* reset integral if disarmed */
9     if (!_armed.armed || !_vehicle_status.is_rotary_wing) {
10         _rates_int.zero();
11     }
12
13     /* current body angular rates */
14     math::Vector<3> rates;
15     rates(0) = _ctrl_state.roll_rate;
16     rates(1) = _ctrl_state.pitch_rate;
17     rates(2) = _ctrl_state.yaw_rate;
18
19     /* angular rates error */
20     math::Vector<3> rates_err = _rates_sp - rates; //目标姿态-当前姿态
21     _att_control = _params.rate_p.emult(rates_err) +
22         _params.rate_d.emult(_rates_prev - rates) / dt +
23         _rates_int +
24         _params.rate_ff.emult(_rates_sp - _rates_sp_prev) / dt;
25     _rates_sp_prev = _rates_sp;
26     _rates_prev = rates;
27
28     /* update integral only if not saturated on low limit and if motor commands are not
29     saturated */
30     if (_thrust_sp > MIN_TAKEOFF_THRUST && !_motor_limits.lower_limit
```

```
31     && !_motor_limits.upper_limit) {  
32     for (int i = 0; i < 3; i++) {  
33         if (fabsf(_att_control(i)) < _thrust_sp) {  
34             float rate_i = _rates_int(i) + _params.rate_i(i) * rates_err(i)  
35                 * dt;  
36  
37             if (PX4_ISFINITE(rate_i) && rate_i > -RATES_I_LIMIT && rate_i  
38                 < RATES_I_LIMIT &&  
39                 _att_control(i) > -RATES_I_LIMIT && _att_control(i)  
40                 < RATES_I_LIMIT) {  
41                 _rates_int(i) = rate_i;  
42             }  
43         }  
44     }  
45 }  
46 }
```

主要就是通过`_ctrl_state` 数据结构把需要的有效数据赋值给 `rates`, 然后通过 `rates` 进行一系列的算法处理。该过程中最最最需要注意的就是这个 `_ctrl_state` 变量的获取过程, 其实还是通过 `uORB`。前面也涉及过多次, 比如 `control_attitude()` 函数内部使用它构造状态四元数。

数据获取过程:

Quaternion_CF 姿态解算算法: (关于姿态解算部分使用的 CF 时, 在 `PX4Firmware/src/module/attitude_estimator_q` 中)。首先是通过姿态解算部分获取当前的姿态信息 (Quaternion_CF), 获取之后通过 `uORB` 模型发布:

```
PX4Firmware/src/module/attitude_estimator_q/attitude_estimator_q_main.cpp
```

```
1  /* publish to control state topic */ (638)
2  orb_publish_auto(ORB_ID(control_state), &ctrl_state_pub, &ctrl_state, &ctrl_inst,
3  ORB_PRIO_HIGH);
```

EKF2 姿态解算算法：（关于姿态解算部分使用的 ekf2 时，在 PX4Firmware/src/module/ekf2 中）。首先是通过姿态解算部分获取当前的姿态信息（ekf2），获取之后通过 uORB 模型发布：

```
// publish control state data (508)

1  if (_control_state_pub == nullptr) {
2      _control_state_pub = orb_advertise(ORB_ID(control_state), &ctrl_state);
3
4  } else {
5      orb_publish(ORB_ID(control_state), _control_state_pub, &ctrl_state);
6  }
```

关于到底使用哪种解算算法在启动脚本 rc_mc_app 里面涉及了关于姿态解算用什么算法的问题，里面给了一个宏，通过宏定义选取的。而且在使用四元数的互补算法和 ekf2 的算法里面都对结算到的姿态信息进行了发布处理，以便供姿态控制时订阅使用。

然后再姿态控制中通过 uORB 模型订阅：

```
/PX4Firmware/src/modules/mc_att_control/mc_att_control_main.cpp
```

```
1  _ctrl_state_sub = orb_subscribe(ORB_ID(control_state)); (815)
2  orb_copy(ORB_ID(control_state), _ctrl_state_sub, &_ctrl_state); (867)
```

再然后就是姿态控制量（_att_control）的获取：获取原则是由预期姿态控制获取的角速度值与通过 uORB 获得的角速度值做差（该部分差值代表 $\text{error} = \text{target} - \text{current}$ ，_ctrl_state 应该是要控制的控制量）。rates_err 的获取就是通过经典的 PD 控制器了，然后再加个前馈。还未使用 I 控制器；在后面会单独使用。

/PX4Firmware/src/modules/mc_att_control/mc_att_control_main.cpp

```
1  /* angular rates error */
2  math::Vector<3> rates_err = _rates_sp - rates;
3  _att_control = _params.rate_p.emult(rates_err) +
4                _params.rate_d.emult(_rates_prev - rates) / dt + _rates_int +
5                _params.rate_ff.emult(_rates_sp - _rates_sp_prev) / dt;
6  _rates_sp_prev = _rates_sp;
7  _rates_prev = rates;
8
9  /* update integral only if not saturated on low limit and if motor commands are not
10 saturated */
11 if (_thrust_sp > MIN_TAKEOFF_THRUST && !_motor_limits.lower_limit
12 && !_motor_limits.upper_limit) {
13     for (int i = 0; i < 3; i++) {
14         if (fabsf(_att_control(i)) < _thrust_sp) {
15             float rate_i = _rates_int(i) + _params.rate_i(i) * rates_err(i) *
16                             dt;
17
18             if (PX4_ISFINITE(rate_i) && rate_i > -RATES_I_LIMIT && rate_i <
```



```

19         RATES_I_LIMIT &&_att_control(i) > -RATES_I_LIMIT &&
20         _att_control(i) < RATES_I_LIMIT) {
21             _rates_int(i) = rate_i;
22         }
23     }
24 }
25 ]
  
```

注意上面的 `fabsf(_att_control(i)) < _thrust_sp` 这个判断项，符合就执行积分。这个做主要是为了安全考虑，当 `roll` 的变化值需要很大时，就停止积分项的累加以防止积分项产生较大的误差。

别看这个 `_thrust_sp` 单单的一个控制量，其实它可麻烦了，不对整体核心的解算和控制（姿态解算姿态控制、位置解算位置控制）有个深入理解的话，很难看懂这部分。下面详细介绍一下这个控制量的获取过程。

MPC_THR_MAX	Maximum thrust in auto thrust control	0.0	0.95	0.9
	Limit max allowed thrust. Setting a value of one can put the system into actuator saturation as no spread between the motors is possible any more. A value of 0.8 - 0.9 is recommended.			

首先是 `_v_att_sp_sub`
`=orb_subscribe(ORB_ID(vehicle_attitude_setpoint));` 订阅所需的控制量。
 然后再 `attitude control` 里面处理：`_thrust_sp =_v_att_sp.thrust`

上面是订阅拷贝和使用部分，下面就是发布部分。

发布分为两个地方，一个是 `mc_pos_control` 和 `mavlink_receiver.cpp`。
 主要考虑前者。

```
ID 重定义: _attitude_setpoint_id=  
ORB_ID(vehicle_attitude_setpoint);
```

正式发布给:

```
mc_att_control:  
orb_publish(_attitude_setpoint_id, _att_sp_pub, &_att_sp);
```

为何称为正式发布呢？主要是在 `mc_pos_control` 里面根据不懂的模式进行了多次发布处理，比如 `idle` 状态下这个 `_thrust_sp` 就赋值为 0 发布出去。这个正式发布出来的才是我们飞行控制过程中需要考虑的控制量。

任务间通信（IPC）都是靠的 uORB，找不到来源就查 ID

5.6、发布控制量

```
/PX4Firmware/src/modules/mc_att_control/mc_att_control_main.cpp
```

```
1  /* publish actuator controls */  
2      _actuators.control[0] = (PX4_ISFINITE(_att_control(0))) ?  
3          _att_control(0) : 0.0f;  
4      _actuators.control[1] = (PX4_ISFINITE(_att_control(1))) ?  
5          _att_control(1) : 0.0f;  
6      _actuators.control[2] = (PX4_ISFINITE(_att_control(2))) ?  
7          _att_control(2) : 0.0f;  
8      _actuators.control[3] = (PX4_ISFINITE(_thrust_sp)) ?  
9          _thrust_sp : 0.0f;  
10     _actuators.timestamp = hrt_absolute_time();  
11     _actuators.timestamp_sample = _ctrl_state.timestamp;
```

```
12
13     _controller_status.roll_rate_integ = _rates_int(0);
14     _controller_status.pitch_rate_integ = _rates_int(1);
15     _controller_status.yaw_rate_integ = _rates_int(2);
16     _controller_status.timestamp = hrt_absolute_time();
17
18     if (!_actuators_0_circuit_breaker_enabled) {
19         if (_actuators_0_pub != nullptr) {
20
21             orb_publish(_actuators_id, _actuators_0_pub,
22
23                 &_amp;_actuators);
24
25             perf_end(_controller_latency_perf);
26
27             } else if (_actuators_id) {
28
29                 _actuators_0_pub = orb_advertise(_actuators_id,
30
31                     &_amp;_actuators);
32
33             }
34
35     }
36
37     /* publish controller status */
38
39     if (_controller_status_pub != nullptr) {
40
41         orb_publish(ORB_ID(mc_att_ctrl_status),
42
43             _controller_status_pub, &_amp;_controller_status);
```

```
36
37         } else {
38             _controller_status_pub =
39                 orb_advertise(ORB_ID(mc_att_ctrl_status),
40                             &_controller_status);
41         }
42     }
43 }
44
45 perf_end(_loop_perf);
46 }
47
48 _control_task = -1;
49 return;
```

5.7、总结

其实在 `mc_att_control` 里面就完全涵盖了姿态控制的内环和外环（即角速度控制、角度控制）。主要就是 `attitude control` 和 `attitude rate control` 两个部分，前者是控制角度后者是控制角速度并把控制量输入给 `mixer`。在控制过程中是通过控制电机的速度以实现多旋翼的整体的 `rpy` 的速度，通过这个速度随时间的累加实现角度控制。

`attitude_control` 输入是体轴矩阵 `R` 和期望的体轴矩阵 `Rsp`，角度环只是一个 `P` 控制，算出来之后输出的是期望的角速度值 `rate_sp`（这一段已经完成了所需要的角度变化，并将角度的变化值转换到了需要的角速度值）。并且把

加速度值直接输出给 attitude rate control，再经过角速度环的 pid 控制，输出值直接就给 mixer，然后控制电机输出了。

关于这些，主要还是需要理解这个控制过程：一方面是通过姿态解算部分获取的实时的姿态信息，并通过 uORB 模型把姿态信息发布出去；姿态控制部分订阅姿态解算得到的姿态信息。然后通过 attitude control 获取目标姿态和当前姿态的角度差值并经过算法处理得到对应的角速度值，并把这个角速度值输出给 attitude rate control 最终获取到需求的控制量。输出给 mixer。其实 attitude control 输出是需要达到这个误差角度时所需要的角速度值，用这个值与当前的角速度值做差，求出现在需要的角速度值而已。这个就是为什么控制角速度的原因，进而达到控制角度的效果。

第六章 飞行模式切换

6.1、整体流程

1、sensors.cpp 发布 ORB_ID(manual_control_setpoint);

2、commander.cpp 里 set_main_state_rc() 函数里的 main_state_transition() 函数根

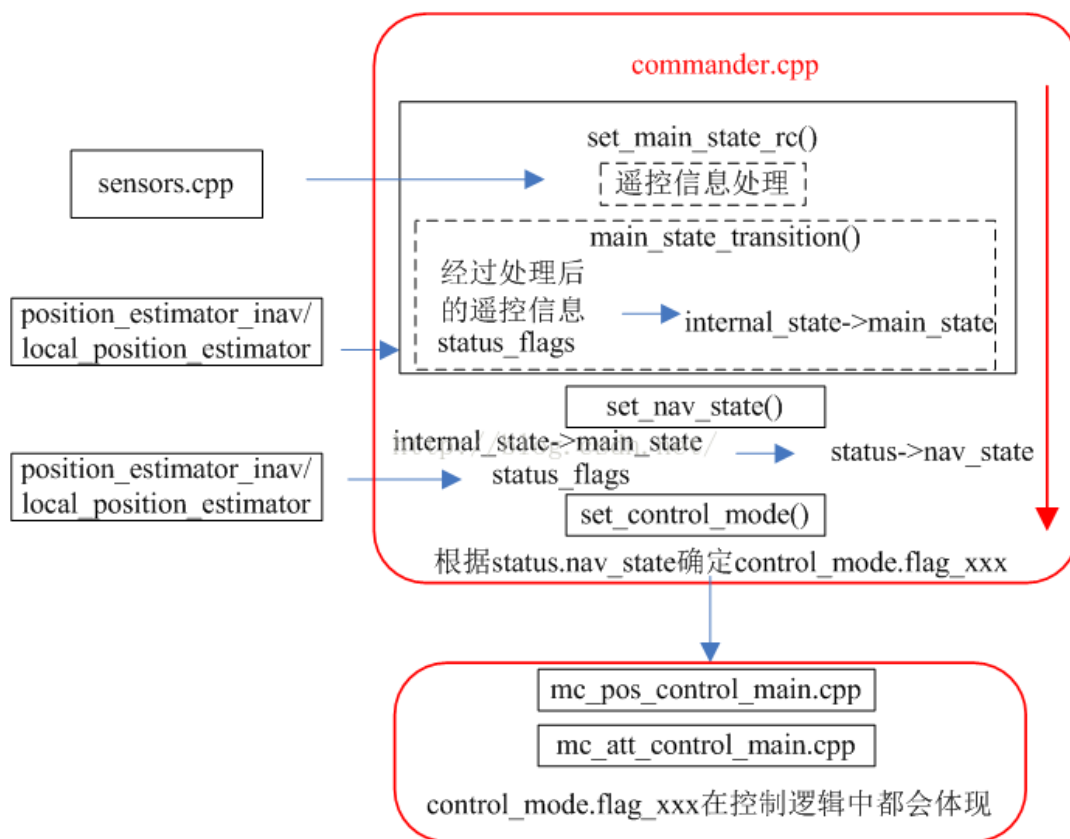
据遥控信息和飞行器状态 status_flags 决定是否能更变

internal_state->main_state;

3、commander.cpp 里 set_nav_state() 函数根据 internal_state->main_state 和飞行器状态 status_flags (传感器等硬件正常否) 确定能否完成

internal_state->main_state 所指定的模式，若飞行器状态不行，则将模式更新为 status->nav_state;

4、commander.cpp 里 set_control_mode() 函数根据 status.nav_state 确定 control_mode.flag_xxx。



6.2、遥控器端

Firmware/src/modules/sensors/sensors.cpp 中 void Sensors::rc_poll()

发布 ORB_ID(manual_control_setpoint)

```

1  /* only publish manual control if the signal is still present */
2  if (!signal_lost) {
3
4      /* initialize manual setpoint */
5      struct manual_control_setpoint_s manual = {};
6
7      /* set mode slot to unassigned */
8      manual.mode_slot = manual_control_setpoint_s::MODE_SLOT_NONE;
9
10     /* set the timestamp to the last signal time */
  
```

```

9      manual.timestamp = rc_input.timestamp_last_signal;

10

11     /* limit controls */

12     manual.y = get_rc_value(rc_channels_s::RC_CHANNELS_FUNCTION_ROLL, -1.0,

13                             1.0);

14     manual.x = get_rc_value(rc_channels_s::RC_CHANNELS_FUNCTION_PITCH, -1.0,

15                             1.0);

16     manual.r = get_rc_value(rc_channels_s::RC_CHANNELS_FUNCTION_YAW, -1.0,

17                             1.0);

18     manual.z = get_rc_value(rc_channels_s::RC_CHANNELS_FUNCTION_THROTTLE, 0.0, 1.0);

19     manual.flaps = get_rc_value(rc_channels_s::RC_CHANNELS_FUNCTION_FLAPS, -1.0,

20                                 1.0);

21     manual.aux1 = get_rc_value(rc_channels_s::RC_CHANNELS_FUNCTION_AUX_1, -1.0, 1.0);

22     manual.aux2 = get_rc_value(rc_channels_s::RC_CHANNELS_FUNCTION_AUX_2, -1.0, 1.0);

23     manual.aux3 = get_rc_value(rc_channels_s::RC_CHANNELS_FUNCTION_AUX_3, -1.0, 1.0);

24     manual.aux4 = get_rc_value(rc_channels_s::RC_CHANNELS_FUNCTION_AUX_4, -1.0, 1.0);

25     manual.aux5 = get_rc_value(rc_channels_s::RC_CHANNELS_FUNCTION_AUX_5, -1.0, 1.0);

26

27     if (_parameters.rc_map_flightmode > 0) {

28

29         /* the number of valid slots equals the index of the max marker minus one */

30         const unsigned num_slots = manual_control_setpoint_s::MODE_SLOT_MAX;

31

32         /* the half width of the range of a slot is the total range
  
```



```
33     * divided by the number of slots, again divided by two
34     */
35     const float slot_width_half = 2.0f / num_slots / 2.0f;
36
37     /* min is -1, max is +1, range is 2. We offset below min and max */
38     const float slot_min = -1.0f - slot_width_half;
39     const float slot_max = 1.0f + slot_width_half;
40
41     /* the slot gets mapped by first normalizing into a 0..1 interval using min
42     * and max. Then the right slot is obtained by multiplying with the number of
43     * slots. And finally we add half a slot width to ensure that integer rounding
44     * will take us to the correct final index.
45     */
46     manual.mode_slot = (((((rc.channels[_parameters.rc_map_flightmode -
47                             1] - slot_min) * num_slots) + slot_width_half) /
48                             (slot_max - slot_min)) + (1.0f / num_slots));
49 }
50
51 /* mode switches */
52 manual.mode_switch = get_rc_sw3pos_position(
53     rc_channels_s::RC_CHANNELS_FUNCTION_MODE,
54     _parameters.rc_auto_th,
55     _parameters.rc_auto_inv,
56     _parameters.rc_assist_th,
57     _parameters.rc_assist_inv);
```

```
58     manual.rattitude_switch = get_rc_sw2pos_position(  
59         rc_channels_s::RC_CHANNELS_FUNCTION_RATTITUDE,  
60         _parameters.rc_rattitude_th,  
61         _parameters.rc_rattitude_inv);  
62     manual.posctl_switch = get_rc_sw2pos_position(  
63         rc_channels_s::RC_CHANNELS_FUNCTION_POSCTL,  
64         _parameters.rc_posctl_th,  
65         _parameters.rc_posctl_inv);  
66     manual.return_switch = get_rc_sw2pos_position(  
67         rc_channels_s::RC_CHANNELS_FUNCTION_RETURN,  
68         _parameters.rc_return_th,  
69         _parameters.rc_return_inv);  
70     manual.loiter_switch = get_rc_sw2pos_position(  
71         rc_channels_s::RC_CHANNELS_FUNCTION_LOITER,  
72         _parameters.rc_loiter_th,  
73         _parameters.rc_loiter_inv);  
74     manual.acro_switch = get_rc_sw2pos_position(  
75         rc_channels_s::RC_CHANNELS_FUNCTION_ACRO,  
76         _parameters.rc_acro_th,  
77         _parameters.rc_acro_inv);  
78     manual.offboard_switch = get_rc_sw2pos_position(  
79         rc_channels_s::RC_CHANNELS_FUNCTION_OFFBOARD,  
80         _parameters.rc_offboard_th,  
81         _parameters.rc_offboard_inv);
```

```
82     manual.kill_switch = get_rc_sw2pos_position(  
83         rc_channels_s::RC_CHANNELS_FUNCTION_KILLSWITCH,  
84         _parameters.rc_killswitch_th,  
85         _parameters.rc_killswitch_inv);  
86  
87     /* publish manual_control_setpoint topic */  
88     if (_manual_control_pub != nullptr) {  
89         orb_publish(ORB_ID(manual_control_setpoint), _manual_control_pub,  
90             &manual);  
91  
92     } else {  
93         _manual_control_pub = orb_advertise(ORB_ID(manual_control_setpoint),  
94             &manual);  
95     }
```

Commander.cpp 的主程序 commander_thread_main 中

PX4Firmware/src/modules/commander/commander.cpp

```
1     /* RC input check */  
2     if (!status.rc_input_blocked && sp_man.timestamp != 0 &&  
3         (hrt_absolute_time() < sp_man.timestamp + (uint64_t)(rc_loss_timeout * 1e6f))) {  
4         /* handle the case where RC signal was regained */  
5         /* 处理信号失而复得的情况 */  
6         if (!status.rc_signal_found_once) {  
7             status.rc_signal_found_once = true;
```

```
8      status_changed = true;
9
10     } else {
11         if (status.rc_signal_lost) {
12             mavlink_log_info(&mavlink_log_pub, "MANUAL CONTROL REGAINED
13 after %llums",
14
15                               (hrt_absolute_time() - status.rc_signal_lost_timestamp) /
16                               1000);
17
18             status_changed = true;
19
20         }
21     }
22
23     status.rc_signal_lost = false;
24
25
26     /* check if left stick is in lower left position and we are in MANUAL, *Rattitude,
27 or AUTO_READY mode or (ASSIST mode and landed) -> disarm
28
29     * do it only for rotary wings
30
31     */
32
33     /*检查油门杆在左下角的位置&&在手动&&(Rattitude||AUTO_READY mode||ASSIST
34
35     * mode and landed, 如果是, 则上锁
36
37     */
38
39     if (status.is_rotary_wing && status.rc_input_mode !=
40 vehicle_status_s::RC_IN_MODE_OFF &&
41
42         (status.arming_state == vehicle_status_s::ARMING_STATE_ARMED ||
43 status.arming_state == vehicle_status_s::ARMING_STATE_ARMED_ERROR) &&
44
45         (status.main_state == vehicle_status_s::MAIN_STATE_MANUAL ||
```

```
34     status.main_state = vehicle_status_s::MAIN_STATE_ACRO ||
35     status.main_state = vehicle_status_s::MAIN_STATE_STAB ||
36     status.main_state = vehicle_status_s::MAIN_STATE_RATTITUDE ||
37     status.condition_landed) &&
38     sp_man.r < -STICK_ON_OFF_LIMIT && sp_man.z < 0.1f) {
39
40     if (stick_off_counter > STICK_ON_OFF_COUNTER_LIMIT) {
41         /* disarm to STANDBY if ARMED or to STANDBY_ERROR if ARMED_ERROR */
42         arming_state_t new_arming_state = (status.arming_state =
43             vehicle_status_s::ARMING_STATE_ARMED ?
44             vehicle_status_s::ARMING_STATE_STANDBY :
45             vehicle_status_s::ARMING_STATE_STANDBY_ERROR);
46         arming_ret = arming_state_transition(&status,
47             &safety,
48             new_arming_state,
49             &armed,
50             true /* fRunPreArmChecks */,
51             &mavlink_log_pub);
52
53         if (arming_ret == TRANSITION_CHANGED) {
54             arming_state_changed = true;
55         }
56
57         stick_off_counter = 0;
```

```
58
59     } else {
60         stick_off_counter++;
61     }
62
63 } else {
64     stick_off_counter = 0;
65 }
66
67 /* check if left stick is in lower right position
68  * and we're in MANUAL mode -> arm
69  */
70 /* 检查油门杆在右下角的位置&&手动模式，如果是，则解锁 */
71 if (sp_man.r > STICK_ON_OFF_LIMIT && sp_man.z < 0.1f && status.rc_input_mode !=
72     vehicle_status_s::RC_IN_MODE_OFF ) {
73     if (stick_on_counter > STICK_ON_OFF_COUNTER_LIMIT) {
74
75         /* we check outside of the transition function here because the *requirement
76          for being in manual mode only applies to manual arming
77
78         *actions.the system can be armed in auto if armed via the GCS.
79
80         */
81
82         if ((status.main_state != vehicle_status_s::MAIN_STATE_MANUAL)
83             && (status.main_state != vehicle_status_s::MAIN_STATE_ACRO)
84             && (status.main_state != vehicle_status_s::MAIN_STATE_STAB)
```

```
83         && (status.main_state != vehicle_status_s::MAIN_STATE_ALTCTL))
84     {
85         print_reject_arm("NOT ARMING: Switch to a manual mode first.");
86
87     } else if (!status.condition_home_position_valid &&
88               geofence_action == geofence_result_s::GF_ACTION_RTL) {
89         print_reject_arm("NOT ARMING: Geofence RTL requires valid home");
90
91     } else if (status.arming_state ==
92               vehicle_status_s::ARMING_STATE_STANDBY)
93     {
94         arming_ret = arming_state_transition(&status,
95                                             &safety,
96                                             vehicle_status_s::ARMING_STATE_ARMED,
97                                             &armed,
98                                             true /* fRunPreArmChecks */,
99                                             &mavlink_log_pub);
100
101         if (arming_ret == TRANSITION_CHANGED) {
102             arming_state_changed = true;
103         } else {
104             usleep(100000);
105             print_reject_arm("NOT ARMING: Configuration error");
106         }
107     }
```

```
107     }
108     stick_on_counter = 0;
109
110     } else {
111         stick_on_counter++;
112     }
113
114     } else {
115         stick_on_counter = 0;
116     }
117
118     if (arming_ret == TRANSITION_CHANGED) {
119         if (status.arming_state == vehicle_status_s::ARMING_STATE_ARMED) {
120             mavlink_log_info(&mavlink_log_pub, "ARMED by RC");
121
122         } else {
123             mavlink_log_info(&mavlink_log_pub, "DISARMED by RC");
124         }
125
126         arming_state_changed = true;
127
128     } else if (arming_ret == TRANSITION_DENIED) {
129         /*
130         * the arming transition can be denied to a number of reasons:
```



```
131     * - pre-flight check failed (sensors not ok or not calibrated)
132     * - safety not disabled
133     * - system not in manual mode
134     */
135     tune_negative(true);
136 }
137
138 /* evaluate the main state machine according to mode switches */
139 bool first_rc_eval = (_last_sp_man.timestamp == 0) && (sp_man.timestamp > 0);
140 transition_result_t main_res = set_main_state_rc(&status, &sp_man);
141
142 /* play tune on mode change only if armed, blink LED always */
143 if (main_res == TRANSITION_CHANGED || first_rc_eval) {
144     tune_positive(armed.armed);
145     main_state_changed = true;
146
147 } else if (main_res == TRANSITION_DENIED) {
148     /* DENIED here indicates bug in the commander */
149     mavlink_log_critical(&mavlink_log_pub, "main state transition denied");
150 }
151
152 /* check throttle kill switch */
153 if (sp_man.kill_switch == manual_control_setpoint_s::SWITCH_POS_ON) {
154     /* set lockdown flag */
```

```
155      /* 设置锁定标志 */
156      if (!armed.lockdown) {
157          mavlink_log_emergency(&mavlink_log_pub, "MANUAL KILL SWITCH ENGAGED");
158      }
159      armed.lockdown = true;
160  } else if (sp_man.kill_switch == manual_control_setpoint_s::SWITCH_POS_OFF) {
161      if (armed.lockdown) {
162          mavlink_log_emergency(&mavlink_log_pub, "MANUAL KILL SWITCH OFF");
163      }
164      armed.lockdown = false;
165  }
166      /* no else case: do not change lockdown flag in unconfigured case */
167
168  } else {
169      if (!status.rc_input_blocked && !status.rc_signal_lost) {
170          mavlink_log_critical(&mavlink_log_pub, "MANUAL CONTROL LOST (at t=%llums)",
171 hrt_absolute_time() / 1000);
172          status.rc_signal_lost = true;
173          status.rc_signal_lost_timestamp = sp_man.timestamp;
174          status_changed = true;
175      }
176  }
```

6.3、飞控端

set_main_state_rc() 函数判断 RC 传来的数据，然后调用 main_state_transition(), main_state_transition() 根据遥控信息和飞行器状态 status_flags 决定是否变更 internal_state→main_state。

PX4Firmware/src/modules/commander/state_machine_helper.cpp

```
1 transition_result_t
2 main_state_transition(struct vehicle_status_s *status, main_state_t new_main_state)
3 {
4     transition_result_t ret = TRANSITION_DENIED;
5
6     /* transition may be denied even if the same state is requested because conditions
7    may have changed */
8
9     switch (new_main_state) {
10
11         case vehicle_status_s::MAIN_STATE_MANUAL:
12
13         case vehicle_status_s::MAIN_STATE_ACRO:
14
15         case vehicle_status_s::MAIN_STATE_RATTITUDE:
16
17         case vehicle_status_s::MAIN_STATE_STAB:
18
19             ret = TRANSITION_CHANGED;
20
21             break;
22
23         case vehicle_status_s::MAIN_STATE_ALTCTL:
24
25             /* need at minimum altitude estimate */
26
27             /* TODO: add this for fixedwing as well */
28
29             if (!status->is_rotary_wing ||
```

```
20         (status->condition_local_altitude_valid ||
21         status->condition_global_position_valid)) {
22         ret = TRANSITION_CHANGED;
23     }
24     break;
25
26     case vehicle_status_s::MAIN_STATE_POSCTL:
27         /* need at minimum local position estimate */
28         if (status->condition_local_position_valid ||
29         status->condition_global_position_valid) {
30         ret = TRANSITION_CHANGED;
31     }
32     break;
33
34     case vehicle_status_s::MAIN_STATE_AUTO_LOITER:
35         /* need global position estimate */
36         if (status->condition_global_position_valid) {
37         ret = TRANSITION_CHANGED;
38     }
39     break;
40
41     case vehicle_status_s::MAIN_STATE_AUTO_FOLLOW_TARGET:
42     case vehicle_status_s::MAIN_STATE_AUTO_MISSION:
43     case vehicle_status_s::MAIN_STATE_AUTO_RTL:
```

```
44     case vehicle_status_s::MAIN_STATE_AUTO_TAKEOFF:
45
46         /* need global position and home position */
47
48         if (status->condition_global_position_valid &&
49 status->condition_home_position_valid) {
50
51             ret = TRANSITION_CHANGED;
52
53             }
54
55             break;
56
57
58
59
60
61
62     case vehicle_status_s::MAIN_STATE_OFFBOARD:
63
64         /* need offboard signal */
65
66         if (!status->offboard_control_signal_lost) {
67
68             ret = TRANSITION_CHANGED;
69
70             }
71
72             break;
73
74
75
76
77
78
79
80
81
82     case vehicle_status_s::MAIN_STATE_MAX:
83
84     default:
85
86         break;
87
88     }
89
90     if (ret == TRANSITION_CHANGED) {
91
92         if (status->main_state != new_main_state) {
```

```
68         status->main_state_prev = status->main_state;
69         status->main_state = new_main_state;
70     } else {
71         ret = TRANSITION_NOT_CHANGED;
72     }
73 }
74
75 return ret;
76 }
```

当遥控器的开关处于不同位置时，调用函数 `main_state_transition()` 输入的是不同的控制模式，也就是说通过这段函数，才将开关位置与飞行模式连接起来，最重要的是函数 `main_state_transition()` 还给一个重要的标志位赋值 `state->main_state`。

`set_nav_state()` 根据 `state->main_state` 和飞行器状态 `status_flags` (传感器等硬件正常否) 确定能否完成 `state->main_state` 所指定的模式，若飞行器状态不行，则将模式跟新为 `status->nav_state`。

PX4Firmware/src/modules/commander/state_machine_helper.cpp

```
1  /**
2   * Check failsafe and main status and set navigation status for navigator accordingly
3   */
4  bool set_nav_state(struct vehicle_status_s *status, const bool data_link_loss_enabled,
5  const bool mission_finished,
6          const bool stay_in_failsafe)
7  {
```

```
8     navigation_state_t nav_state_old = status->nav_state;
9
10    bool armed = (status->arming_state == vehicle_status_s::ARMING_STATE_ARMED ||
11 status->arming_state == vehicle_status_s::ARMING_STATE_ARMED_ERROR);
12
13    status->failsafe = false;
14
15    /* evaluate main state to decide in normal (non-failsafe) mode */
16
17    switch (status->main_state) {
18
19    case vehicle_status_s::MAIN_STATE_ACRO:
20
21    case vehicle_status_s::MAIN_STATE_MANUAL:
22
23    case vehicle_status_s::MAIN_STATE_RATTITUDE:
24
25    case vehicle_status_s::MAIN_STATE_STAB:
26
27    case vehicle_status_s::MAIN_STATE_ALTCTL:
28
29    case vehicle_status_s::MAIN_STATE_POSCTL:
30
31    /* require RC for all manual modes */
32
33    if ((status->rc_signal_lost || status->rc_signal_lost_cmd) && armed
34 && !status->condition_landed) {
35
36    status->failsafe = true;
37
38
39    if (status->condition_global_position_valid &&
40 status->condition_home_position_valid) {
41
42    status->nav_state =
43
44    vehicle_status_s::NAVIGATION_STATE_AUTO_RCRECOVER;
45
46    } else if (status->condition_local_position_valid) {
47
48    status->nav_state =
```

```
33         vehicle_status_s::NAVIGATION_STATE_AUTO_LAND;
34     } else if (status->condition_local_altitude_valid) {
35         status->nav_state =
36             vehicle_status_s::NAVIGATION_STATE_DESCEND;
37     } else {
38         status->nav_state =
39             vehicle_status_s::NAVIGATION_STATE_TERMINATION;
40     }
41
42 } else {
43     switch (status->main_state) {
44     case vehicle_status_s::MAIN_STATE_ACRO:
45         status->nav_state = vehicle_status_s::NAVIGATION_STATE_ACRO;
46         break;
47
48     case vehicle_status_s::MAIN_STATE_MANUAL:
49         status->nav_state = vehicle_status_s::NAVIGATION_STATE_MANUAL;
50         break;
51
52     case vehicle_status_s::MAIN_STATE_RATTITUDE:
53         status->nav_state =
54             vehicle_status_s::NAVIGATION_STATE_RATTITUDE;
55         break;
56
```



```
57         case vehicle_status_s::MAIN_STATE_STAB:
58             status->nav_state = vehicle_status_s::NAVIGATION_STATE_STAB;
59             break;
60
61         case vehicle_status_s::MAIN_STATE_ALTCTL:
62             status->nav_state = vehicle_status_s::NAVIGATION_STATE_ALTCTL;
63             break;
64
65         case vehicle_status_s::MAIN_STATE_POSCTL:
66             status->nav_state = vehicle_status_s::NAVIGATION_STATE_POSCTL;
67             break;
68
69         default:
70             status->nav_state = vehicle_status_s::NAVIGATION_STATE_MANUAL;
71             break;
72     }
73 }
74 break;
75
76 case vehicle_status_s::MAIN_STATE_AUTO_MISSION:
77
78     /* go into failsafe
79     * - if commanded to do so
80     * - if we have an engine failure
```

```
81      * - if we have vtol transition failure
82
83      * - depending on datalink, RC and if the mission is finished */
84
85      /* first look at the commands */
86
87      if (status->engine_failure_cmd) {
88
89          status->nav_state =
90
91              vehicle_status_s::NAVIGATION_STATE_AUTO_LANDENGFAIL;
92
93      } else if (status->data_link_lost_cmd) {
94
95          status->nav_state = vehicle_status_s::NAVIGATION_STATE_AUTO_RTGS;
96
97      } else if (status->gps_failure_cmd) {
98
99          status->nav_state =
100
101              vehicle_status_s::NAVIGATION_STATE_AUTO_LANDGPSFAIL;
102
103      } else if (status->rc_signal_lost_cmd) {
104
105          status->nav_state =
106
107              vehicle_status_s::NAVIGATION_STATE_AUTO_RCRECOVER;
108
109      } else if (status->vtol_transition_failure_cmd) {
110
111          status->nav_state = vehicle_status_s::NAVIGATION_STATE_AUTO_RTL;
112
113      }
114
115      /* finished handling commands which have priority, now handle failures */
116
117      } else if (status->gps_failure) {
118
119          status->nav_state =
120
121              vehicle_status_s::NAVIGATION_STATE_AUTO_LANDGPSFAIL;
122
123      } else if (status->engine_failure) {
124
125          status->nav_state =
```

```
105         vehicle_status_s::NAVIGATION_STATE_AUTO_LANDENGFAIL;
106     } else if (status->vtol_transition_failure) {
107         status->nav_state = vehicle_status_s::NAVIGATION_STATE_AUTO_RTL;
108     } else if (status->mission_failure) {
109         status->nav_state = vehicle_status_s::NAVIGATION_STATE_AUTO_RTL;
110
111     /* datalink loss enabled:
112      * check for datalink lost: this should always trigger RTGS */
113     } else if (data_link_loss_enabled && status->data_link_lost) {
114         status->failsafe = true;
115
116         if (status->condition_global_position_valid &&
117 status->condition_home_position_valid) {
118             status->nav_state =
119                 vehicle_status_s::NAVIGATION_STATE_AUTO_RTGS;
120         } else if (status->condition_local_position_valid) {
121             status->nav_state =
122                 vehicle_status_s::NAVIGATION_STATE_AUTO_LAND;
123         } else if (status->condition_local_altitude_valid) {
124             status->nav_state =
125                 vehicle_status_s::NAVIGATION_STATE_DESCEND;
126         } else {
127             status->nav_state =
128                 vehicle_status_s::NAVIGATION_STATE_TERMINATION;
```

```

129         }
130
131         /* datalink loss disabled:
132
133         * check if both, RC and datalink are lost during the mission
134
135         * or RC is lost after the mission is finished: this should always trigger
136         RCRECOVER */
137
138         } else if (!data_link_loss_enabled && ((status->rc_signal_lost &&
139         status->data_link_lost) ||
140
141                 (status->rc_signal_lost && mission_finished))) {
142
143             status->failsafe = true;
144
145
146             if (status->condition_global_position_valid &&
147             status->condition_home_position_valid) {
148
149                 status->nav_state =
150
151                 vehicle_status_s::NAVIGATION_STATE_AUTO_RCRECOVER;
152
153             } else if (status->condition_local_position_valid) {
154
155                 status->nav_state =
156
157                 vehicle_status_s::NAVIGATION_STATE_AUTO_LAND;
158
159             } else if (status->condition_local_altitude_valid) {
160
161                 status->nav_state =
162
163                 vehicle_status_s::NAVIGATION_STATE_DESCEND;
164
165             } else {
166
167                 status->nav_state =
168
169                 vehicle_status_s::NAVIGATION_STATE_TERMINATION;
170
171             }
172

```

```
154
155     /* stay where you are if you should stay in failsafe, otherwise everything is
156 perfect */
157     } else if (!stay_in_failsafe) {
158         status->nav_state =
159             vehicle_status_s::NAVIGATION_STATE_AUTO_MISSION;
160     }
161     break;
162
163     case vehicle_status_s::MAIN_STATE_AUTO_LOITER:
164         /* go into failsafe on a engine failure */
165         if (status->engine_failure) {
166             status->nav_state =
167                 vehicle_status_s::NAVIGATION_STATE_AUTO_LANDENGFAIL;
168             /* also go into failsafe if just datalink is lost */
169             } else if (status->data_link_lost && data_link_loss_enabled) {
170                 status->failsafe = true;
171
172                 if (status->condition_global_position_valid &&
173 status->condition_home_position_valid) {
174                     status->nav_state =
175                         vehicle_status_s::NAVIGATION_STATE_AUTO_RTGS;
176                 } else if (status->condition_local_position_valid) {
177                     status->nav_state =
178                         vehicle_status_s::NAVIGATION_STATE_AUTO_LAND;
```

```

179         } else if (status->condition_local_altitude_valid) {
180             status->nav_state =
181                 vehicle_status_s::NAVIGATION_STATE_DESCEND;
182         } else {
183             status->nav_state =
184                 vehicle_status_s::NAVIGATION_STATE_TERMINATION;
185         }
186
187         /* go into failsafe if RC is lost and datalink loss is not set up */
188         } else if (status->rc_signal_lost && !data_link_loss_enabled) {
189             status->failsafe = true;
190
191             if (status->condition_global_position_valid &&
192 status->condition_home_position_valid) {
193                 status->nav_state =
194                     vehicle_status_s::NAVIGATION_STATE_AUTO_RTGS;
195             } else if (status->condition_local_position_valid) {
196                 status->nav_state =
197                     vehicle_status_s::NAVIGATION_STATE_AUTO_LAND;
198             } else if (status->condition_local_altitude_valid) {
199                 status->nav_state =
200                     vehicle_status_s::NAVIGATION_STATE_DESCEND;
201             } else {
202                 status->nav_state =

```

```
203         vehicle_status_s::NAVIGATION_STATE_TERMINATION;
204     }
205
206     /* don't bother if RC is lost if datalink is connected */
207     } else if (status->rc_signal_lost) {
208
209         /* this mode is ok, we don't need RC for loitering */
210         status->nav_state =
211             vehicle_status_s::NAVIGATION_STATE_AUTO_LOITER;
212     } else {
213         /* everything is perfect */
214         status->nav_state =
215             vehicle_status_s::NAVIGATION_STATE_AUTO_LOITER;
216     }
217     break;
218
219     case vehicle_status_s::MAIN_STATE_AUTO_RTL:
220         /* require global position and home, also go into failsafe on an engine failure
221         */
222
223         if (status->engine_failure) {
224             status->nav_state =
225                 vehicle_status_s::NAVIGATION_STATE_AUTO_LANDENGFAIL;
226         } else if ((!status->condition_global_position_valid ||
```

```
227         !status->condition_home_position_valid)) {
228             status->failsafe = true;
229
230             if (status->condition_local_position_valid) {
231                 status->nav_state =
232                     vehicle_status_s::NAVIGATION_STATE_AUTO_LAND;
233             } else if (status->condition_local_altitude_valid) {
234                 status->nav_state =
235                     vehicle_status_s::NAVIGATION_STATE_DESCEND;
236             } else {
237                 status->nav_state =
238                     vehicle_status_s::NAVIGATION_STATE_TERMINATION;
239             }
240         } else {
241             status->nav_state = vehicle_status_s::NAVIGATION_STATE_AUTO_RTL;
242         }
243         break;
244
245     case vehicle_status_s::MAIN_STATE_AUTO_FOLLOW_TARGET:
246         /* require global position and home */
247
248         if (status->engine_failure) {
249             status->nav_state =
250                 vehicle_status_s::NAVIGATION_STATE_AUTO_LANDENGFAIL;
```



```
251     } else if (!status->condition_global_position_valid) {
252         status->failsafe = true;
253
254         if (status->condition_local_position_valid) {
255             status->nav_state =
256                 vehicle_status_s::NAVIGATION_STATE_AUTO_LAND;
257         } else if (status->condition_local_altitude_valid) {
258             status->nav_state =
259                 vehicle_status_s::NAVIGATION_STATE_DESCEND;
260         } else {
261             status->nav_state =
262                 vehicle_status_s::NAVIGATION_STATE_TERMINATION;
263         }
264     } else {
265         status->nav_state =
266             vehicle_status_s::NAVIGATION_STATE_AUTO_FOLLOW_TARGET;
267     }
268     break;
269
270     case vehicle_status_s::MAIN_STATE_AUTO_TAKEOFF:
271         /* require global position and home */
272
273         if (status->engine_failure) {
274             status->nav_state =
```

```
275         vehicle_status_s::NAVIGATION_STATE_AUTO_LANDENGFAIL;
276     } else if ((!status->condition_global_position_valid ||
277               !status->condition_home_position_valid)) {
278         status->failsafe = true;
279
280         if (status->condition_local_position_valid) {
281             status->nav_state =
282                 vehicle_status_s::NAVIGATION_STATE_AUTO_LAND;
283         } else if (status->condition_local_altitude_valid) {
284             status->nav_state =
285                 vehicle_status_s::NAVIGATION_STATE_DESCEND;
286         } else {
287             status->nav_state =
288                 vehicle_status_s::NAVIGATION_STATE_TERMINATION;
289         }
290     } else {
291         status->nav_state =
292             vehicle_status_s::NAVIGATION_STATE_AUTO_TAKEOFF;
293     }
294     break;
295
296 case vehicle_status_s::MAIN_STATE_AUTO_LAND:
297     /* require global position and home */
298
```

```
299     if (status->engine_failure) {
300         status->nav_state =
301             vehicle_status_s::NAVIGATION_STATE_AUTO_LANDENGFAIL;
302     } else if ((!status->condition_global_position_valid ||
303         !status->condition_home_position_valid)) {
304         status->failsafe = true;
305
306         if (status->condition_local_altitude_valid) {
307             status->nav_state =
308                 vehicle_status_s::NAVIGATION_STATE_DESCEND;
309         } else {
310             status->nav_state =
311                 vehicle_status_s::NAVIGATION_STATE_TERMINATION;
312         }
313     } else {
314         status->nav_state = vehicle_status_s::NAVIGATION_STATE_AUTO_LAND;
315     }
316     break;
317
318 case vehicle_status_s::MAIN_STATE_OFFBOARD:
319     /* require offboard control, otherwise stay where you are */
320     if (status->offboard_control_signal_lost && !status->rc_signal_lost) {
321         status->failsafe = true;
322     }
```

```
323         status->nav_state = vehicle_status_s::NAVIGATION_STATE_POSCTL;
324     } else if (status->offboard_control_signal_lost && status->rc_signal_lost) {
325         status->failsafe = true;
326
327         if (status->condition_local_position_valid) {
328             status->nav_state =
329                 vehicle_status_s::NAVIGATION_STATE_AUTO_LAND;
330         } else if (status->condition_local_altitude_valid) {
331             status->nav_state =
332                 vehicle_status_s::NAVIGATION_STATE_DESCEND;
333         } else {
334             status->nav_state =
335                 vehicle_status_s::NAVIGATION_STATE_TERMINATION;
336         }
337     } else {
338         status->nav_state = vehicle_status_s::NAVIGATION_STATE_OFFBOARD;
339     }
340     default:
341         break;
342 }
343
344 return status->nav_state != nav_state_old;
```

set_control_mode() 根据 status.nav_state 确定控制模式

PX4Firmware/src/modules/commander/commander.cpp

```
1 void set_control_mode()
2 {
3     /* set vehicle_control_mode according to set_navigation_state */
4     control_mode.flag_armed = armed.armed;
5     control_mode.flag_external_manual_override_ok = (!status.is_rotary_wing
6                                                     && !status.is_vtol);
7     control_mode.flag_system_hil_enabled = status.hil_state ==
8                                           vehicle_status_s::HIL_STATE_ON;
9     control_mode.flag_control_offboard_enabled = false;
10
11     switch (status.nav_state) {
12     case vehicle_status_s::NAVIGATION_STATE_MANUAL:
13         control_mode.flag_control_manual_enabled = true;
14         control_mode.flag_control_auto_enabled = false;
15         control_mode.flag_control_rates_enabled = stabilization_required();
16         control_mode.flag_control_attitude_enabled =
17                                           stabilization_required();
18         control_mode.flag_control_altitude_enabled = false;
19         control_mode.flag_control_climb_rate_enabled = false;
20         control_mode.flag_control_position_enabled = false;
21         control_mode.flag_control_velocity_enabled = false;
22         control_mode.flag_control_termination_enabled = false;
23         break;
```

```
24
25     case vehicle_status_s::NAVIGATION_STATE_STAB:
26         control_mode.flag_control_manual_enabled = true;
27         control_mode.flag_control_auto_enabled = false;
28         control_mode.flag_control_rates_enabled = true;
29         control_mode.flag_control_attitude_enabled = true;
30         control_mode.flag_control_altitude_enabled = false;
31         control_mode.flag_control_climb_rate_enabled = false;
32         control_mode.flag_control_position_enabled = false;
33         control_mode.flag_control_velocity_enabled = false;
34         control_mode.flag_control_termination_enabled = false;
35         /* override is not ok in stabilized mode */
36         control_mode.flag_external_manual_override_ok = false;
37         break;
38
39     case vehicle_status_s::NAVIGATION_STATE_RATTITUDE:
40         control_mode.flag_control_manual_enabled = true;
41         control_mode.flag_control_auto_enabled = false;
42         control_mode.flag_control_rates_enabled = true;
43         control_mode.flag_control_attitude_enabled = true;
44         control_mode.flag_control_altitude_enabled = false;
45         control_mode.flag_control_climb_rate_enabled = false;
46         control_mode.flag_control_position_enabled = false;
47         control_mode.flag_control_velocity_enabled = false;
```

```
48     control_mode.flag_control_termination_enabled = false;
49     break;
50
51     case vehicle_status_s::NAVIGATION_STATE_ALTCTL:
52         control_mode.flag_control_manual_enabled = true;
53         control_mode.flag_control_auto_enabled = false;
54         control_mode.flag_control_rates_enabled = true;
55         control_mode.flag_control_attitude_enabled = true;
56         control_mode.flag_control_altitude_enabled = true;
57         control_mode.flag_control_climb_rate_enabled = true;
58         control_mode.flag_control_position_enabled = false;
59         control_mode.flag_control_velocity_enabled = false;
60         control_mode.flag_control_termination_enabled = false;
61         break;
62
63     case vehicle_status_s::NAVIGATION_STATE_POSCTL:
64         control_mode.flag_control_manual_enabled = true;
65         control_mode.flag_control_auto_enabled = false;
66         control_mode.flag_control_rates_enabled = true;
67         control_mode.flag_control_attitude_enabled = true;
68         control_mode.flag_control_altitude_enabled = true;
69         control_mode.flag_control_climb_rate_enabled = true;
70         control_mode.flag_control_position_enabled =
71                                     !status.in_transition_mode;
```

```
72     control_mode.flag_control_velocity_enabled =
73
74                                     !status.in_transition_mode;
75
76     control_mode.flag_control_termination_enabled = false;
77
78     break;
79
80 case vehicle_status_s::NAVIGATION_STATE_AUTO_RTL:
81
82 case vehicle_status_s::NAVIGATION_STATE_AUTO_RCRECOVER:
83
84     /* override is not ok for the RTL and recovery mode */
85
86     control_mode.flag_external_manual_override_ok = false;
87
88     /* fallback */
89
90 case vehicle_status_s::NAVIGATION_STATE_AUTO_FOLLOW_TARGET:
91
92 case vehicle_status_s::NAVIGATION_STATE_AUTO_RTGS:
93
94 case vehicle_status_s::NAVIGATION_STATE_AUTO_LAND:
95
96 case vehicle_status_s::NAVIGATION_STATE_AUTO_LANDENGFAIL:
97
98 case vehicle_status_s::NAVIGATION_STATE_AUTO_MISSION:
99
100 case vehicle_status_s::NAVIGATION_STATE_AUTO_LOITER:
101
102 case vehicle_status_s::NAVIGATION_STATE_AUTO_TAKEOFF:
103
104     control_mode.flag_control_manual_enabled = false;
105
106     control_mode.flag_control_auto_enabled = true;
107
108     control_mode.flag_control_rates_enabled = true;
109
110     control_mode.flag_control_attitude_enabled = true;
111
112     control_mode.flag_control_altitude_enabled = true;
113
114     control_mode.flag_control_climb_rate_enabled = true;
115
116     control_mode.flag_control_position_enabled =
```



```
96                                     !status.in_transition_mode;
97     control_mode.flag_control_velocity_enabled =
98                                     !status.in_transition_mode;
99     control_mode.flag_control_termination_enabled = false;
100     break;
101
102     case vehicle_status_s::NAVIGATION_STATE_AUTO_LANDGPSFAIL:
103         control_mode.flag_control_manual_enabled = false;
104         control_mode.flag_control_auto_enabled = false;
105         control_mode.flag_control_rates_enabled = true;
106         control_mode.flag_control_attitude_enabled = true;
107         control_mode.flag_control_altitude_enabled = false;
108         control_mode.flag_control_climb_rate_enabled = true;
109         control_mode.flag_control_position_enabled = false;
110         control_mode.flag_control_velocity_enabled = false;
111         control_mode.flag_control_termination_enabled = false;
112         break;
113
114     case vehicle_status_s::NAVIGATION_STATE_ACRO:
115         control_mode.flag_control_manual_enabled = true;
116         control_mode.flag_control_auto_enabled = false;
117         control_mode.flag_control_rates_enabled = true;
118         control_mode.flag_control_attitude_enabled = false;
119         control_mode.flag_control_altitude_enabled = false;
```

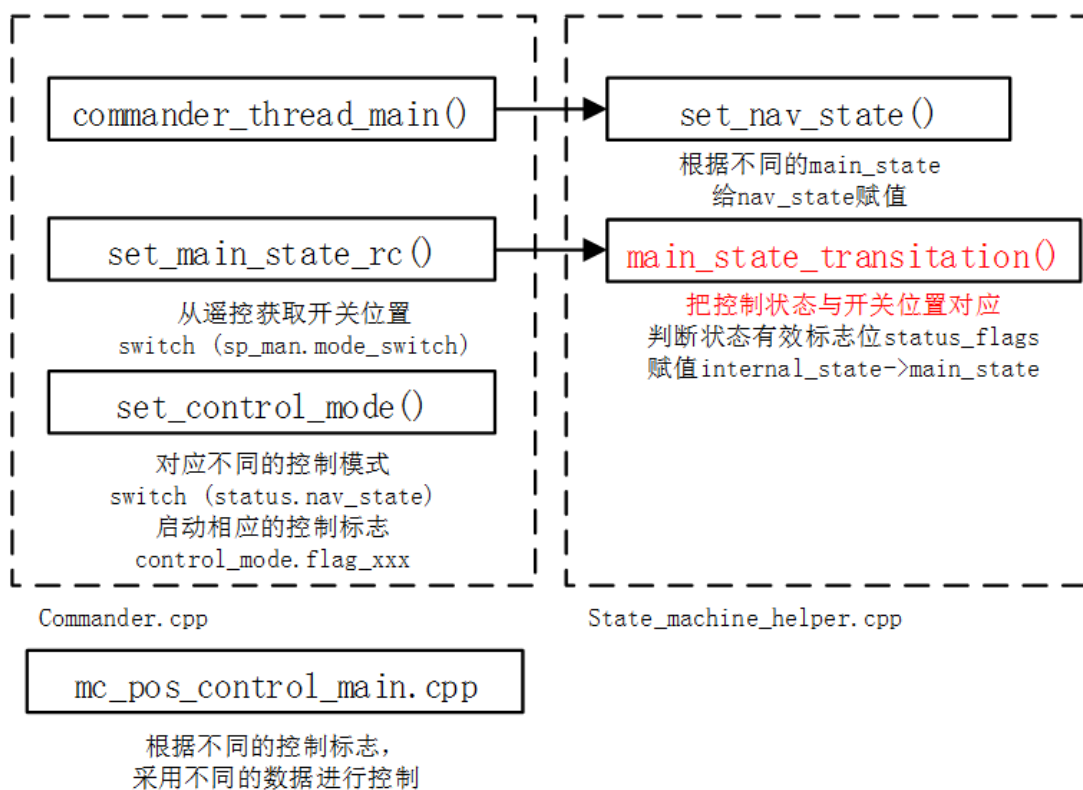
```
120     control_mode.flag_control_climb_rate_enabled = false;
121     control_mode.flag_control_position_enabled = false;
122     control_mode.flag_control_velocity_enabled = false;
123     control_mode.flag_control_termination_enabled = false;
124     break;
125
126 case vehicle_status_s::NAVIGATION_STATE_DESCEND:
127     /* TODO: check if this makes sense */
128     control_mode.flag_control_manual_enabled = false;
129     control_mode.flag_control_auto_enabled = true;
130     control_mode.flag_control_rates_enabled = true;
131     control_mode.flag_control_attitude_enabled = true;
132     control_mode.flag_control_position_enabled = false;
133     control_mode.flag_control_velocity_enabled = false;
134     control_mode.flag_control_altitude_enabled = false;
135     control_mode.flag_control_climb_rate_enabled = true;
136     control_mode.flag_control_termination_enabled = false;
137     break;
138
139 case vehicle_status_s::NAVIGATION_STATE_TERMINATION:
140     /* disable all controllers on termination */
141     control_mode.flag_control_manual_enabled = false;
142     control_mode.flag_control_auto_enabled = false;
143     control_mode.flag_control_rates_enabled = false;
```

```
144     control_mode.flag_control_attitude_enabled = false;
145     control_mode.flag_control_position_enabled = false;
146     control_mode.flag_control_velocity_enabled = false;
147     control_mode.flag_control_altitude_enabled = false;
148     control_mode.flag_control_climb_rate_enabled = false;
149     control_mode.flag_control_termination_enabled = true;
150     break;
151
152     case vehicle_status_s::NAVIGATION_STATE_OFFBOARD:
153         control_mode.flag_control_manual_enabled = false;
154         control_mode.flag_control_auto_enabled = false;
155         control_mode.flag_control_offboard_enabled = true;
156
157         /*
158          * The control flags depend on what is ignored according to the offboard control
159 mode topic
160          * Inner loop flags (e.g. attitude) also depend on outer loop ignore flags (e.g.
161 position)
162          */
163         control_mode.flag_control_rates_enabled =
164             !offboard_control_mode.ignore_bodyrate ||
165             !offboard_control_mode.ignore_attitude ||
166             !offboard_control_mode.ignore_position ||
167             !offboard_control_mode.ignore_velocity ||
168             !offboard_control_mode.ignore_acceleration_force;
```

```
169
170     control_mode.flag_control_attitude_enabled =
171
172         !offboard_control_mode.ignore_attitude ||
173
174         !offboard_control_mode.ignore_position ||
175
176         !offboard_control_mode.ignore_velocity ||
177
178         !offboard_control_mode.ignore_acceleration_force;
179
180
181     control_mode.flag_control_velocity_enabled =
182
183         (!offboard_control_mode.ignore_velocity ||
184
185         !offboard_control_mode.ignore_position)
186
187         && !status.in_transition_mode;
188
189
190     control_mode.flag_control_climb_rate_enabled =
191
192         !offboard_control_mode.ignore_velocity ||
193
194         !offboard_control_mode.ignore_position;
195
196
197     control_mode.flag_control_position_enabled =
198
199         !offboard_control_mode.ignore_position &&
200
201         !status.in_transition_mode;
202
203
204     control_mode.flag_control_altitude_enabled =
205
206         !offboard_control_mode.ignore_velocity ||
207
208         !offboard_control_mode.ignore_position;
```

```

193     break;
194
195     default:
196         break;
197 }
198 }
    
```



现在再回头看这幅流程图，开关位置，飞行模式，控制模式的切换应该会有进一步的理解。

第七章 Demo 程序讲解

7.1、Demo 程序功能介绍

Demo 程序实现的功能是无人机上电初始化完成后，按下控制板上的一键启动按钮后，飞机自动解锁起飞，当飞机爬升到 70cm~100cm 范围内时飞机向前飞行，当飞行至前方 2m 的距离时飞机悬停 5s，悬停结束后继续向前飞行，飞行至距前方 1m 处时，飞机自动降落。

7.2、Demo 程序通信协议介绍

飞控与超声波控制板之间通过串口进行通信，通信的波特率为 38400，飞控与超声波控制板之间的通信协议如下：

参数请求： 0x51 0x52 0x53 0x0D	控制板 → 飞控
------------------------------------	----------

参数传递： 0x54 XXXX XXXX 0x0D
头 --19 个参数，共 38 个字节-- 尾 飞控 → 控制板

19 个参数为：

前超声波的动作距离(当前方超声波探测距离小于此距离时开始动作)

后超声波的动作距离；

左超声波的动作距离；

右超声波的动作距离；

期望飞机爬升的高度；

前后控制量的最大值；

前后控制量的最小值；

前后控制量的计算系数；

左右控制量的最大值；

左右控制量的最小值；

左右控制量的计算系数；

高度控制量的最大值；

高度控制量的最小值；

高度控制量的计算系数；

飞机悬停时间（单位：秒）；

悬停时据前方的距离；

降落时据前方的距离；

飞向悬停距离的控制量；

飞向降落距离的控制量。

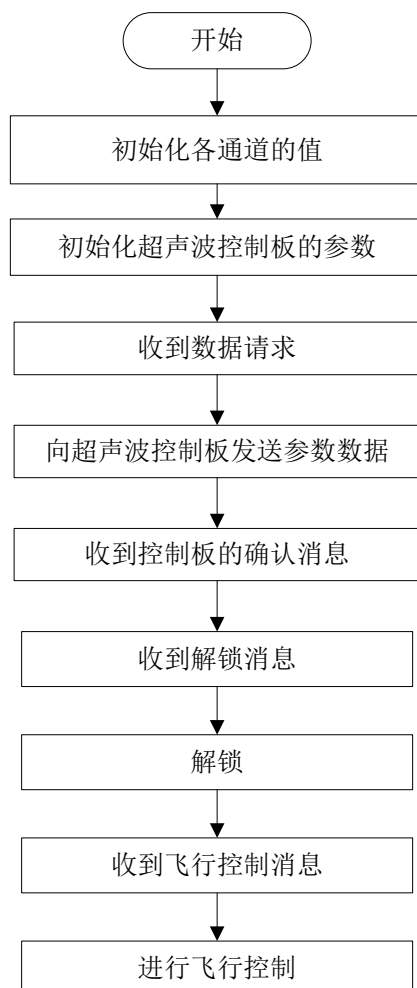
参数确认： 0x50 0x53 0x0D 控制板 → 飞控

解锁： 0x55 xxxx xxxx 0x0D
头 -油门- -YAW- 尾 控制板 → 飞控

控制： 0x52 xxxx xxxx xxxx xxxx 0x0D
头 前后控制量 左右控制量 油门控制量 模式 尾
控制板 → 飞控

7.3、Demo 程序分析

7.3.1、工作流程



飞控上电后进行控制通道和参数的初始化，然后每 20 毫秒执行一次 ReadExtData_loop() 函数，当 Send_End_Flage 为 1 时，飞控向控制板请求数据，当收到控制板的参数请求消息时，将 Send_End_Flage 置为 0，即不再执行 ReadExtData_loop() 函数，参数发送函数 SendData2Board_loop() 每 1 秒执行一次，当 Send_End_Flag 为 0 时，发送参数数据，当收到控制板的确认消息时，将 Send_End_Flage 置为 1，参数发送完毕，执行 ReadExtData_loop() 函数控制飞机飞行。

7.3.2、参数设置

Demo 程序的 19 个参数可以通过修改 `static void init_parameter()` 函数中各个参数的值来进行设置。

ardupilot/ArduCopter/ArduCopter.pde

```
1 static void init_parameter()
2 {
3     //期望距离：前后左右取值范围[100, 300]
4     //高取值范围[50, -130]
5     //单位：cm
6     Expectation_cm[SONAR_F] = 200;           //前超声波的动作距离
7     Expectation_cm[SONAR_B] = 200;           //后超声波的动作距离
8     Expectation_cm[SONAR_L] = 100;           //左超声波的动作距离
9     Expectation_cm[SONAR_R] = 100;           //右超声波的动作距离
10    Expectation_cm[SONAR_Alt] = 70;           //期望飞机爬升的高度
11
12    Control[CONTROL_FB].Control_limit_max = 450; //前后控制量的最大值，
13                                                    //取值范围[0, 550]
14    Control[CONTROL_FB].Control_limit_min = -450; //前后控制量的最大值，
15                                                    //取值范围[-550, 0]
16    Control[CONTROL_FB].Control_ratio = 5;       //前后控制量的计算系数，
17                                                    //取值范围[1, 15]
18
19    Control[CONTROL_LR].Control_limit_max = 450; //左右控制量的最大值，
20                                                    //取值范围[0, 550]
```

```

21     Control[CONTROL_LR].Control_limit_min = -450;      //左右控制量的最小值,
22                                                     //取值范围[-550, 0]
23     Control[CONTROL_LR].Control_ratio = 5;             //左右控制量的计算系数,
24                                                     //取值范围[1, 15]
25
26     Control[CONTROL_UD].Control_limit_max = 150;       //高度控制量的最大值,
27                                                     //取值范围[0, 160]
28     Control[CONTROL_UD].Control_limit_min = -130;      //高度控制量的最小值,
29                                                     //取值范围[-160, 0]
30     Control[CONTROL_UD].Control_ratio = 6;             //高度控制量的计算系数,
31                                                     //取值范围[1, 15]
32
33     FlyparameterItem.Hold_time = 5;                    //飞机悬停时间（单位：秒），//
34                                                     //取值范围[1, 20]
35     FlyparameterItem.Hold_distance = 220;              //悬停时据前方的距离,
36                                                     //取值范围[151, 300]
37     FlyparameterItem.Land_distance = 120;              //降落时据前方的距离,
38                                                     //取值范围[100, 150]
39     FlyparameterItem.To_hold_control_value = -110;     //飞向悬停距离的控制量,
40                                                     //取值范围[-200, -50]
41     FlyparameterItem.To_land_control_value = -110;     //飞向降落距离的控制量，//取值
42                                                     //范围[-200, -50]
43 }
  
```

控制量和控制系数的值越大，飞机反应越灵敏，但操控起来就越困难。

7.3.3、数据发送代码介绍

ardupilot/ArduCopter/ArduCopter.pde

```
1 static void SendData2Board_loop()
2 {
3     int16_t NewBytes = 0;
4     uint16_t i;
5     if(!Send_End_Flage)
6     {
7         hal. uartE->write(0x54);
8         for(i = SONAR_F; i < SONAR_NUM; i++)
9         {
10             hal. uartE->write((Expectation_cm[i] >> 8)&0xff);
11             hal. uartE->write(Expectation_cm[i]&0xff);
12         }
13
14         hal. uartE->write((Control[CONTROL_FB].Control_limit_max >> 8)&0xff);
15         hal. uartE->write(Control[CONTROL_FB].Control_limit_max&0xff);
16         hal. uartE->write((Control[CONTROL_FB].Control_limit_min >> 8)&0xff);
17         hal. uartE->write(Control[CONTROL_FB].Control_limit_min&0xff);
18         hal. uartE->write((Control[CONTROL_FB].Control_ratio >> 8)&0xff);
19         hal. uartE->write(Control[CONTROL_FB].Control_ratio&0xff);
20
```

```
21      hal. uartE->write((Control[CONTROL_LR].Control_limit_max >> 8)&0xff);
22      hal. uartE->write(Control[CONTROL_LR].Control_limit_max&0xff);
23      hal. uartE->write((Control[CONTROL_LR].Control_limit_min >> 8)&0xff);
24      hal. uartE->write(Control[CONTROL_LR].Control_limit_min&0xff);
25      hal. uartE->write((Control[CONTROL_LR].Control_ratio >> 8)&0xff);
26      hal. uartE->write(Control[CONTROL_LR].Control_ratio&0xff);
27
28      hal. uartE->write((Control[CONTROL_UD].Control_limit_max >> 8)&0xff);
29      hal. uartE->write(Control[CONTROL_UD].Control_limit_max&0xff);
30      hal. uartE->write((Control[CONTROL_UD].Control_limit_min >> 8)&0xff);
31      hal. uartE->write(Control[CONTROL_UD].Control_limit_min&0xff);
32      hal. uartE->write((Control[CONTROL_UD].Control_ratio >> 8)&0xff);
33      hal. uartE->write(Control[CONTROL_UD].Control_ratio&0xff);
34
35      hal. uartE->write((FlyparameterItem.Hold_time >> 8)&0xff);
36      hal. uartE->write(FlyparameterItem.Hold_time&0xff);
37
38      hal. uartE->write((FlyparameterItem.Hold_distance >> 8)&0xff);
39      hal. uartE->write(FlyparameterItem.Hold_distance&0xff);
40
41      hal. uartE->write((FlyparameterItem.Land_distance >> 8)&0xff);
42      hal. uartE->write(FlyparameterItem.Land_distance&0xff);
43
44      hal. uartE->write((FlyparameterItem.To_hold_control_value >>
```

```
45                                     8)&0xff);
46     hal. uartE->write(FlyparameterItem.To_hold_control_value&0xff);
47
48     hal. uartE->write((FlyparameterItem.To_land_control_value >>
49                                     8)&0xff);
50     hal. uartE->write(FlyparameterItem.To_land_control_value&0xff);
51
52     hal. uartE->write(0X0D);
53
54     if (hal. uartE == NULL)
55     {
56         return;
57     }
58
59     NewBytes = hal. uartE->available();
60     hal. uartD->printf(PSTR("NewBytes = %d\n\n"), NewBytes);
61     if(NewBytes >= 3)
62     {
63         if('P' == hal. uartE->read())
64         {
65             if('S' == hal. uartE->read())
66             {
67                 if(hal. uartE->read() == 0X0D)
68                 {
```

```
69         Send_End_Flage = 1;  
70     }  
71 }  
72 }  
73 }  
74 }  
75 }
```

当数据发送结束标志位 `Send_End_Flage` 为 0 时，飞控通过串口将 `init_parameter()` 中设置好的参数发送给超声波控制板，并读取串口的数据，当读取到控制板的确认消息时，将数据发送结束标志位 `Send_End_Flage` 置 1，即

不再执行发送函数 `SendData2Board_loop()`，转而执行数据接收函数 `ReadExtData_loop()`。

7.3.4、数据接收代码介绍

ardupilot/ArduCopter/ArduCopter.pde

```
1 static void ReadExtData_loop()  
2 {  
3     int16_t Sonar_NewBytes = 0;  
4     SonarPitchControl=0;  
5     SonarRollControl=0;  
6     SonarThrottleControl = 0;  
7     ControlMode = 0;  
8 }
```

```
9      if(Send_End_Flage)
10     {
11         hal. uartE->printf(PSTR("Y"));
12         if (hal. uartE == NULL)
13         {
14             return;
15         }
16
17         Sonar_NewBytes = hal. uartE->available();
18
19         hal. uartD->printf(PSTR("Sonar_NewBytes = %d\n\n"), Sonar_NewBytes);
20
21         if(Sonar_NewBytes > 0)
22         {
23             obstacle_packet[0] = hal. uartE->read();
24             unlock_packet[0] = obstacle_packet[0];
25             restet_flage = obstacle_packet[0];
26
27             if(obstacle_packet[0] == 'R')
28             {
29
30                 obstacle_packet[1] = hal. uartE->read();
31                 obstacle_packet[2] = hal. uartE->read();
32                 obstacle_packet[3] = hal. uartE->read();
```

```
33     obstacle_packet[4] = hal.uartE->read();
34
35     obstacle_packet[5] = hal.uartE->read();
36
37     obstacle_packet[6] = hal.uartE->read();
38
39     obstacle_packet[7] = hal.uartE->read();
40
41     obstacle_packet[8] = hal.uartE->read();
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56

if(hal.uartE->read() == 0x0d)
{
    SonarPitchControl = ((uint16_t)obstacle_packet[1] << 8) |
                        obstacle_packet[2];

    hal.uartD->printf(PSTR("op1 = %d, op2=%d\n"),
                    obstacle_packet[1], obstacle_packet[2]);

    hal.uartD->printf(PSTR("PitchControl = %d\n"),
                    SonarPitchControl);

    SonarRollControl = ((uint16_t)obstacle_packet[3] << 8) |
                      obstacle_packet[4];

    hal.uartD->printf(PSTR("or1 = %d, or2=%d\n"),
                    obstacle_packet[3], obstacle_packet[4]);

    hal.uartD->printf(PSTR("RollControl = %d\n"),
                    SonarRollControl);

    SonarThrottleControl = ((uint16_t)obstacle_packet[5] << 8)
                          | obstacle_packet[6];
```



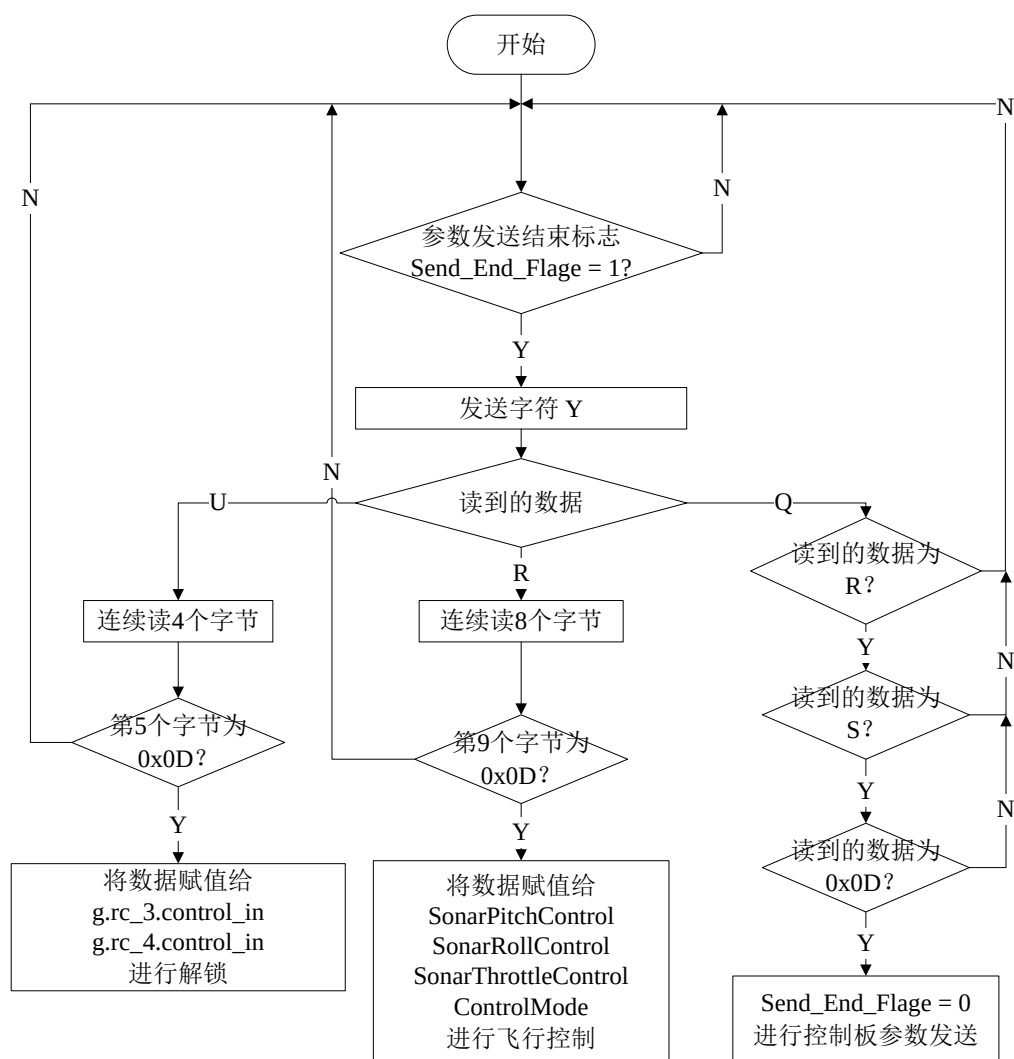
```
57         hal. uartD->printf (PSTR("ot1 = %d, ot2=%d\n"),
58
59             obstacle_packet[5], obstacle_packet[6]);
60
61         hal. uartD->printf (PSTR("ThrottleControl = %d\n"),
62
63             SonarThrottleControl);
64
65         ControlMode = ((uint16_t)obstacle_packet[7] << 8) |
66
67             obstacle_packet[8];
68
69         g. rc_5. set_pwm(1716+ControlMode);
70
71         hal. uartD->printf (PSTR("ControlMode = %d\n"),
72
73             ControlMode);
74
75     }
76
77 }
78
79 else if(unlock_packet[0] == 'U')
80
81 {
82
83     g. rc_5. set_pwm(1716);
84
85
86
87     unlock_packet[1] = hal. uartE->read();
88
89     unlock_packet[2] = hal. uartE->read();
90
91     unlock_packet[3] = hal. uartE->read();
92
93     unlock_packet[4] = hal. uartE->read();
94
95
96
97
98     if(hal. uartE->read() == 0x0d)
99
100 {
101
102     g. rc_3. control_in = ((uint16_t)unlock_packet[1] << 8) |
```

```
81         unlock_packet[2];
82
83         set_throttle_zero_flag(g.rc_3.control_in);
84
85         hal._uartD->printf(PSTR("g.rc_3.control_in = %d\n"),
86
87             g.rc_3.control_in);
88
89         hal._uartD->printf(PSTR("ap.throttle_zero = %d\n"),
90
91             ap.throttle_zero);
92
93
94         g.rc_4.control_in = ((uint16_t)unlock_packet[3] << 8) |
95
96             unlock_packet[4];
97
98         hal._uartD->printf(PSTR("g.rc_4.control_in = %d\n"),
99
100             g.rc_4.control_in);
101
102     }
103
104     }
105
106     else if(reset_flg == 'Q')
107     {
108         if('R' == hal._uartE->read())
109         {
110             if('S' == hal._uartE->read())
111             {
112                 if(0x0D == hal._uartE->read())
113                 {
114                     Send_End_Flage = 0;
115                 }
116             }
117         }
118     }
```

```
105         }  
106     }  
107     memset(obstacle_packet, '0', strlen(obstacle_packet));  
108     memset(unlock_packet, '0', strlen(unlock_packet));  
109 }  
110 }  
111 }
```

数据接收函数接收的数据分为 3 种：解锁数据，飞行控制数据，参数请求。
3 种不同的数据以不同的数据头标识：U 代表解锁数据，R 代表飞行控制数据，Q 代表参数请求数据。

ReadExtData_loop() 流程图：



ReadExtData_loop() 函数首先判断发送结束标志位 Send_End_Flage 是否为 1，若是 1，说明要进行数据的读取。先向控制板发送数据请求标志 Y，然后判断串口上是否有数据，若有数据且数据据头是否为 3 种数据的其中一种，则继续读取相应的字节然后判断是否到数据尾，若到数据尾则执行相应的操作。